



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

Ph.D. Dissertation of Engineering

Construction of Multi-layered Spatial Knowledge Graphs in LPG Databases: Integrating LLM-based Ontology Design and Graph Spatial Indexing

LPG 데이터베이스에서의 다층형 공간 지식
그래프 구축: LLM 기반 온톨로지 설계와 그래프
공간 인덱싱의 통합

February 2026

Seoul National University
Department of Civil and Environmental Engineering

Juyoung Kim

Construction of Multi-layered Spatial Knowledge Graphs in LPG Databases: Integrating LLM-based Ontology Design and Graph Spatial Indexing

Advisor Kiyun Yu

Submitting a Ph.D. Dissertation

October 2025

Graduate School of Civil and Environmental
Engineering
Seoul National University

Juyoung Kim

Confirming the Ph.D. Dissertation written by
Juyoung Kim
January 2026

Chair	_____	(Seal)
Vice Chair	_____	(Seal)
Examiner	_____	(Seal)
Examiner	_____	(Seal)
Examiner	_____	(Seal)

Abstract

This dissertation proposes the Multi-layered Spatial Knowledge Graph (MSKG), a comprehensive architecture designed to efficiently construct and utilize domain-specific spatial Knowledge Graphs (KGs) in Labeled Property Graph (LPG) environments. Conventional spatial KG construction has relied heavily on time-consuming manual processes and faced significant challenges in integrating spatial information. Furthermore, the absence of explicit ontological structures in LPG environments has limited the capacity for complex semantic reasoning. To address these issues, this research integrates three core layers—a Semantic Graph, a Data Graph, and a Spatial Index Graph—into a unified graph database.

The proposed MSKG architecture is established through a four-phase process: (1) analyzing KG requirements and extracting an initial LPG ontology through semi-automatic collaboration between Large Language Models (LLMs) and domain experts; (2) completing the LPG ontology by conducting graph completion, generating validating schema and inference rules via iterative LLM-expert interaction; (3) constructing Semantic and Data Graphs using three distinct LPG modeling approaches (Direct, Singleton, and Statement); and (4) finalizing the MSKG by building a Spatial Index Graph via SGIR-Tree indexing.

To validate the architecture's applicability, a housing exploration spatial KG was constructed using real-world data from Seoul, South Korea. Experimental results clarified the performance trade-offs among the LPG modeling approaches, leading to specific design guidelines: the Direct model is optimal for real-time interactive applications, the Statement model for metadata-rich domains, and the Singleton model for offline analytical workloads. Additionally, The SGIR-Tree achieves substantial query speedup by enabling unified optimization across spatial and graph operations. Semi-automatic methodology achieves high KG coverage of domain requirements.

This research makes four primary contributions: First, it defines

an LPG ontology and establishes a systematic methodology for integrating ontologies with instances in an LPG environment. Second, it demonstrates that high-quality KGs can be constructed efficiently through strategic task decomposition and semi-automatic LLM-expert collaboration. Third, it resolves the impedance mismatch between spatial operations and graph traversals through graph-based spatial indexing. Fourth, it presents a lightweight spatial KG structure that enables spatial-semantic querying by replacing pre-computed spatial relationships with on-demand computation. These contributions establish domain-independent methodologies for spatial KG construction in LPG environments, providing validated design guidelines applicable to any domain requiring integrated spatial-semantic reasoning. Future research directions include developing incremental update mechanisms for real-time applications, integrating native reasoning capabilities comparable to RDF-based systems, and validating cross-domain generalizability beyond the housing exploration case study.

Keyword : Multi-layered Spatial Knowledge Graph, Labeled Property Graph, Large Language Model, Ontology Engineering, Graph Spatial Indexing, Housing Exploration

Student Number : 2019-31529

Table of Contents

Abstract.....	ii
Chapter 1. Introduction.....	1
1.1. Study Background	1
1.2. Purpose of Research	5
Chapter 2. Related Works.....	9
2.1. Knowledge Graphs	9
2.2. LLM-based KG Construction	14
2.3. Spatial Knowledge Graphs	22
2.4. Spatial Indexing in Graph DBMSs.....	26
2.5. Summary	34
Chapter 3. Methodology.....	37
3.1. Preliminaries.....	37
3.2. Multi-Layered Spatial Knowledge Graph Architecture	40
3.3. Specification & Ontology Extraction	43
3.4. Ontology Completion	47
3.5. Semantic & Data Graph Construction.....	52
3.6. Spatial Index Graph Construction.....	58
Chapter 4. A Case Study of Housing Exploration	69
4.1. Application	69
4.2. Evaluation	80
4.3. Discussion.....	110
Chapter 5. Conclusions	115
Bibliography	118
Abstract (in Korean).....	124
Appendix	126

List of Tables

[Table 2–1] Comparison of RDF and LPG Data Models	1 1
[Table 2–2] LLM–based KG Construction Approaches: Basic Information and Pipeline Coverage	1 4
[Table 2–3] LLM–based KG Construction Approaches: Key Characteristics	1 6
[Table 2–4] Comparative Analysis of Spatial KGs	2 2
[Table 2–5] Taxonomy of Spatial Indexing Approaches in GDBMSs	3 0
[Table 2–6] Architectural Trade–offs in Spatial Graph Indexing ..	3 2
[Table 3–1] MSKG Subgraph–Phase Correspondence and Key Element Technologies	4 1
[Table 3–2] MBR computation and storage mechanism	6 0
[Table 3–3] KNN Query Algorithm	6 4
[Table 3–4] Range Query Algorithm	6 5
[Table 3–5] Spatial Join Algorithm.....	6 7
[Table 4–1] Evaluation Metrics and Their Application Sections.	7 9
[Table 4–2] Storage Comparison for LPG Modeling Approaches	8 2
[Table 4–3] KG Construction Time by LPG Models	8 3
[Table 4–4] Representative Cluster Queries.....	8 4
[Table 4–5] Spatial Index Storage and Construction Comparison..	8 8
[Table 4–6] Spatial Join Query.....	9 1
[Table 4–7] KNN Query	9 3
[Table 4–8] Spatial Range Query	9 6
[Table 4–9] Spatial Graph Queries for Comparison	9 9
[Table 4–10] Spatial Range Query for STR–Tree.....	1 0 5
[Table 4–11] Multi–Layer Query Characteristics and Execution Results.....	1 0 7
[Table A–1] Comparative Query Implementations Across LPG Models	1 2 6
[Table B–1] Multi–Layer Structure Queries	1 3 0
[Table B– 2] Comparison of CQs: This Dissertation vs. Hong (2025)	1 3 2
[Table B– 3] Real World User Queries	1 3 4

List of Figures

[Figure 2–1] Structural Comparison of RDF and LPG Models.....	1 2
[Figure 2–2] R–Tree illustration (Guttman, 1984).....	2 8
[Figure 2–3] Quad–Tree illustration (Finkel & Bentley, 1974) .	2 9
[Figure 3–1] Multi–layered Spatial Knowledge Graph Structure in GDBMS	3 8
[Figure 3–2] Implementation Workflow for MSKG Construction	4 2
[Figure 3–3] Phase1: Specification & Ontology Extraction pipeline	4 4
[Figure 3–4] Phase2: Ontology Completion pipeline	4 8
[Figure 3–5] Phase 3: KG Construction pipeline	5 3
[Figure 3–6] Direct Modeling	5 4
[Figure 3–7] Singleton Modeling	5 4
[Figure 3–8] Statement Modeling.....	5 5
[Figure 3–9] Phase 4: Spatial Indexing Pipeline.....	5 9
[Figure 4–1] Running Example Overview	7 0
[Figure 4–2] LLM Prompting Strategy for Concept Extraction...	7 5
[Figure 4–3] Query Execution Time Comparison Across LPG Models	8 5
[Figure 4–4] Spatial Join Query Time Performance	9 1
[Figure 4–5] Spatial Join Query Memory Overhead.....	9 2
[Figure 4–6] KNN Query Time Performance	9 4
[Figure 4–7] KNN Query Memory Overhead.....	9 5
[Figure 4–8] Spatial Range Query Performance.....	9 7
[Figure 4–9] Spatial Range Query Memory Overhead	9 8
[Figure 4–10] SGIR–Tree vs Disconnected Index Spatial Join Query Comparison.....	1 0 1
[Figure 4–11] SGIR–Tree vs Disconnected Index KNN Query Comparison.....	1 0 2
[Figure 4–12] SGIR–Tree vs Disconnected Index Range Query Comparison.....	1 0 3
[Figure 4–13] SGIR–Tree vs Out–DBMS Index Range Query Comparison.....	1 0 5
[Figure 4–14] OZ’s House Homepage on October 2, 2025.....	1 1 4

Chapter 1. Introduction

1.1. Study Background

Contemporary decision-making in complex domains increasingly requires simultaneous consideration of both spatial and semantic dimensions. Urban planning authorities must evaluate development proposals by analyzing accessibility to transportation networks, proximity to essential services, and compliance with zoning regulations (Sharma & Patil, 2024). Healthcare administrators optimize hospital-patient accessibility by considering geographic coverage, service capacity, and demographic distribution patterns (J. Pan et al., 2023). Transportation planners design multimodal networks by examining route connectivity, transfer convenience, and population catchment areas (Ye et al., 2021). Housing seekers evaluate residential options through multifaceted criteria encompassing neighborhood amenities, commute feasibility, school quality, and environmental conditions (MALCZEWSKI & JELOKHANI-NIARAKI, 2012).

These complex decision-making processes share three critical common requirements. First, they demand explicit representation of intricate domain knowledge. Each domain requires sophisticated knowledge structures encompassing hundreds of concept types, dozens of relationship types, and rich attribute dimensions, all of which must be systematically organized and expressed in reusable forms. Second, they require efficient processing of large-scale spatial data. Urban-scale applications must handle hundreds of thousands of spatial objects and their topological relationships, necessitating spatial indexing and efficient spatial operators. Third, they need real-time integrated spatial-semantic query processing capabilities. Queries such as "find apartments near good schools accessible by public transit" must simultaneously process semantic constraints, multi-hop graph traversals, and spatial predicates.

These requirements collectively motivate the need for spatial

knowledge graphs (KGs). KGs provide explicit knowledge representation through formally defined concepts, relationships, and constraints, enabling both human comprehension and machine reasoning (Hogan et al., 2022). Spatial KGs extend this foundation by incorporating geographic entities, spatial attributes, and topological relationships, addressing the critical gap that conventional KGs cannot adequately handle spatial queries essential to location-based decision-making (Karalis et al., 2019). The integration of semantic and spatial dimensions in a unified graph structure enables sophisticated query answering and inference capabilities that neither pure geographic information systems nor conventional knowledge bases can provide independently.

Two dominant data models shape contemporary spatial KG research: the Resource Description Framework (RDF) and the Labeled Property Graph (LPG). Each model presents distinct architectural characteristics that influence how spatial-semantic integration can be achieved. RDF has dominated spatial KG research through decades of standardization efforts and ecosystem maturity. The RDF triple-store paradigm enables definition of class hierarchies, property domains and ranges, and logical constraints through W3C-standardized ontology languages including RDFS and OWL, supporting automated reasoning (Hitzler et al., 2012). GeoSPARQL extends SPARQL with spatial operators for RDF data, providing standardized spatial querying aligned with the Open Geospatial Consortium's Simple Features specification (Battle & Kolas, 2012). Representative systems such as LinkedGeoData (Stadler et al., 2012), YAGO2Geo (Karalis et al., 2019), WorldKG (Dsouza et al., 2023), and KnowWhereGraph (R. Zhu et al., 2025) demonstrate RDF's broad adoption across diverse application domains. This mature ecosystem has established RDF as the de facto standard for semantic interoperability, federated queries, and formal ontology-driven reasoning.

However, RDF's triple-based structure introduces fundamental limitations for applications requiring property-rich entities and complex graph traversals. Each entity attribute requires a separate

triple, making representation verbose where property graphs could express content naturally. Multi-hop path queries often exhibit suboptimal performance in triple stores compared to native graph databases optimized for traversal operations (Hernández et al., 2016). Furthermore, while GeoSPARQL provides standard spatial operators, the computational overhead of on-the-fly spatial relationship evaluation without native spatial indexing can severely impact query response times for large-scale urban datasets (Huang et al., 2019).

These limitations have driven growing adoption of the LPG model in both industry and academia. Industrial adoption of LPG has been extensive. Leading Graph Database Management Systems (GDBMSs) including Neo4j, Amazon Neptune, TigerGraph, and JanusGraph have adopted the LPG model, demonstrating its practical viability for large-scale deployments (Patras et al., 2021). Recent ISO standardization efforts through GQL (Graph Query Language) further validate LPG's industrial significance (Angles et al., 2023; *ISO/IEC 39075*, 2024).

Academic recognition of LPG's practical advantages has grown substantially in recent years. Several domain-specific studies demonstrate LPG's effectiveness for knowledge representation across diverse applications. Di Pierro et al. (2023) survey LPG-based KGs across multiple domains, cataloging implementations that leverage property-rich structures for flexible knowledge modeling. Nguyen et al. (2019) propose the Singleton approach for relationship reification in Neo4j, enabling metadata attachment to relationships at the instance level. Domain-specific applications in urban knowledge systems (Y. Liu et al., 2023), Traffic Accident (Park, 2023), and Housing Exploration (Hong, 2025) collectively demonstrate LPG's growing adoption across diverse semantic knowledge representation tasks, reflecting increasing scholarly interest in property graph models for knowledge engineering.

The practical advantages of LPG manifest across multiple dimensions. Unlike RDF's requirement for separate triples per attribute, LPG allows nodes and edges to carry multiple key-value property pairs directly, enabling more intuitive and efficient

representation of property-rich entities. Edge properties prove especially valuable for representing relationship metadata such as timestamps, weights, and provenance information without requiring additional reification constructs that would be necessary in RDF. Furthermore, native graph storage optimized for traversal operations achieves superior performance for multi-hop relationship queries compared to relational databases or RDF triple stores (Lissandrini et al., 2018). Declarative query languages like Cypher further enhance this capability by supporting efficient pattern matching, making LPG particularly attractive for applications requiring complex graph traversals and real-time performance (Francis et al., 2018; J. Zhu et al., 2022). For spatial domains, LPG's ability to attach geometric properties directly to nodes and to execute pattern-matching queries efficiently provides substantial advantages over triple-store architectures. The intuitive graph structure naturally mirrors real-world topological relationships, facilitating both schema design and query formulation.

Despite these recognized advantages, systematic methodologies for ontology engineering in LPG environments remain critically underdeveloped. Unlike RDF, where the TBox-ABox separation enables automated reasoning through formally defined ontology languages, LPG databases lack explicit mechanisms for representing and enforcing ontological structures. This absence becomes increasingly problematic in the era of AI-powered applications. As retrieval-augmented generation (RAG) systems and question-answering applications expand, the ability to perform semantic reasoning over knowledge graphs becomes essential for accurate and contextual responses (S. Pan et al., 2024). RDF's mature ecosystem enables such reasoning through pre-defined ontological semantics, whereas LPG environments cannot support inference without explicit ontology representation.

Furthermore, integrating ontological structures within the same database instance provides significant practical advantages: unified query optimization across schema and instance layers, consistent maintenance without synchronization overhead between external

ontologies and internal data, and enhanced interoperability through self-contained knowledge representation. This dissertation addresses these critical gaps by developing comprehensive architecture for constructing domain-specific spatial KGs in LPG environments, validated through the housing exploration domain as a representative application case.

1.2. Purpose of Research

To address the limitations identified in Section 1.1, this dissertation proposes a multi-layered spatial knowledge graph (MSKG) architecture for LPG database environments. The MSKG integrates three distinct subgraphs within a unified graph structure: a Semantic Graph containing ontological definitions and concept hierarchies, a Data Graph comprising domain instances and their relationships, and a Spatial Index Graph enabling efficient spatial operations. This research focuses on establishing the core element technologies necessary for constructing each subgraph and achieving their seamless integration. Realizing this architecture requires addressing fundamental gaps in current methodologies, which the following subsections examine in detail.

1.2.1. Research Gaps

Current domain-specific spatial KG construction methodologies face four critical gaps. First, as established in Section 1.1, ontological structures in LPG environments are essential for semantic reasoning and AI-powered applications, yet unlike RDF's mature ecosystem with established ontology languages (RDFS, OWL) and accumulated design principles, LPG environments lack systematic ontology engineering methodologies. Existing work such as Di Pierro et al. (2023) surveys LPG-based KGs but provides no design methodology, while Nguyen et al. (2019) addresses only instance-level relationship reification without ontological design guidance. Critical questions remain unanswered: how should concept hierarchies be represented in LPG? How do different modeling

patterns impact semantic expressiveness and query performance? Without systematic foundations, LPG-based KG construction lacks principles for concept-to-structure mapping and schema constraint enforcement.

Second, manual KG construction remains prohibitively expensive despite recent LLM advances. While existing work demonstrates LLM capabilities for isolated subtasks (B. Zhang & Soh, 2024; H. Zhang et al., 2024; Y. Zhu et al., 2024), integrated methodologies orchestrating these capabilities across complete construction lifecycles are absent. Current approaches lack systematic decomposition strategies determining optimal task granularity, principled human-in-the-loop design specifying where expert validation maximizes quality-cost trade-offs, and structured data integration mechanisms for authoritative spatial databases essential for spatial KGs.

Third, graph databases exhibit fundamental architectural limitations in spatial data operations. Existing approaches adopt either external library-based indices or disconnected internal structures that maintain spatial indices separately from graph topology, creating performance bottlenecks through data marshaling overhead and inability to unify query optimization. The problem becomes critically acute for hybrid queries requiring tight integration of spatial operations with graph traversals, as existing architectures cannot efficiently optimize these combined operations.

Fourth, domain-specific spatial KGs face a fundamental trade-off between query flexibility and storage efficiency. On-the-fly spatial relationship computation minimizes storage but incurs substantial cost without sophisticated indexing, while pre-materialization causes storage explosion and limits query flexibility. In urban-scale applications, pre-computing all proximity relationships generates hundreds of millions of edges, most never queried, fundamentally conflicting with the exploratory nature of spatial decision-making where users iteratively refine spatial constraints.

1.2.2. Research Contributions

Based on these four research gaps, this dissertation makes the following primary contributions to the field of spatial KG construction.

First, we establish systematic methodologies for integrating ontological structures with instance data in unified LPG representations. While prior LPG implementations have implicitly or partially employed Direct, Singleton, and Statement patterns, this research explicitly formalizes these approaches through mathematical definitions and implements them systematically for comparative evaluation in spatial KG contexts. Through empirical performance analysis, we provide guidelines for LPG schema design decisions, enabling practitioners to select appropriate modeling approaches based on their application requirements.

Second, we present a comprehensive semi-automatic construction methodology that systematically integrates LLM-based ontology engineering with expert refinement across all ontology creation phases. This architecture provides systematic task decomposition strategies, principled human-in-the-loop integration points, and structured data extraction mechanisms, making domain-specific spatial KG development practically feasible while maintaining production-grade quality.

Third, we integrate spatial indices as native subgraphs within GDBMSs by representing R-Tree structures directly as graph elements. By converting R-Tree nodes and pointers into graph nodes and edges, our SGIR-Tree enables unified query optimization across spatial and graph operations, seamless index-to-entity traversal without ID mapping overhead, and efficient hybrid query execution.

Fourth, we introduce a lightweight architecture that eliminates pre-computed spatial relationship edges, replacing them with on-demand computation powered by spatial graph indexing. This approach resolves the fundamental trade-off between query flexibility and storage efficiency in existing spatial KG systems, enabling user-specified distance thresholds and arbitrary spatial predicates while maintaining practical query performance.

The remainder of this dissertation is organized as follows.

Chapter 2 reviews related work and identifies research gaps. Chapter 3 presents the proposed methodology. Chapter 4 reports experimental evaluation and results, demonstrating effectiveness through the housing exploration case study. Chapter 5 concludes the dissertation with a summary of contributions and future research directions.

Chapter 2. Related Works

This chapter reviews foundational research areas for constructing domain-specific spatial KGs. Section 2.1 introduces KG fundamentals and data models. Section 2.2 examines LLM-assisted KG construction. Section 2.3 explores spatial KGs. Section 2.4 discusses spatial indexing in graph databases. Section 2.5 synthesizes these streams and identifies research gaps.

2.1. Knowledge Graphs

KGs have emerged as a fundamental technology for representing and reasoning about structured knowledge in modern information systems (Hogan et al., 2022). This section traces the historical evolution of knowledge representation paradigms, establishes conceptual relationships among knowledge bases, KGs, and ontologies, and examines the two dominant data models that shape contemporary research and applications.

2.1.1. Historical Evolution and Conceptual Foundations

A KG is a structured representation of entities and their relationships, organized as a graph where nodes represent entities and edges represent relationships between them (Hogan et al., 2022). This graph-based approach has evolved through several distinct phases of knowledge representation research. The evolution began with early semantic networks and expert systems in the 1960s–1970s, which introduced the fundamental idea of representing knowledge as interconnected nodes and links (Di Pierro et al., 2023). These pioneering systems, though limited in scale and formalization, established the graph-based metaphor for knowledge that persists today.

The 1980s witnessed the emergence of frame-based knowledge representation systems, which introduced more sophisticated structuring mechanisms for organizing domain knowledge through

inheritance hierarchies and slot–filler representations (Chaudhri et al., 2022). These systems were grounded in the theoretical foundations of Knowledge Representation and Reasoning (KRR), a core discipline within Artificial Intelligence that formalized knowledge structures enabling intelligent systems to tackle sophisticated problem–solving tasks (Shortliffe, 2012). These advances provided the conceptual framework that would later materialize into practical knowledge management systems—knowledge bases, KGs, and ontologies—each serving distinct but complementary roles in organizing and utilizing structured information.

A knowledge base (KB) serves as the broadest concept: a centralized repository of information that can take various forms. KGs represent a more structured subset of KBs, specifically employing graph structures to capture entity–relationship patterns with explicit semantic meaning. Ontologies provide the formal specification layer defining concepts, relationships, and constraints that govern KG construction, establishing taxonomies, defining properties, and specifying logical rules for reasoning (Hitzler et al., 2012). In practical terms, a KG comprises two complementary layers: the ontological layer (schema/TBox) that defines the conceptual structure, and the instance layer (data/ABox) that populates this structure with concrete entities and their relationships. While a KB may contain unstructured or semi–structured information, a KG enforces graph–based structure with typed entities and relations, and an ontology provides the formal semantic framework underlying that structure (Di Pierro et al., 2023).

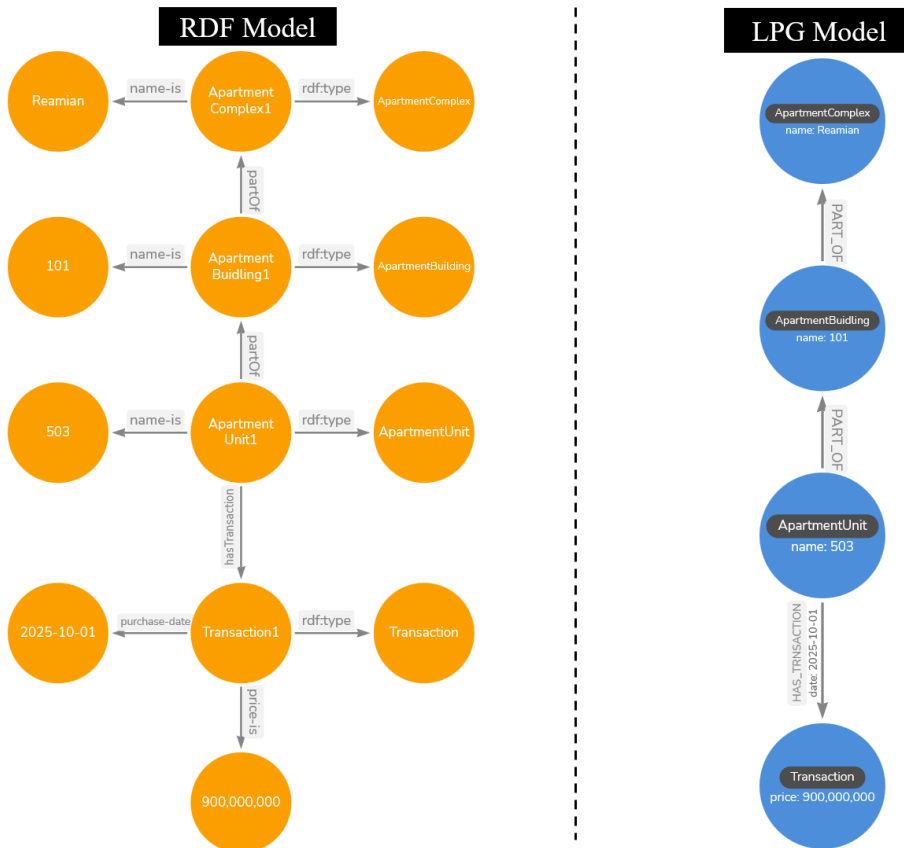
2.1.2. Comparative Analysis of RDF and LPG Models

As established in Chapter 1, the RDF model has dominated spatial KG research through its mature ecosystem and standardized reasoning capabilities (Section 1.1), while the LPG model has gained significant traction in both industry and academia due to its operational efficiency for graph traversal and intuitive representation of property–rich entities. Having introduced the fundamental

characteristics of both models, this section systematically compares their architectural trade-offs to contextualize the design choices underlying our architecture. Table 2-1 synthesizes the key dimensions along which RDF and LPG models diverge, highlighting the implications for spatial KG construction.

[Table 2-1] Comparison of RDF and LPG Data Models

Aspect	RDF	LPG
Data Model	Triple-based (subject-predicate-object)	Node-and-edge with properties and labels
Property Attachment	Properties as separate triples	Properties directly on nodes/edges
Schema Definition	RDFS, OWL for ontology	property constraints
Semantic Expressiveness	High (Description Logic, reasoning)	Medium (limited native reasoning)
Query Language	SPARQL	GQL, Cypher, Gremlin
Path Query Performance	Complex SPARQL, lower efficiency	Native support, high efficiency
Storage Efficiency	Verbose for complex structures	More compact representation
Reasoning Support	Native (OWL, RDFS)	Limited, requires external tools
Interoperability	High (Linked Data, federation)	Lower (proprietary implementations)
Primary Use Cases	Semantic Web, data integration	Graph analytics, path analysis
Major GDBMSs	Virtuoso, GraphDB, Stardog	Neo4j, TigerGraph, Neptune



[Figure 2–1] Structural Comparison of RDF and LPG Models

The choice between RDF and LPG models involves fundamental trade-offs that impact system design and performance characteristics. Figure 2–1 illustrates these structural differences using a concrete apartment transaction example. The RDF model requires 13 nodes and 12 edges to represent information that LPG captures with only 4 nodes and 3 edges. Critically, the transaction date must be stored as a separate literal node in RDF, whereas LPG allows it to be attached directly to the HAS_TRANSACTION edge as a property. This native support for edge properties makes LPG particularly suitable for domains like housing exploration, where relationships frequently carry temporal or spatial metadata.

RDF excels in scenarios requiring semantic interoperability, federated queries across heterogeneous data sources, and automated reasoning based on formal ontologies. The model's foundation in Web

standards facilitates data publishing and integration across organizational boundaries. However, this semantic richness comes at the cost of increased storage requirements and reduced performance for certain graph operations. Conversely, LPG models optimize for operational efficiency in graph traversal and pattern matching, making them particularly suitable for applications requiring real-time query responses over complex network structures. The ability to attach properties directly to edges proves especially valuable for representing relationship metadata—timestamps, weights, confidences—without requiring additional triples or reification constructs. However, the lack of standardized reasoning capabilities and the proprietary nature of many LPG implementations create challenges for knowledge integration across systems.

Recognizing the complementary strengths of RDF and LPG, researchers have explored hybrid approaches to bridge these paradigms, including compatibility layers (Di Pierro et al., 2023; Schmidt et al., 2024) and the Singleton approach for relationship reification (Nguyen et al., 2019). However, while these LPG-based approaches demonstrate the flexibility of property graph models, they lack systematic methodologies for ontology-driven KG construction. Nguyen et al.'s Singleton approach focuses primarily on instance-level representation in Neo4j, providing a mechanism for attaching metadata to relationships but offering no guidance on how to design ontologies specifically for LPG environments or how to systematically transform semantic schemas into graph structures. Similarly, while Di Pierro et al.'s survey comprehensively catalogs existing LPG-based KGs, it does not propose design principles or methodologies for creating new ontologies optimized for property graph databases. The fundamental question of how to systematically design and implement ontologies that leverage LPG's native capabilities—such as property-rich edges, flexible labeling, and efficient pattern matching—while maintaining semantic rigor remains largely unaddressed in the literature.

For spatial KGs in domains, these trade-offs become particularly salient. The requirement for efficient multi-hop traversals combined

with complex spatial predicates favors LPG's performance characteristics, while the need for ontological rigor and semantic validation motivates systematic methodology development—a research gap addressed in this dissertation.

2.2. LLM-based KG Construction

The emergence of LLMs presents new opportunities for automating aspects of KG construction, leveraging their extensive pre-trained knowledge and natural language understanding capabilities (Hofer et al., 2024; Y. Zhu et al., 2024). Recent research explores LLM-assisted approaches for various KG engineering subtasks, ranging from ontology extraction to instance population. However, existing LLM-based approaches exhibit significant heterogeneity in their construction strategies, target graph models, and pipeline coverage, necessitating systematic analysis to identify gaps and opportunities for advancement. Tables 2-2 and 2-3 provide a comprehensive comparison of surveyed LLM-based KG construction approaches: Table 2-2 summarizes basic information, input data sources, and pipeline task coverage based on Hofer et al.'s (2024) framework, while Table 2-3 presents key strengths and limitations of each approach.

[Table 2-2] LLM-based KG Construction Approaches: Basic Information and Pipeline Coverage

Approach	Strategy	Graph Model	Construction Scope	Input Data Source	Pipeline Coverage	Reference
EDC	Prompting	RDF	Instance only	Unstructured text	Prepo, OM _o , KE _• , ER _o	(B. Zhang & Soh, 2024)
Feng et al.	Prompting	RDF/O WL	Ontology +	Knowledge base	Init _• Prepo,	(Feng et al.,

			Instance	docs	OM●, KE●	(2024)
OntoChat	Prompting	RDF/O WL	Ontology only	Expert interaction	Init●	(B. Zhang et al., 2024)
Kommini et al.	Prompting	RDF/O WL	Ontology + Instance	Literature	Init●, Prepo, OM●, KE○	(Kommini et al., 2024)
OntoKGen	Prompting	LPG	Ontology + Instance	Technical documents	Init●, Prepo, KE●, QA○	(Abolhasani & Pan, 2024)
Yang et al.	Prompting	RDF/O WL	Ontology + Instance	Medical literature	Init○, Prepo, KE●	(Yang et al., 2024)
AutoKG	Prompting	Simplified	Instance only	Text blocks	Prepo, KE●, ER○	(Chen & Bertozzi, 2023)
AIO	Prompting	RDF/O WL	Ontology only	AI/ML literature	OM●, QA○	(Joachim iak et al., 2024)
SAKA	Fine-tuning	LPG	Ontology + Instance	Audio + JSON	Prepo, KE●	(H. Zhang et al., 2024)
UrbanKGent	Fine-tuning	Not specified	Instance only	Urban text + spatial data	Init○, Prepo, KE●, KC●	(Ning & Liu, 2024)

Pipeline Coverage Codes: Init = KG Initialization; Prep = Data Preprocessing; OM = Ontology Management; KE = Knowledge Extraction; ER = Entity Resolution/Fusion; QA = Quality Assurance; KC = Knowledge Completion; Incr = Incremental Integration

Legend: ● = Strong LLM support; ○ = Weak/partial LLM support; Tasks not listed = Not LLM supported

[Table 2–3] LLM–based KG Construction Approaches: Key Characteristics

Approach	Key Strengths	Key Limitations
EDC	Schema Alignment; natural language relation definitions	No ontology creation and managing; no quality assurance; high hallucination risk; limited to text
Feng et al.	Wikidata alignment; formal OWL ontologies; semantic interoperability	Constrained to Wikidata; no ER/EF; limited to text
OntoChat	Conversational interface; iterative expert refinement	No instance population; requires extensive expert involvement; LLM usage for only CQs
Kommineni et al.	Bootstrap from expert knowledge; end–to–end pipeline	Remain initial Ontology; no ER/EF
OntoKGen	Adaptive Chain–of–Thought; domain–specific engineering focus; LPG model	LPG for instance; no ontology de facto schema
Yang et al.	Generate CQs; systematic knowledge scope discovery	Limited LLM usage for CQ, instance creation; partial completeness (3/5)
AutoKG	Graph Laplace learning; lightweight KG; RAG–optimized	No formal ontology; No Graph Model for KG; limited to retrieval use cases
AIO	Focusing on Ontology Management; BioPortal integration	No instance population
SAKA	Full KB construction and KBQA pipeline; multi–version management; LPG	Audio/dialogue input only; Manual Ontology creation and Management
UrbanKGent	Utilizing GPT–4’s CoT output for Llama fine–tuning; external geospatial	Extract ontology elements and instances simultaneously; low quality

	tools for GPT	ontology by redundancy
--	---------------	------------------------

2.2.1. LLM Strategies and Construction Scope

Analysis of surveyed approaches reveals distinct construction strategies. Prompting-based methods represent the dominant paradigm, where carefully engineered prompts guide LLMs to perform specific tasks without model parameter updates. These approaches rely on zero-shot, few-shot, or chain-of-thought prompting to elicit desired outputs. EDC, Feng et al., OntoChat, Kommineni et al., OntoKGen, Yang et al., AutoKG, and AIO all employ prompting strategies, demonstrating the widespread adoption of this approach due to its flexibility and lower computational requirements compared to fine-tuning.

In contrast, fine-tuning approaches adapt pre-trained LLMs to domain-specific KG construction tasks through gradient updates on curated training datasets. SAKA and UrbanKGen represent this category, achieving improved performance for specialized domains but requiring substantial annotated data and computational resources. Fine-tuning trades generality for specialization, producing models optimized for specific construction tasks but potentially less adaptable to new domains without retraining.

Beyond strategy selection, approaches differ fundamentally in their construction scope—what components of the KG they aim to produce. Instance-only approaches like EDC and AutoKG focus solely on populating knowledge instances from text, assuming pre-existing ontologies. These methods excel at extraction tasks but provide no support for ontology creation or evolution, limiting their utility when domain ontologies are unavailable or require adaptation. Ontology-only approaches such as OntoChat and AIO address the opposite problem, constructing conceptual schemas without populating instances. OntoChat provides conversational interfaces where LLMs generate competency questions (CQs) to guide requirements elicitation, but domain experts must manually construct actual ontologies. AIO focuses on ontology management for AI/ML literature with BioPortal integration but similarly does not consider

instance population.

The most comprehensive approaches pursue ontology & instance construction, simultaneously developing conceptual schemas and populating them with knowledge. Feng et al., Kommineni et al., OntoKGen, Yang et al., and SAKA provide this dual capability. Feng et al.'s framework generates formal OWL ontologies aligned with Wikidata through a four-stage pipeline incorporating property matching for semantic interoperability. Kommineni et al. bootstrap from expert knowledge through CQ generation, developing RDF/OWL ontologies from literature while simultaneously extracting instances. Despite this dual capability, most approaches still exhibit gaps in comprehensive pipeline coverage, as discussed in the following section.

2.2.2. Graph Models and Comprehensive Pipeline Coverage

A critical yet often underspecified dimension in LLM-based KG construction is the choice of graph model. As Table 2-2's "Graph Model" column reveals, most surveyed approaches default to RDF/OWL representations, reflecting the field's historical bias toward semantic web standards. RDF-targeted approaches—including EDC, Feng et al.'s framework, OntoChat, Kommineni et al., Yang et al.'s medical KG, and AIO—produce ontologies conforming to OWL standards, enabling semantic reasoning through RDFS/OWL inference rules and facilitating integration with established semantic web knowledge bases like Wikidata and DBpedia.

However, RDF's triple-based representation introduces overhead for property-rich entities, as discussed in Section 2.1. Conversely, LPG models optimize operational efficiency in graph traversal and pattern matching, making them particularly suitable for applications requiring real-time query responses. Yet despite these practical advantages, LPG-targeted approaches remain surprisingly rare in the LLM-based KG construction literature. As shown in Table 2-2, only OntoKGen and SAKA explicitly target LPG models. OntoKGen employs an adaptive Chain-of-Thought approach for technical document analysis, producing LPG-structured KGs for

domain-specific engineering knowledge, though the ontology remains at the DB schema level. SAKA focuses on audio dialogue processing, constructing LPG-based knowledge bases with multi-version management capabilities, though ontology creation and management remain manual processes stored in Relational Database Management Systems (RDBMS).

Beyond graph model choice, the extent of LLM involvement across pipeline tasks distinguishes mature frameworks from task-specific research prototypes. Hofer et al. (2024) identify eight essential construction tasks spanning the full KG lifecycle: KG Initialization (building initial ontologies), Data Preprocessing (source selection and data cleaning), Ontology Management (ontology alignment, extension, evolution), Knowledge Extraction (extracting instances), Entity Resolution and Fusion (duplicate detection and merging entities), Quality Assurance (validation and error detection), Knowledge Completion (filling missing information), and Incremental Integration (updating existing KGs).

Table 2-2's "Pipeline Coverage" column systematically analyzes where LLMs are actively employed within this task. The analysis reveals that most methods leverage LLMs for only 1-4 pipeline tasks, with significant gaps in LLM-assisted automation across the full construction lifecycle. For KG Initialization, only four approaches employ LLMs with strong support to build initial ontologies: Feng et al., Kommineni et al., OntoKGen, and OntoChat. Data Preprocessing receives only weak LLM support across most approaches. Ontology Management shows strong differentiation, with Feng et al., Kommineni et al., and AIO providing strong LLM support for ontology alignment, extension, and curation.

Knowledge Extraction demonstrates the broadest LLM adoption, receiving strong support from eight approaches (EDC, Feng et al., Kommineni et al., OntoKGen, Yang et al., AutoKG, SAKA, UrbanKGen). However, ER and EF remain largely unaddressed—only EDC and AutoKG employ LLMs with weak support for relation canonicalization. Quality Assurance mechanisms similarly show limited LLM involvement, with only OntoKGen and AIO incorporating

LLMs for validation steps. Knowledge Completion receives LLM attention only from UrbanKGent through fine-tuning for urban KG completion tasks. Perhaps most critically, Incremental Integration receives no LLM support across all surveyed approaches—all methods assume batch construction from static data sources, requiring complete KG reconstruction when source data changes.

2.2.3. Challenges and Limitations

Despite significant methodological advances, LLM-based KG construction faces several fundamental challenges that limit practical deployment. Understanding these challenges is essential for identifying productive research directions bridging the gap between experimental systems and production-ready frameworks.

Hallucination and domain-specific accuracy remain the most critical barrier to automated KG construction. As highlighted in Table 2–3's "Key Limitations" column, prompting-based approaches exhibit high hallucination risks, particularly for specialized domain knowledge not well-represented in pre-training corpora. Yang et al.'s evaluation reveals that LLM-generated CQ answers achieve only partial completeness (3/5 rating) for medical domain queries, frequently omitting critical information despite high relevance scores. This accuracy-completeness trade-off becomes particularly acute for safety-critical domains like healthcare and housing, where incorrect or incomplete information can have serious consequences. The fundamental issue is that LLMs generate false information confidently and plausibly, making detection difficult without domain expertise or external validation mechanisms. While fine-tuning approaches like UrbanKGent partially address this limitation through domain-specific instruction datasets, they introduce new challenges of requiring substantial annotated training data and producing non-generalizable models.

Closely related but distinct in its implications is the challenge of structured data integration. As evidenced by Table 2–2's "Input Data Source" column, most surveyed approaches target only unstructured textual corpora, with structured sources—government databases,

geospatial datasets, real estate registries—that provide authoritative information essential for domain-specific KGs remaining largely neglected. Only SAKA and UrbanKGent explicitly incorporate structured data sources, while the remaining eight approaches focus exclusively on text-based extraction. Even those few approaches that do address structured data focus primarily on schema-level integration rather than systematic strategies for combining structured and unstructured sources with appropriate provenance tracking and quality validation.

A third critical challenge is the incomplete coverage of essential pipeline tasks, particularly the severe underdevelopment of systematic LLM-assisted ontology creation and management. Among the ten surveyed approaches, only four employ LLMs for initial ontology construction and merely three provide strong support for ongoing ontology management. This is particularly problematic because ontology quality fundamentally determines KG utility poorly designed ontology with missing concepts or incorrect relationships renders even perfectly extracted instances useless. Yet seven approaches provide strong support for Knowledge Extraction while offering weak or no support for ontology construction, revealing a fundamental imbalance where instance automation is optimized while the conceptual foundation remains manual. Other critical gaps include limited Quality Assurance support (only OntoKGen and AIO incorporate validation) and minimal ER and EF attention (only EDC and AutoKG provide weak support). While Incremental Integration also receives no support, the fundamental barrier is the lack of systematic frameworks for creating and evolving ontological schemas.

These three challenges are not independent but deeply interconnected, creating complex dependency networks that compound deployment difficulties. Hallucination problems motivate increased human validation to ensure accuracy, but without principled collaboration design, validation becomes a scalability bottleneck that negates automation benefits. Structured data integration requires robust pipeline components that current approaches do not provide. Incomplete pipeline coverage makes production deployment

impractical, limiting real-world validation opportunities that could inform collaboration design improvements. The choice of graph model further compounds these challenges: RDF-targeted approaches benefit from established semantic web reasoning capabilities, but their triple-based representations create efficiency issues for property-rich entities; conversely, LPG-targeted approaches offer operational efficiency but lack standardized reasoning frameworks.

These limitations become particularly acute for spatial KGs in domains, where ontologies must simultaneously capture semantic concepts, spatial entities with geometric properties, and complex attributes—each dimension requiring different LLM prompting strategies, validation approaches, and integration methodologies.

2.3. Spatial Knowledge Graphs

Spatial KGs extend traditional KGs by explicitly incorporating geographic information and spatial relationships, enabling applications that require both semantic reasoning and spatial analysis. This section examines research trends in spatial KG development across diverse domains and identifies critical limitations constraining current approaches.

2.3.1. Research Trends

Spatial KGs have evolved along two trajectories: general-purpose systems providing broad geographic coverage with standardized spatial semantics, and domain-specific implementations integrating specialized knowledge to address targeted application requirements. Table 2-4 presents a comparative analysis of representative systems, revealing fundamental architectural dimensions that shape their capabilities and performance characteristics.

[Table 2-4] Comparative Analysis of Spatial KGs

KG	Scope	Domain	Graph	Spatial	Database	Referen
----	-------	--------	-------	---------	----------	---------

			Model	Comput ation Strateg y	& Indexing	ces
LinkedGeo Data	General – purpose	Geogra phic	RDF	On– the–fly	Virtuoso; R–Tree	(Stadle r et al., 2012)
YAGO2	General – purpose	Geogra phic	RDF	On– the–fly	PostGIS; GiST	(Hoffar t et al., 2013)
YAGO2Geo	General – purpose	Geogra phic	RDF	On– the–fly	Not specified	(Karali s et al., 2019)
OSMTTL	General – purpose	Geogra phic	RDF	Hybrid (option al pre– materia lization)	QLever SPARQL engine; envelope extensio n	(Bast et al., 2021)
WorldKG	General – purpose	Geogra phic	RDF	On– the–fly	Not specified	(Dsouz a et al., 2023)
KnowWher eGraph	Domain – specific	Environ mental	RDF	Pre– materia lized	GraphDB ; S2 cell– based DGG indexing	(R. Zhu et al., 2025)
WHOW– KG	Domain – specific	Water monitor ing	RDF	On– the–fly	Not specified	(Lippoli s et al., 2025)
TA–SKG	Domain – specific	Traffic Accide nt	LPG	Pre– materia lized	Not specified	(Park, 2023)
MALCZEW SKI et al.	Domain – specific	Housin g explora tion	RDF	On– the–fly	Apache Jena	(MALC ZEWSK I & JELOK

						HANI-NIARAKI, 2012)
Hong et al.	Domain-specific	Housing exploration	OWL/LPG	Pre-materialized	Not specified	(Hong, 2025)

Early general-purpose systems focused on establishing large-scale geographic knowledge bases with standardized spatial semantics. LinkedGeoData pioneered this by extracting structured information from OpenStreetMap using RDF with GeoSPARQL ontologies. YAGO2 incorporated point coordinates for distance-based queries, while YAGO2Geo extended this with full GeoSPARQL support including diverse geometry types. OSMTTL adopted a hybrid approach supporting both on-the-fly computation and optional pre-materialization of spatial relations, enabling fast queries on SPARQL engines without GeoSPARQL support. More recently, WorldKG has advanced spatial-semantic integration for urban applications.

Domain-specific spatial KGs have been developed where spatial relationships critically impact decision-making. KnowWhereGraph demonstrated large-scale environmental applications using S2 grid-based materialization at levels 8–13, trading storage overhead for rapid query evaluation. WHOW-KG integrated spatial and temporal dimensions for water quality monitoring and public health analysis. TA-SKG represents another application where integrated emergency response data with road network topology, pre-materializing spatial relationships between accidents and road networks to enable fast retrieval.

The housing domain has received particular attention due to its fundamental importance to quality of life. Early systems combined ontology-based knowledge representation with multi-criteria decision support (Cho & Kang, 2011; MALCZEWSKI & JELOKHANI-NIARAKI, 2012), demonstrating how ontology-based

standardization enables semantic query expansion and user preference modeling through Analytic Hierarchy Process. Laddada et al. (2020) developed the NARO ontology that richly models neighborhood characteristics with collaborative filtering and GeoSPARQL-based reasoning, though housing-level attributes received limited attention. Hong et al. advanced this by developing a user-centric ontology construction methodology deriving conceptual structures from query clustering, demonstrating partial automation of requirements elicitation while maintaining semantic coherence through SWRL rule-based reasoning.

2.3.2. Limitations

Despite significant advances, existing approaches exhibit persistent limitations across four dimensions: ontology construction, data model selection, spatial relationship representation, and spatial indexing integration.

All surveyed systems rely on manual ontology engineering requiring months or years of expert effort. While recent work has explored partial automation through query clustering (Hong, 2025), core ontological design remains predominantly manual. This becomes particularly acute for spatial KGs where ontologies must simultaneously capture semantic concepts, spatial entities with geometric properties, and complex relationship structures.

Existing systems exhibit strong bias toward RDF, inheriting standardization benefits but also performance limitations. While recent work has been explored LPG models (Hong, 2025; Park, 2023), these lack systematic methodologies for LPG-optimized ontology design that integrates ontological structures with instance data.

Current systems adopt divergent spatial relationship strategies: on-the-fly computation minimizes storage but incurs substantial computational cost without sophisticated indexing, while pre-materialization accelerates queries but causes storage explosion and lacks flexibility for dynamic thresholds (R. Zhu et al., 2025). For dense urban context, pre-materializing all proximity relationships

generates millions of edges, most never queried.

Systems maintaining spatial indices external to or disconnected from graph databases face data marshaling overhead, inability to unify query optimization, and maintenance complexity (Y. Sun & Sarwat, 2021). Many systems avoid this by relying on implicit database-level indexing without optimization or eliminating spatial indexing altogether, as seen where pre-materialized relationships substitute for indexed computation. This creates a trade-off where systems must choose between operational simplicity and query performance without achieving both simultaneously.

These limitations reveal fundamental gaps requiring integrated solutions: ontology construction automation through intelligent LLM-expert collaboration, systematic LPG-optimized ontology design methodologies, lightweight spatial relationship representation balancing storage and query flexibility, and native spatial indexing integration within graph architectures.

2.4. Spatial Indexing in Graph DBMSs

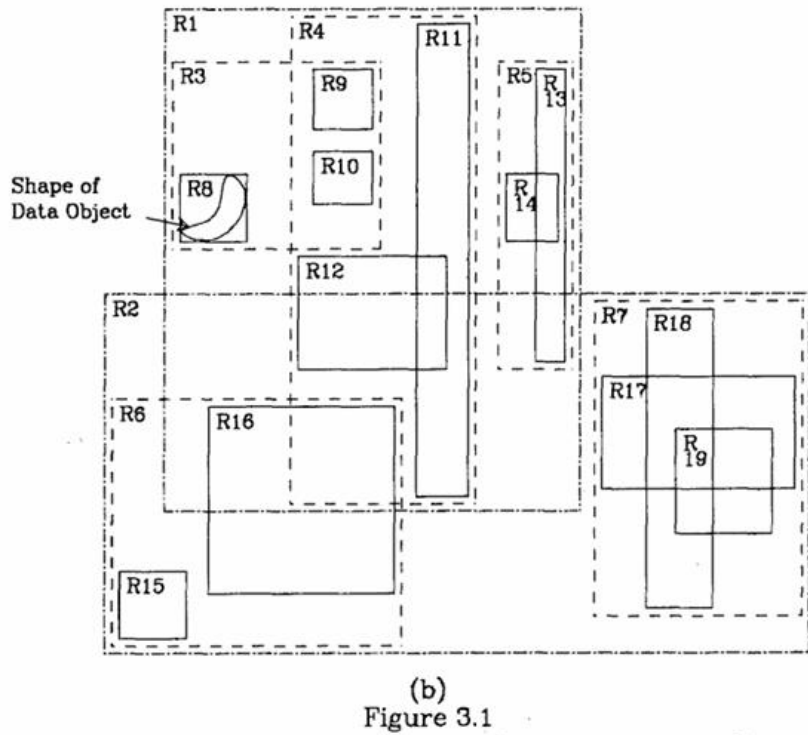
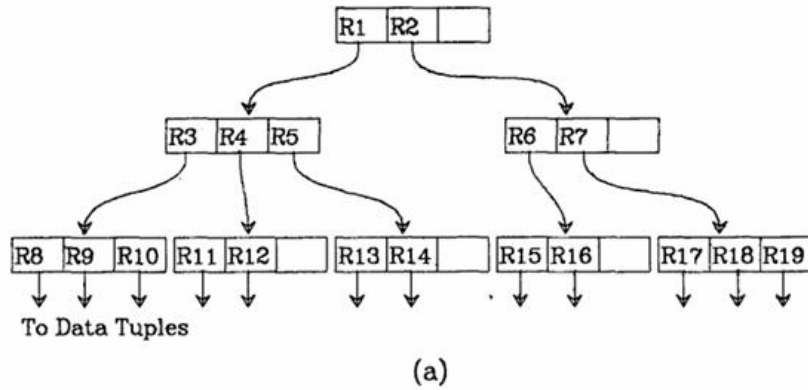
Efficient spatial query processing in GDBMSs has become increasingly important owing to the prevalence of spatial graph data. However, current GDBMSs lack effective spatial indexing, causing performance issues with complex spatial graph queries. This section reviews spatial indexing fundamentals, integration architectures in graph databases, and persistent limitations that motivate novel indexing approaches.

2.4.1. Spatial Index Fundamentals

Spatial indices are crucial in database systems for efficiently managing geospatial data and optimizing spatial query processing, which is essential for applications such as location-based services, urban planning, and GIS (L. Sun & Jin, 2023). A spatial index creates a specific order and arrangement for spatial objects, facilitating the rough filtering of unnecessary data during spatial queries and aiding in quickly finding target objects. In GDBMSs, where complex queries

require both spatial search and graph traversal, the lack of an effective spatial index can render even simple queries computationally expensive, severely affecting performance. Using an efficient spatial index such as R-Tree can reduce query complexity to $O(\log_M N)$ on average, where M is the capacity of each node in the R-Tree, representing the maximum number of children it can contain, and N is the total number of spatial objects.

Two primary paradigms dominate spatial indexing: data-driven methods and space-driven methods. Data-driven approaches, exemplified by the R-Tree family, organize spatial data based on the geometries of objects themselves using Minimum Bounding Rectangles (MBRs) to approximate object extents. The R-Tree hierarchically groups nearby objects into MBRs, with each internal node storing the MBR that bounds all its children's MBRs. This hierarchical structure enables efficient pruning during spatial queries—nodes whose MBRs do not intersect the query region can be eliminated without examining their contents. Variants such as R*-Tree optimize node splitting strategies to minimize overlap and coverage, while Hilbert R-Tree uses space-filling curves to improve insertion order (Beckmann et al., 1990).



[Figure 2-2] R-Tree illustration (Guttman, 1984)

Space-driven methods partition space independently of data distribution. Grid-based methods divide space into uniform cells, while hierarchical structures like Quad-Trees recursively subdivide regions based on quadrant partitioning (Finkel & Bentley, 1974). Space-filling curves (Hilbert, Peano, Morton/Z-order) map multidimensional space to one-dimensional sequences using various traversal patterns (Amiri et al., 2015). Geohash combines

hierarchical subdivision with Z-order encoding for hierarchical spatial indexing. While simpler to maintain, space-driven methods suffer from approximation errors when grid boundaries misalign with object geometries and cross-cell allocation errors when objects span multiple cells, leading to duplication and reduced efficiency (L. Sun & Jin, 2023).

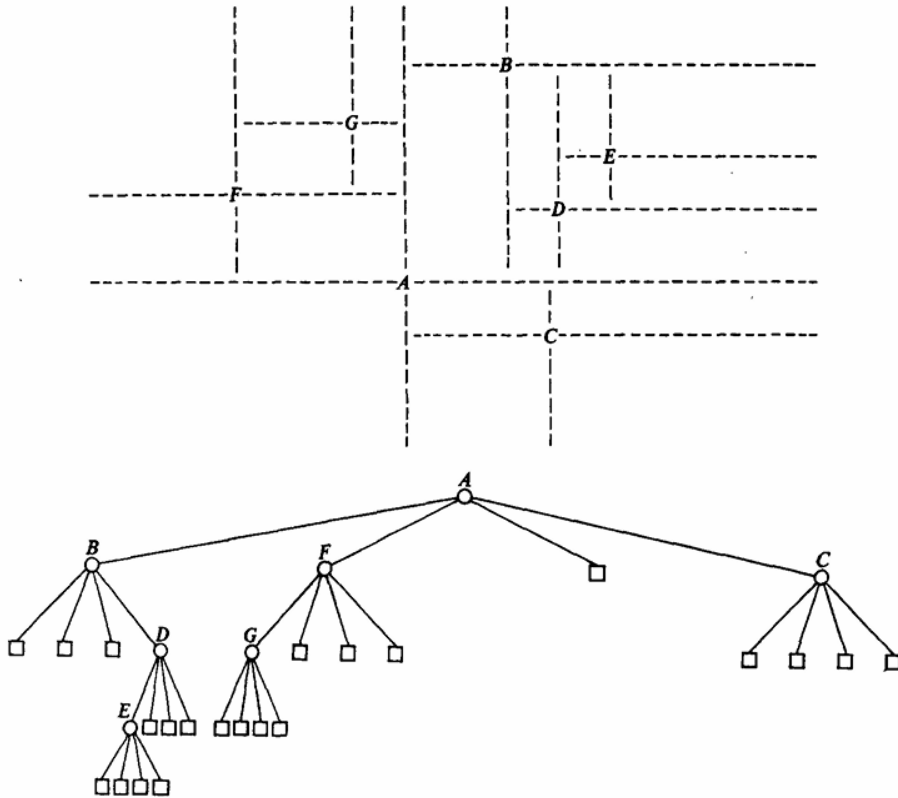


Fig. 1. Correspondence of a quad tree to the records it represents. RECORDS *A, B, C, D, E, F, G*. Null subtrees are indicated by boxes, but they do not appear explicitly in computer memory

[Figure 2–3] Quad-Tree illustration (Finkel & Bentley, 1974)

2.4.2. Integration of Architectures in Graph Databases

The deployment of spatial indices in graph databases can follow three distinct architectural patterns, each presenting unique trade-offs between performance, implementation complexity, and system integration depth. Table 2–5 provides a comprehensive taxonomy of

existing approaches organized by deployment strategy, graph model, and integration characteristics.

[Table 2–5] Taxonomy of Spatial Indexing Approaches in GDBMSs

Architecture Type	Graph Model	Index Type	Integration Mechanism	Representative Systems	References
Out–DBMS (External)	RDF	R–Tree	libspatialindex	RDF–3X, SPARQL engines	Brodt et al., 2010
	RDF	R–Tree, R*–Tree	Sail library	Various RDF stores	Liagouris et al., 2014; Theocharidis et al., 2019
	RDF	STR–Tree	JTS	GeoSPARQL–Jena	Osman, 2022
	LPG	BKD–Tree	ElasticSearch	JanusGraph	Procopiuc et al., 2003
Disconnected In–DBMS	RDF	R–Tree	Dedicated spatial table	Virtuoso	Virtuoso Documentation
	LPG	R–Tree	Spatial plugin with separate index graph	Neo4j Spatial	Neo4j Spatial Plugin Document
	LPG	R–Tree with reachability	Storing label paths in R–Tree	Riso–Tree	Sun & Sarwat, 2021
Space–Driven In–DBMS	RDF	Quad/Geohash–Prefix–Tree	Text search integration (Lucene)	RDF4j, Stardog, GraphDB	Huang et al., 2019
	RDF	Hilbert	1D	Geo–Store	Wang et al.,

		Curve	mapping		2012
	LPG	Hilbert Curve	Google S2 library	NebulaGraph	NebulaGraph Documentation
	LPG	Uniform Grid	Spatial Indexing Properties in nodes	GeoExpand	Sun et al., 2019
	LPG	Uniform Grid	Grids as nodes, spatial edges	TigerGraph	TigerGraph Blog

External deployment approaches maintain spatial indices outside the primary graph database using established libraries such as libspatialindex (Brodth et al., 2010) or JTS (Java Topology Suite). RDF-based systems like RDF-3X and various SPARQL engines leverage the SaIL (Spatial Abstraction and Indexing Library) to integrate R-Tree structures with external storage (Hadjieleftheriou et al., 2005; Liagouris et al., 2014; Theocharidis et al., 2019). For LPG systems, JanusGraph employs ElasticSearch with BKD-Tree indexing for distributed spatial operations (Procopiuc et al., 2003). This architectural pattern separates spatial indexing logic from the core database engine, enabling reuse of mature spatial indexing implementations across different graph database platforms.

Disconnected approaches store spatial indices within the GDBMS infrastructure while maintaining logical separation from the main graph structure. Implementation strategies vary by system design. Virtuoso maintains R-Tree indices in dedicated spatial tables separate from RDF triple storage. Neo4j's spatial plugin creates a parallel index graph containing duplicate copies of spatial nodes organized in R-Tree structures, which are queried independently and then correlated with the main graph. Riso-Tree extends the disconnected model by augmenting R-Tree nodes with reachability metadata, storing connectivity information such as label paths and

reachable node identifiers as node properties (Y. Sun & Sarwat, 2021). This augmentation enables more efficient processing of combined spatial-graph queries without fundamentally merging the spatial index with the graph topology.

Space-driven methods achieve direct integration by representing spatial partitioning structures as native graph elements. Text-search-based approaches in RDF platforms (RDF4j, Stardog, GraphDB) leverage Lucene infrastructure with Quad-Prefix-Tree or Geohash-Prefix-Tree encodings, storing spatial codes as indexed properties (Huang et al., 2019). Geo-Store and NebulaGraph applies Hilbert curve transformations to convert 2D coordinates into 1D sequences for spatial indexing (Wang et al., 2012). Uniform Grid-based implementations adopt two primary strategies: property-based methods like GeoExpand embed grid cell identifiers directly in node properties (Y. Sun & Sarwat, 2019), while graph-native approaches in TigerGraph model grid cells as explicit nodes with spatial adjacency represented as edges. These methods achieve tight integration with the graph query engine by eliminating external dependencies and representing spatial relationships through native graph mechanisms.

This architectural taxonomy reveals diverse implementation strategies, each optimizing different aspects of spatial-graph integration. The subsequent section examines the inherent limitations and performance trade-offs that constrain these approaches in production deployments.

2.4.3. Limitations and Design Trade-offs

Analysis of existing approaches reveals challenges that constrain spatial-graph query performance. Table 2-6 synthesizes the fundamental trade-offs inherent in different architectural choices.

[Table 2-6] Architectural Trade-offs in Spatial Graph Indexing

Dimension	Out-DBMS	Disconnected In-	Reachability -	Space-Driven In-
-----------	----------	------------------	----------------	------------------

		DBMS	Augmented	DBMS
Implementation Complexity	Low	Medium	High	Medium
Index Loading Overhead	Significant for large datasets	None	None	None
ID Matching Overhead	High (serialization)	High (UUID correlation)	None (pre-computed)	Low (grid cell lookup)
Memory Efficiency	Baseline	Significantly higher	Exponential in hop count	Moderate (grid duplication)
Query Performance	Substantially slower	Moderately slower	Limited to shallow hops	Good with approximation errors
Arbitrary-Hop Queries	Supported	Supported	Not supported	Supported
Maintenance Complexity	Low	Medium	Very High	Low

External deployment approaches suffer from substantial data transfer overhead and lack query plan integration, as the architectural separation between spatial and graph processors prevents holistic optimization. Disconnected in-database approaches eliminate loading overhead but require expensive correlation operations to reconcile spatial results with graph entities, introducing significant memory overhead through duplicate spatial representations.

Reachability-augmented approaches like GeoExpand (Y. Sun & Sarwat, 2019) and Riso-Tree (Y. Sun & Sarwat, 2021) attempt to bridge the spatial-graph divide by pre-computing connectivity information, but face fundamental scalability limitations. The volume of stored connectivity information grows exponentially with hop distance, rendering these approaches practical only for shallow traversals of two to three hops. Queries requiring arbitrary-length paths become infeasible, severely constraining application flexibility.

Space-driven integration approaches that represent grid cells as graph nodes achieve native integration with simplified maintenance and low ID matching overhead through direct grid cell lookups. However, they inherit fundamental limitations of fixed spatial partitioning. Grid-based methods inevitably produce approximation errors when cell boundaries misalign with object geometries, while objects spanning multiple cells create allocation ambiguities leading to duplication or incomplete results. Additionally, the limited set of spatial operators constrains query expressiveness compared to geometry-aware approaches.

These persistent limitations motivate the need for fundamentally different architectural designs. One of the ideal spatial indexing solutions should achieve native integration representing spatial structures as first-class graph elements with direct edge-based connectivity, efficient support for arbitrary-length traversals without pre-materialized constraints, hierarchical data-driven partitioning adapting to actual spatial distributions, unified query optimization reasoning holistically about combined operations, and practical scalability for large-scale deployments. Achieving these properties simultaneously while maintaining reasonable implementation complexity remains an open challenge.

2.5. Summary

This chapter reviewed foundational research areas relevant to constructing domain-specific spatial KGs, revealing critical gaps that motivate this dissertation's integrated architecture.

The review of KG fundamentals reveals that two dominant data models shape contemporary research. RDF excels in semantic interoperability and formal reasoning through RDFS/OWL but suffers from verbose representation and reduced performance for complex graph operations. Conversely, LPG optimizes operational efficiency in traversal and pattern matching, particularly for property-rich entities requiring real-time responses, but lack standardized reasoning capabilities. For spatial KGs requiring complex traversals

combined with spatial predicates, LPG offers performance advantages that motivate systematic methodology development, though its limited ontological tooling creates quality assurance challenges requiring novel architectural solutions.

Recent advances demonstrate LLMs' potential for automating KG construction subtasks, but existing approaches exhibit significant limitations. Most methods target only unstructured text, neglecting authoritative structured sources essential for domain-specific KGs. Pipeline coverage analysis reveals fundamental imbalances: seven approaches provide strong support for Knowledge Extraction while only four employ LLMs for ontology initialization and merely three for ontology management, revealing optimization of instance automation while conceptual foundations remain manual. Critical gaps include limited Quality Assurance and no Incremental Integration support across all approaches. Three interconnected challenges limit practical deployment: hallucination and domain-specific accuracy, structured data integration gaps and incomplete pipeline coverage.

The examination of spatial KGs reveals persistent limitations across four dimensions. All surveyed systems rely on manual ontology engineering requiring months or years of expert effort, even with recent attempts at partial automation. The field exhibits strong bias toward RDF despite performance limitations for property-rich spatial entities and multi-hop path queries, while recent LPG explorations lack systematic methodologies for ontology-driven construction. Spatial relationship representation faces an efficiency-flexibility trade-off where on-the-fly computation incurs substantial cost without sophisticated indexing, while pre-materialization causes storage explosion and limits query flexibility. Finally, systems maintaining spatial indices externally or disconnected from main graph face data marshaling overhead and inability to unify query optimization, while many systems avoid this complexity by eliminating spatial indexing altogether.

Current spatial indexing architectures in graph databases exhibits fundamental trade-offs constraining spatial-graph query performance. External indices achieve mature implementations but

suffer from data marshaling overhead and cannot support unified query optimization across spatial and graph components. Disconnected indices reduce marshaling costs but require UUID-based correlation between spatial results and graph entities, introducing computational overhead scaling with result set size. Space-driven methods inherit approximation errors from grid-based partitioning. No current system successfully unites hierarchical R-Tree efficiency with seamless native graph integration enabling direct edge-based connectivity between spatial index components and spatial entities.

These gaps collectively motivate this dissertation's integrated architecture combining LLM-assisted semi-automatic KG construction with spatial graph indexing within LPG GDBMSs. The proposed approach addresses ontology engineering challenges through systematic LLM-expert collaboration, leverages LPG models for operational efficiency and integrates SGIR-Tree spatial graph indexing to enable flexible on-the-fly spatial computation without pre-materialization overhead. The integration opens new possibilities for KG construction on scale while maintaining semantic richness and query performance required for production systems.

Chapter 3. Methodology

This chapter presents the integrated methodology for constructing MSKG in LPG GDBMS. The methodology establishes core element technologies for each subgraph's construction.

Section 3.1 establishes foundational concepts including the MSKG structure and LPG ontological components. Section 3.2 presents concrete MSKG architecture and the implementation workflow. Sections 3.3 and 3.4 detail the LPG ontology creation. Section 3.5 describes Semantic and Data Graph construction. Section 3.6 presents the Spatial Index Graph construction.

3.1. Preliminaries

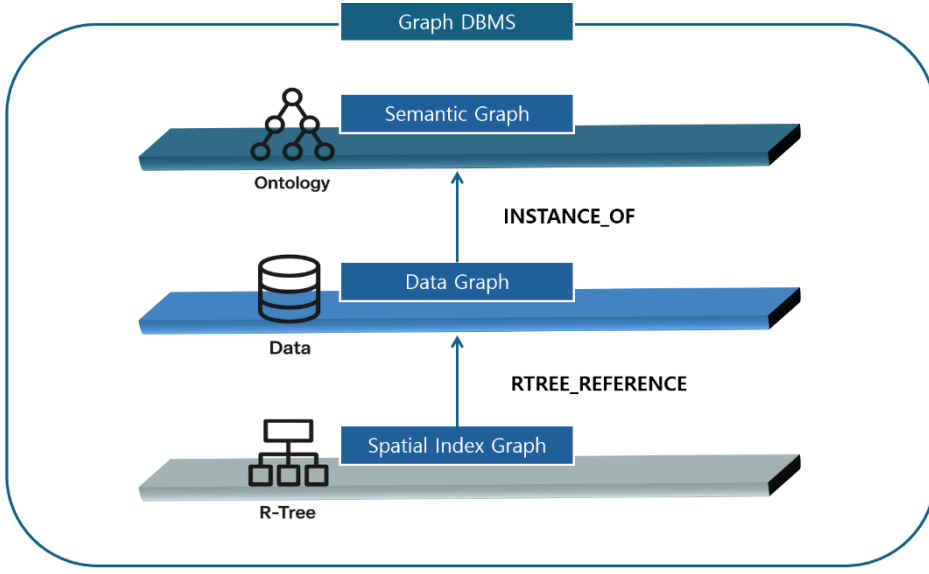
This section establishes the foundational concepts necessary for understanding the proposed architecture. We introduce the MSKG structure that organizes spatial and semantic information within a unified LPG database, followed by the LPG Ontology components that enable formal knowledge representation.

3.1.1. Multi-layered Spatial Knowledge Graph Structure

The architecture organizes a single LPG into three interconnected subgraphs that manage distinct aspects of knowledge representation while enabling seamless integration through native graph edges.

Definition 1 (Multi-layered Spatial Knowledge Graph). An LPG G is decomposed into a multi-layered structure $MSKG = (G_S, G_D, G_I)$, where each layer is a subgraph of G :

$$G = G_S \cup G_D \cup G_I$$



[Figure 3–1] Multi-layered Spatial Knowledge Graph Structure in GDBMS

As illustrated in Figure 3–1, the Architecture organizes these three subgraphs hierarchically to manage distinct aspects of the KG:

- Semantic Graph $G_S = (N_S, E_S)$ represents the ontology layer where $N_S = \{n \in N \mid \varphi(n) \in \{\text{Concept}, \text{Relation}\}\}$ contains semantic entity nodes and $E_S = \{e \in E \mid e = (n_i, n_j), n_i, n_j \in N_S\}$ connects these entities to encode domain knowledge structures including concept hierarchies and semantic constraints.
- Data Graph $G_D = (N_D, E_D)$ contains the instance layer where $N_D = \{n \in N \mid n \notin N_S \wedge n \notin N_I\}$ represents concrete entities conforming to the ontology and $E_D = \{e \in E \mid e = (n_i, n_j), n_i, n_j \in N_D\}$ captures relationships between instances.
- Spatial Index Graph $G_I = (N_I, E_I)$ provides spatial graph indexing where $N_I = \{n \in N \mid \varphi(n) \text{ ends with "RTree"} \wedge \exists (k, v) \in \phi(n), k = \text{"bbox"}\}$ contains hierarchical index nodes and $E_I = \{e \in E \mid \psi(e) \in \{\text{RTREE_ROOT}, \text{RTREE_CHILD}, \text{RTREE_REFERENCE}\}\}$ includes both intra-index edges (RTREE_ROOT, RTREE_CHILD) and connections to spatial data nodes (RTREE_REFERENCE).

Inter-layer connections are established through specialized edge sets $E_{SD} = \{e \in E \mid e = (n_d, n_s), n_d \in N_D, n_s \in N_S, \psi(e) = \text{INSTANCE_OF}\}$

linking semantic concepts to their instances, and $E_{ID} = \{e \in E \mid e = (n_i, n_d), n_i \in N_I, n_d \in N_D, \psi(e) = \text{RTREE_REFERENCE}\}$ connecting spatial indices to indexed entities. These inter-layer edges enable unified query processing that traverses from abstract concepts through concrete instances to spatial computations without requiring external index lookups or data transformation.

3.1.2. LPG-based Ontology

Unlike traditional RDF-based ontologies, ontologies in the LPG environment represent both concepts and relations as nodes, enhancing flexibility in metadata management.

Definition 2 (LPG Ontology). The ontology $O = SS \cup VS$ represented in the semantic graph G_S consist of a semantic schema (SS) and validating schema (VS) defined as follows:

Semantic Schema $SS = (C, R, Ax, Inf)$ captures the conceptual structure where:

- $C = \{n \in N_S \mid \varphi(n) = \text{Concept}\}$ is the set of concept nodes representing domain entities
- $R = \{n \in N_S \mid \varphi(n) = \text{Relation}\}$ is the set of relation nodes defining semantic connections
- $Ax \subseteq (C \cup R) \times R \times (C \cup R)$ is the set of axioms representing terminological knowledge, partitioned into:
 - Hierarchical axioms $Hi = \{(c_i, r, c_j) \in Ax \mid c_i, c_j \in C, \psi(r) \in \{\text{SUBCLASS_OF}, \text{PART_OF}\}\}$ for hierarchical relationships
 - Property axioms $Prop = \{(\omega_1, r, \omega_2) \in Ax \mid \omega_1 \in (C \cup R), \omega_2 \in C, \psi(r) \in \{\text{HAS_ATTRIBUTE}, \text{HAS_PROPERTY}, \text{HAS_TYPE}\}\}$ for attribute and property assignments
 - Domain axioms $Dom = \{(\omega_1, r, \omega_2) \in Ax \mid \omega_1, \omega_2 \in (C \cup R), \psi(r) \notin \{\text{SUBCLASS_OF}, \text{PART_OF}, \text{HAS_ATTRIBUTE}, \text{HAS_PROPERTY}, \text{HAS_TYPE}\}\}$ for domain-specific relationships. Thus $Ax = Hi \cup Prop \cup Dom$.
- $Inf = \{r_i \mid r_i: \text{IF condition THEN conclusion}\}$ is set of inference rules encoding executable knowledge for query transformation

Validating schema $VS = (\text{Schema_Types}, \text{Semantic_Properties}, Ax_{enum})$ extends SS with implementation-level specifications where:

- **Schema_Types** assigns implementation types through $\tau: (C \cup R) \rightarrow \{\text{label}, \text{key}, \text{enum}, \text{semantic}\}$ where **label** indicates instantiable node/edge labels, **key** maps concepts to property keys, **enum** represents subtype values, and **semantic** denotes abstract ontology elements not directly instantiated
- **Semantic_Properties** = $\{(\omega_1, r, \omega_2, \text{dtype}, \text{nullable}, \text{unit}) \mid (\omega_1, r, \omega_2) \in \text{Prop}, \tau(\omega_1) = \text{label}, \tau(\omega_2) = \text{key}\}$ extends property axioms with **dtype** $\in \{\text{String}, \text{Integer}, \text{Float}, \text{Boolean}, \text{Date}, \text{Point}, \text{List}\}$ for type specification, **nullable** $\in \{\text{true}, \text{false}\}$ for constraint enforcement, and **unit** for measurement metadata
- $Ax_{enum} = \{(c_1, r, c_2) \in \text{Prop} \mid c_1, c_2 \in C, \psi(r) = \text{HAS_TYPE}, \tau(c_1) = \text{label}, \tau(c_2) = \text{enum}\}$ defines type enumeration axioms for subtype classification

The Semantic Schema provides an abstract conceptual framework defining what entities and relationships exist in the domain, while the Validating Schema ensures this knowledge can be operationalized through executable database constraints. Together they form the complete ontology that governs KG construction and semantic consistency throughout the system lifecycle.

3.2. Multi-Layered Spatial Knowledge Graph Architecture

This section explains the MSKG architecture and presents the implementation workflow. While Section 3.1 formally defines what each subgraph contains, this section addresses why this subgraph-based separation is necessary and how the construction phases build each subgraph.

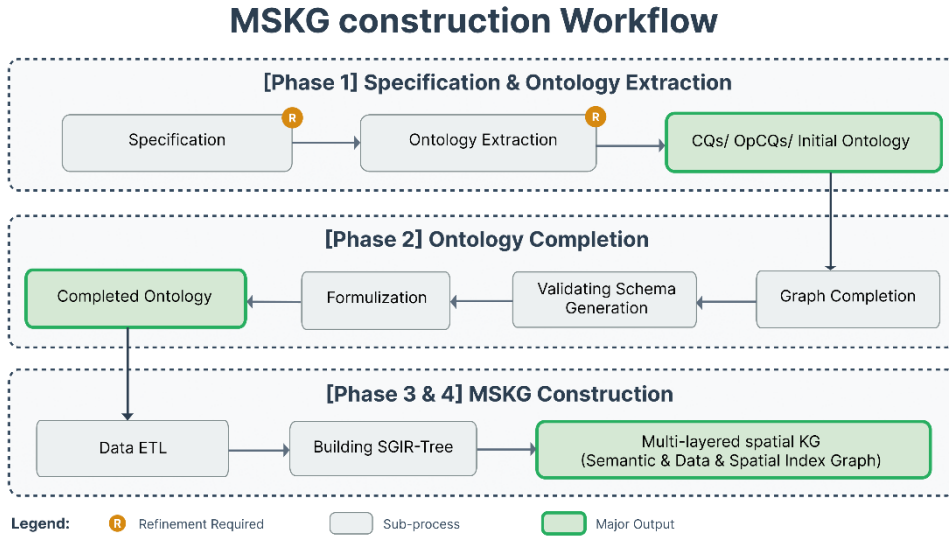
The MSKG architecture addresses three critical gaps in existing LPG-based KG systems. First, unlike RDF systems with RDFS/OWL, LPG environments lack standardized mechanisms for ontology representation—the LPG Ontology, instantiated as G_S , fills this gap

by representing ontological structures as first-class graph elements. Second, transforming ontologies into LPG-compatible formats requires choosing among modeling patterns with different trade-offs— G_D construction involves systematic comparison of Direct, Singleton, and Statement approaches. Third, existing graph databases maintain spatial indices externally or disconnected from graph topology— G_I , constructed using the SGIR-Tree methodology (Kim et al., 2024), integrates spatial indexing as subgraphs. Table 3-1 summarizes the correspondence between MSKG subgraphs and construction phases.

[Table 3-1] MSKG Subgraph-Phase Correspondence and Key Element Technologies

Output	Construction Phase	Key Element Technologies	Section
LPG Ontology	Phase 1-2	LLM-based CQ generation, middle-out ontology extraction, semi-automatic graph completion	3.3, 3.4
Semantic & Data Graph	Phase 3	Direct/Singleton/Statement LPG modeling patterns, schema validation	3.5
Spatial Index Graph	Phase 4	SGIR-Tree spatial graph indexing and algorithms	3.6

A distinctive feature of our ontology construction approach is the adoption of a middle-out strategy (Femi Aminu et al., 2020). Unlike traditional top-down approaches that begin with highly abstract concepts or bottom-up approaches that start with specific instances, the middle-out strategy identifies the most critical domain concepts first—those that naturally appear in user queries in Phase 1—and then expands both upward through generalization and downward through specialization in Phase 2.



[Figure 3–2] Implementation Workflow for MSKG Construction

Figure 3–2 illustrates how the four construction phases connect to produce the integrated MSKG. While the workflow proceeds sequentially, the emphasis is on the element technologies employed at each phase rather than the pipeline structure itself.

The first phase establishes foundational requirements by generating CQs that define what the KG must answer, operationalizing these questions into measurable conditions, and extracting core concepts, relations, and axioms.

The second phase completes the initial ontology by establishing hierarchical relationships, generating validating schema with implementation–level specifications, and formalizing inference rules for query transformation.

The third phase implements the ontology by collecting and transforming domain data, validating it against schema constraints, and instantiating both Semantic and Data Graphs within the database.

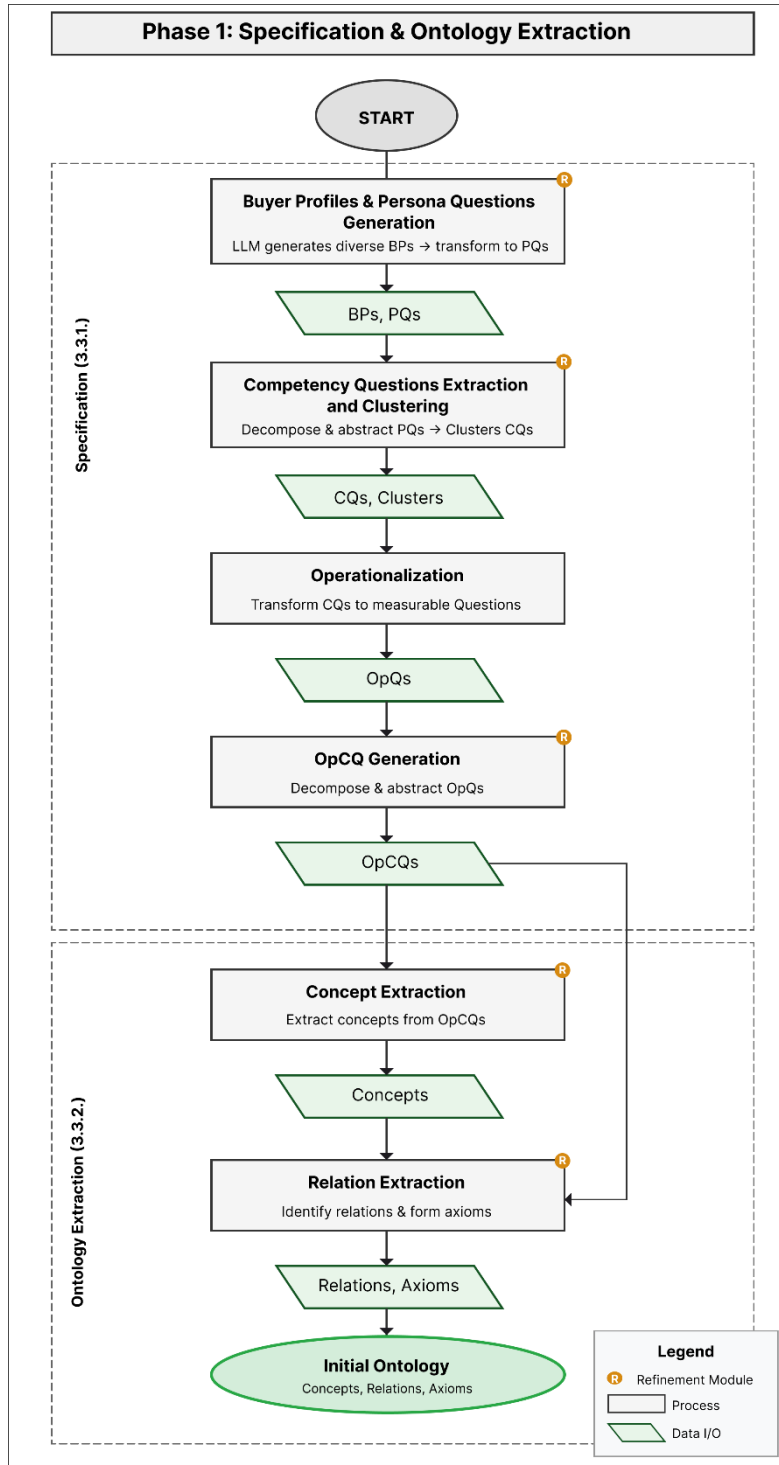
The fourth phase constructs spatial graph indexing through SGIR–Tree methodology, creating the Spatial Index Graph with direct connections to spatial entities. The resulting MSKG integrates all three subgraphs within a single database, enabling unified query

processing that seamlessly combines semantic reasoning, data retrieval, and spatial operations.

3.3. Specification & Ontology Extraction

This section presents the first stage of LPG Ontology Creation, which establishes the foundation for KG construction by systematically defining domain requirements and extracting core ontological elements. As illustrated in Figure 3–3, the phase operates through two major components connected by a sequential pipeline. The Specification component, shown in the upper portion of the pipeline, systematically generates and refines CQs. The Ontology Extraction component, depicted in the lower portion, derives fundamental concepts, relations, and axioms from these CQs.

Throughout the pipeline, all processes follow a semi-automatic approach where LLMs generate initial outputs and human experts modify them. Additionally, refinement modules marked with circled R symbols appear at five specific processes where duplicate entity resolution is essential. These refinement modules implement a clustering-based deduplication mechanism that groups semantically similar entities using DBSCAN clustering, then applies LLM-based entity resolution and fusion within each cluster to identify and merge redundant entities, with human experts providing final validation.



[Figure 3–3] Phase1: Specification & Ontology Extraction pipeline

3.3.1. Specification

The Specification component transforms high-level domain understanding into concrete, queryable requirements through four sequential processes. The first process generates diverse user profiles and their corresponding persona questions. The LLM generates profiles structured as combinations of demographic characteristics and domain-specific considerations, then synthesizes each profile into coherent persona questions that articulate stakeholder information needs. The refinement module applies to profile generation to resolve duplicate profiles arising from semantic similarity.

The second process extracts atomic CQs from persona questions while clustering them by semantic similarity. Persona questions often contain multiple information needs in compound form, requiring decomposition into independent units addressing single, well-defined requirements. Simultaneously, the LLM groups related CQs to reveal thematic structures, identifying which domain aspects receive the most stakeholder attention. The refinement module applies to CQ extraction to eliminate duplicate questions arising from similar information needs expressed with different terminology.

The third process performs operationalization to transform abstract concepts within CQs into concrete, measurable conditions. This transformation requires domain-specific knowledge to map qualitative requirements to quantitative specifications. Abstract notions such as convenience or quality translate into specific distance thresholds, categorical classifications, or numerical ranges evaluable through database queries. The LLM proposes operationalization strategies based on domain conventions, with human experts validating that translations accurately reflect local standards and user expectations.

The fourth and final process generates abstracted versions of operationalized questions suitable for ontology extraction. While operationalized questions contain concrete values and thresholds necessary for implementation, abstracted versions remove specific numerical details to focus on conceptual relationships and entity

types, ensuring that ontology extraction identifies general domain patterns rather than overly specific instances. The refinement module applies to abstracted CQ generation to consolidate questions referring to the same conceptual patterns with different parameter values.

3.3.2. Ontology Extraction

The Ontology Extraction component analyzes the refined abstracted CQs to systematically identify fundamental building blocks of the domain ontology. As shown in the lower portion of Figure 3–3, this component consists of two extraction processes that produce the initial semantic schema.

The concept extraction process identifies domain entities within abstracted CQs. The LLM processes each question to extract concepts characterized by canonical names, descriptive definitions, and synonym lists, balancing generality to align with standard KG conventions while remaining instantiable with domain-specific data. The refinement module applies to concept extraction to merge duplicate concepts arising from semantic variations or synonymous terminology.

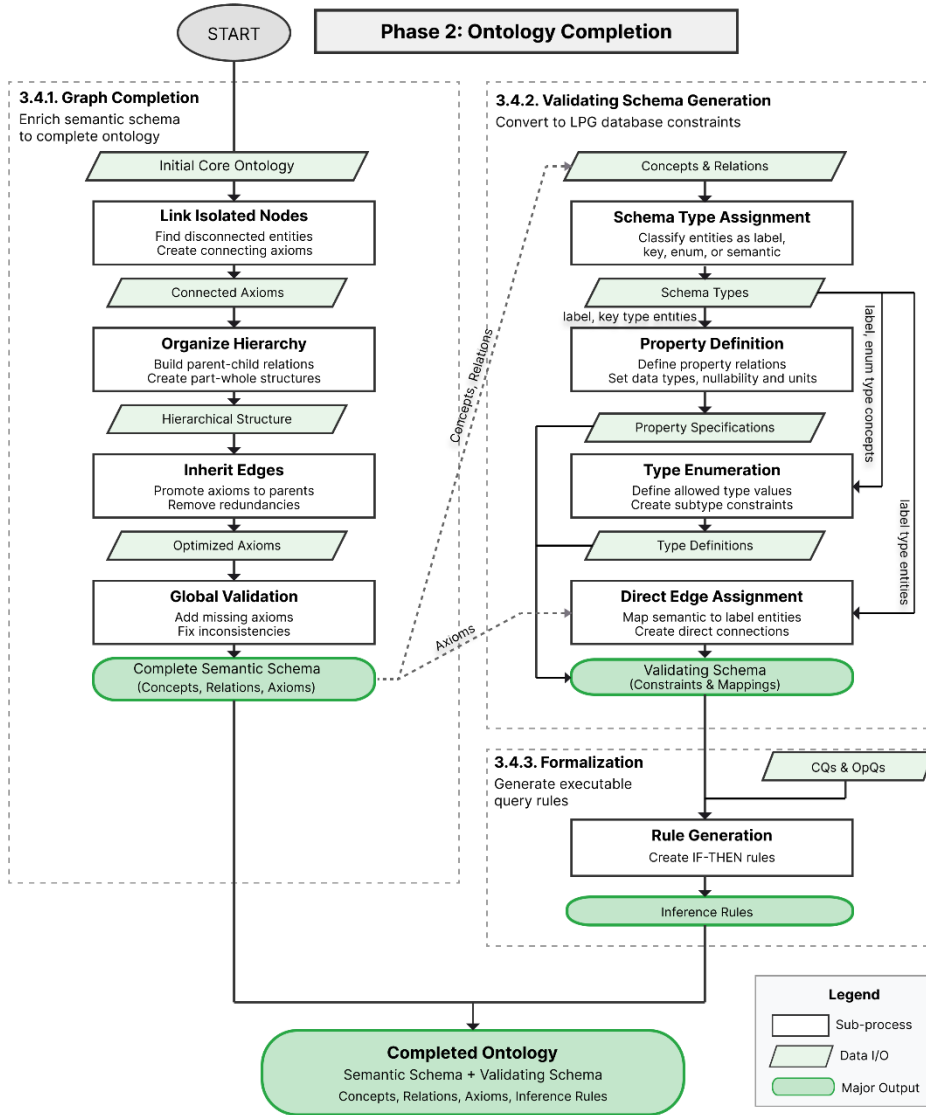
The relation extraction process identifies semantic connections between concepts by analyzing entity interactions within abstracted CQs. The LLM examines each question with its extracted concepts to determine relationships, characterizing each by semantic type, domain constraints specifying valid subject entities, and range constraints specifying valid object entities. The extraction focuses on broadly applicable relationships while avoiding overly domain-specific predicates. Each relation combines with its connected entities to form axioms specifying valid relationship patterns. The refinement module applies to relation extraction to consolidate duplicate relations and their associated axioms.

As indicated by the output indicators in Figure 3–2, this phase produces three primary outputs. The CQs and their operationalized versions at multiple abstraction levels establish comprehensive requirements defining what the KG must support. The initial ontology

comprises core concept set C_{core} , core relation set R_{core} , and core axiom set Ax_{core} provides the foundational semantic structure. This initial semantic schema $SS_{initial} = (C_{core}, R_{core}, Ax_{core}, Inf_{\emptyset})$, where $Inf_{\emptyset}$ represents an empty rule set, captures essential domain knowledge extracted from user requirements, establishing the scope and structural foundation that guide all subsequent construction phases.

3.4. Ontology Completion

The initial ontology extracted in Phase 1 contains only core concepts, relations, and axioms identified through CQ analysis, resulting in a fragmented structure lacking proper hierarchical organization and implementation specifications. This phase systematically transforms the initial semantic schema into a complete ontology $O = SS \cup VS$ ready for database implementation. As illustrated in Figure 3–4, the phase operates through three sequential processes that progressively enrich the semantic structure, generate implementation–level specifications, and formalize inference rules. All processes follow the semi–automatic approach where LLMs generate initial outputs and human experts validate them.



[Figure 3–4] Phase2: Ontology Completion pipeline

3.4.1. Graph Completion

The Graph Completion process enriches the fragmented initial schema into a cohesive semantic structure through four sequential sub-tasks that establish connectivity, organize hierarchies, optimize through inheritance, and validate completeness. This process transforms $SS_{initial}$ into a structurally complete semantic schema $SS = (C, R, Ax, Inf)$ with proper hierarchical organization and comprehensive axiom coverage.

The first sub-task identifies and connects isolated nodes that do not participate in any axioms. Isolated concepts or relations may arise when Phase 1 extraction processes identify entities from different CQs without establishing their relationships to the broader ontology structure. The LLM analyzes each isolated node's definition and semantic context to propose appropriate connections to existing ontology elements, with human experts validating and refining these proposals to ensure semantic correctness.

The second sub-task organizes concepts into hierarchical structures using SUBCLASS_OF and PART_OF relationships. These standard ontological relations form the backbone of concept organization, with SUBCLASS_OF expressing taxonomic is-a relationships and PART_OF capturing mereological whole-part structures (Hofer et al., 2024; Hogan et al., 2022). The LLM processes the concept set to propose hierarchical relationships that balance generalization toward abstract super-classes and specialization toward concrete subclasses, following the middle-out strategy initiated in Phase 1. Human experts validate the proposed hierarchies to ensure they accurately reflect domain knowledge and provide appropriate abstraction levels for both semantic reasoning and data instantiation.

The third sub-task optimizes the ontology through axiom inheritance. After establishing the hierarchical structure, many axioms initially assigned to specific concepts may apply to entire concept subtrees. The LLM analyzes each axiom to determine whether it can be generalized to a parent concept, examining whether the axiom holds for all sibling concepts under the same parent. When an axiom applies universally across a concept's children, it is promoted to the parent level, eliminating redundancy and improving ontology maintainability. Human experts validate these generalizations to ensure they preserve semantic accuracy while achieving conciseness.

The fourth sub-task performs global validation to ensure the completed semantic schema maintains consistency, completeness, and accuracy. The LLM conducts comprehensive review to identify

missing essential axioms, circular references, contradictory relationships, and other structural issues. Human experts validate the identified issues and proposed corrections, making final adjustments to produce a structurally sound and semantically complete schema.

3.4.2. Validating Schema Generation

The Validating Schema Generation process transforms the complete semantic schema into implementation-level specifications necessary for database constraint enforcement. This process generates $VS = (\text{Schema_Types}, \text{Semantic_Properties}, Ax_{enum})$ through four sub-tasks that classify ontology elements for database mapping, define property metadata, establish type enumerations, and reassign axioms for instantiation.

The first sub-task assigns schema types to all ontology elements using the type function $\tau: (C \cup R) \rightarrow \{\text{label}, \text{key}, \text{enum}, \text{semantic}\}$. This classification determines how each concept and relation maps to LPG database constructs. Elements classified as labels become instantiable node or edge labels in the database, those classified as key map to property keys, enum elements represent subtype values for specific labels, and semantic elements remain as abstract ontology nodes not directly instantiated. The LLM analyzes each element's role, instantiability, and intended usage to propose appropriate type assignments, with human experts validating these classifications to ensure they support efficient data modeling and query patterns.

The second sub-task defines semantic properties with validation metadata for all label-type entities. For each entity classified as label, the LLM identifies which key-type concepts should serve as its properties and specifies metadata including data types (String, Integer, Float, Boolean, Date, Point, List), nullability constraints determining whether properties are required or optional, and measurement units for quantitative attributes. Human experts validate these property definitions to ensure they align with domain standards and support anticipated query patterns.

The third sub-task establishes type enumeration axioms that

define subtype classifications. For concepts that have enum-type sub-concepts, the process generates Ax_{enum} axioms specifying valid type values. These enumerations enable categorical classification within the database while maintaining semantic relationships in the ontology layer. The LLM proposes appropriate enumeration structures based on the hierarchical organization established during graph completion, with human experts validating that enumerations capture all relevant subtypes and maintain consistency with domain terminology.

The fourth sub-task performs direct edge assignment to ensure all axioms reference instantiable entities. When axioms connect to semantic-classified entities that remain abstract, those axioms must be reassigned to their concrete label-type descendants to enable actual relationship instantiation in the database. The LLM analyzes each axiom involving semantic entities and proposes reassignments to appropriate label entities based on the established hierarchy and semantic relationships, with human experts validating that reassignments preserve intended semantics while enabling database implementation.

3.4.3. Formalization

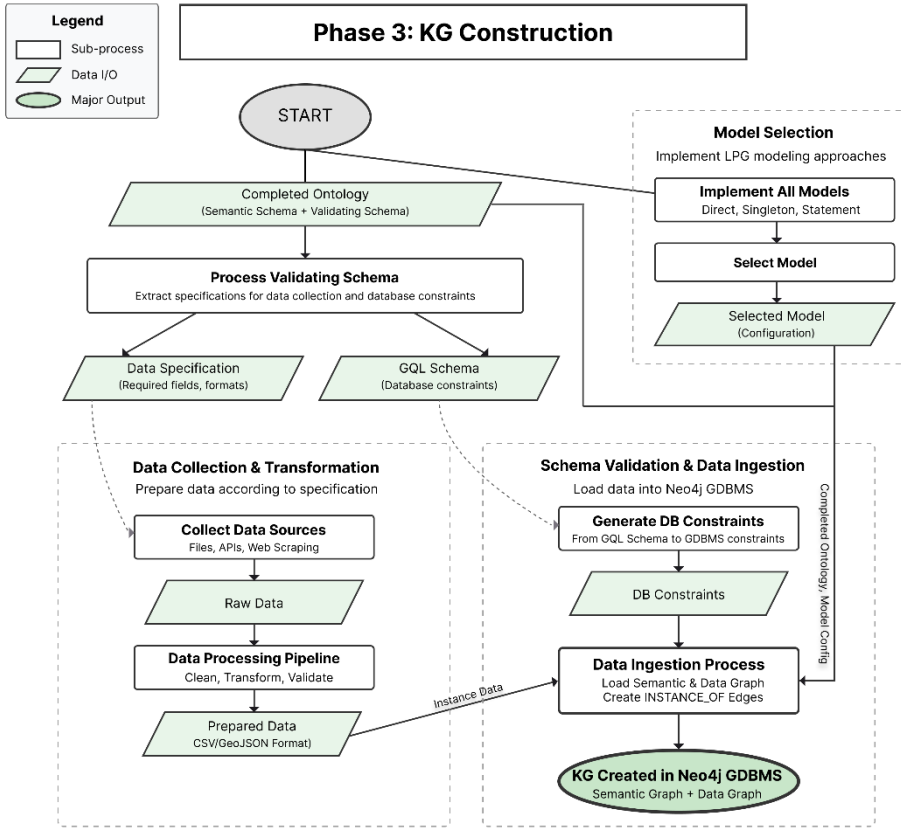
The Formalization process codifies domain knowledge into executable inference rules that enable semantic reasoning and query transformation. These rules, expressed in IF-THEN format, translate abstract user requirements and conceptual queries into concrete database operations. The LLM analyzes the completed ontology structure along with the original CQs to identify patterns requiring rule-based transformation. Each rule specifies conditions that must be satisfied through graph pattern matching in the IF clause and defines derived knowledge or query transformations in the THEN clause. For instance, abstract quality assessments or convenience requirements that cannot be directly queried become rules that map to specific attribute constraints or relationship patterns. Human experts validate the generated rules to ensure they accurately capture domain semantics and produce correct query transformations.

Not all semantic knowledge requires rule-based inference, as straightforward property queries can be answered through direct attribute access without complex transformation. The focus remains on cases requiring semantic interpretation of abstract concepts into measurable conditions.

At the completion of Phase 2, the methodology produces a complete ontology $O = SS \cup VS$ comprising the enriched semantic schema with hierarchical organization and comprehensive axiom coverage, the validating schema with implementation specifications and database mappings, and formalized inference rules for semantic reasoning. This complete ontology serves as the authoritative blueprint governing all subsequent KG construction and query processing, ensuring semantic consistency and enabling efficient database implementation.

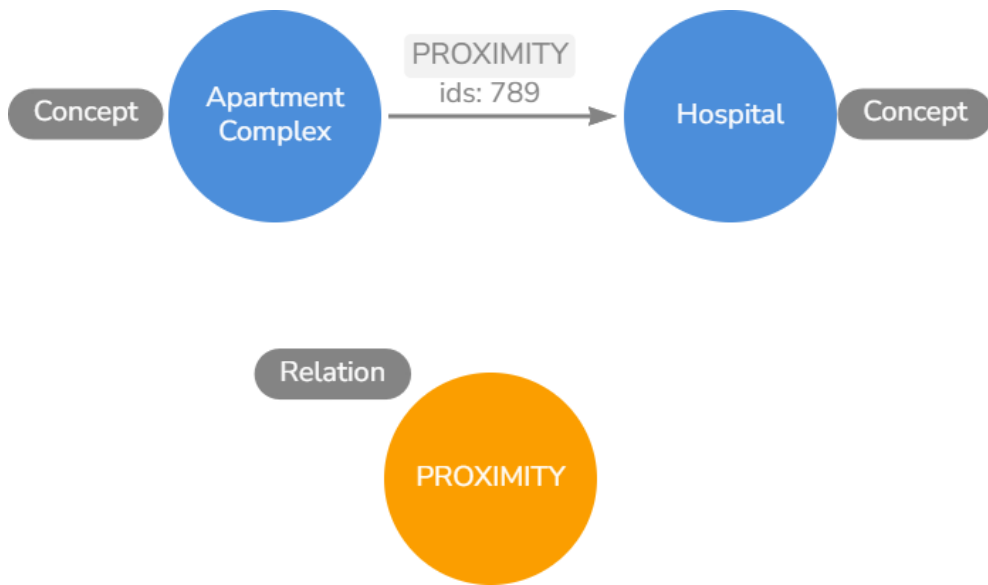
3.5. Semantic & Data Graph Construction

This section presents the Semantic and Data Graph construction, which transforms the completed LPG Ontology into an executable KG within the LPG GDBMS environment. Phase 3 bridges the gap between the abstract ontological design from Phase 2 and the concrete data implementation, establishing both the Semantic Graph G_S containing the ontology structure and the Data Graph G_D actual instance data. As illustrated in Figure 3–5, the pipeline consists of three interconnected components that work in sequence to produce the final integrated KG. The pipeline begins with processing the Validating Schema, which generates two critical outputs that guide all subsequent operations: Data Specification documents defining the required fields, data types, and format requirements for data collection, and the GQL Schema that will be transformed into database constraints. These outputs ensure consistency and completeness throughout the construction phase.

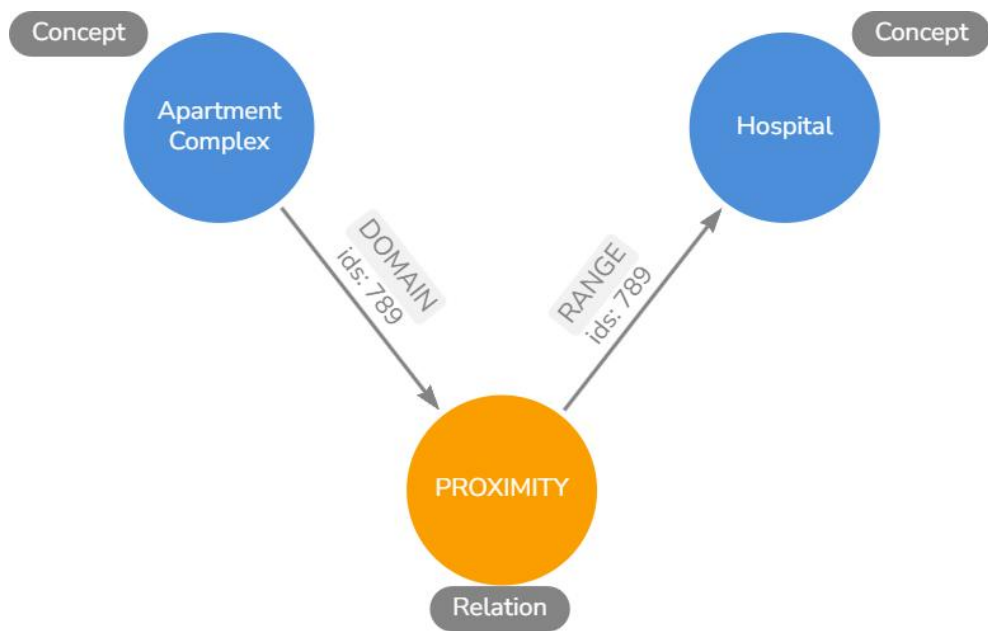


[Figure 3–5] Phase 3: KG Construction pipeline

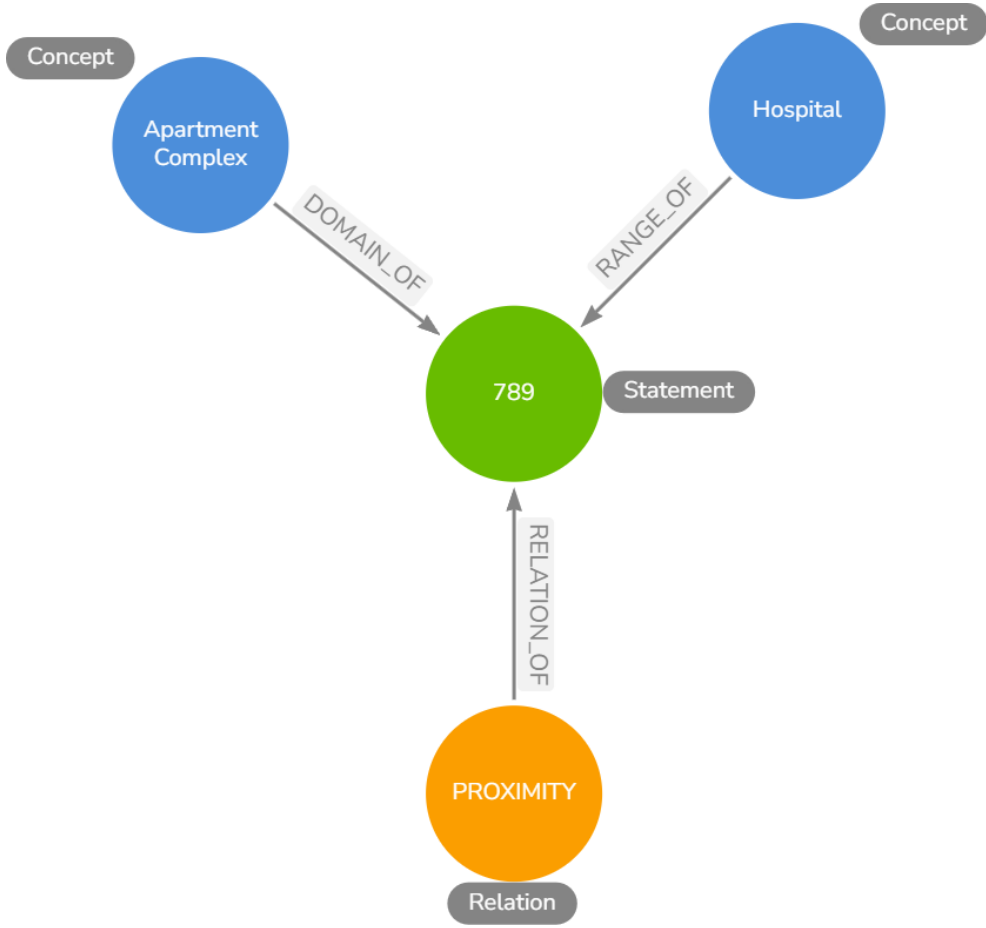
The transformation of ontology into an LPG-compatible format represents a fundamental design decision that significantly impacts both storage efficiency and query performance. While prior work has identified individual LPG modeling patterns (Nguyen et al., 2019), systematic methodologies for integrating ontological structures with instance data within unified LPG representations remain absent. Drawing conceptual inspiration from these works, we design and implement three distinct modeling approaches that each provide systematic solutions for representing both the Semantic Graph G_S and Data Graph G_D within LPG databases. Figures 3–6, 3–7, and 3–8 illustrate how these three approaches represent the same ontological knowledge with different structural patterns.



[Figure 3–6] Direct Modeling



[Figure 3–7] Singleton Modeling



[Figure 3–8] Statement Modeling

Definition 3 (LPG Modeling Approaches). Given the complete ontology $\mathcal{O} = \mathcal{SS} \cup \mathcal{VS}$, three modeling approaches transform it into the Semantic Graph G_S and Data Graph G_D that form the foundation of the MSKG:

- **Direct Modeling (Figure 3–6):** For each axiom $(\omega_1, r, \omega_2) \in \mathcal{Ax}$, create concept nodes and connect them with a typed edge labeled by the relation’s name. In the Semantic Graph G_S , edges directly represent relationships between concepts. In the Data Graph G_D , instance nodes are similarly connected through typed edges. As shown in Figure 3–6, this edge–centric representation follows the natural graph paradigm where relationships serve as direct connectors between entities. All instances maintain INSTANCE_OF connections to their corresponding concepts,

establishing the inter-layer edges E_{SD} .

- **Singleton Modeling (Figure 3–7):** Each relation $r \in R$ becomes a first-class node in the Semantic Graph G_S . For each axiom $(\omega_1, r, \omega_2) \in Ax$, create two edges: a DOMAIN edge from the domain entity to the relation node, and a RANGE edge from the relation node to the range entity. The Data Graph G_D mirrors this structure where instance nodes connect to semantic relation nodes through DOMAIN and RANGE edges, enabling relation-centric queries. Figure 3–7 illustrates this two-hop pattern where the relation node serves as an intermediary hub between domain and range entities.
- **Statement Modeling (Figure 3–8):** For each axiom $(\omega_1, r, \omega_2) \in Ax$, create a dedicated Statement node that serves as a central hub. This Statement node connects to the domain entity via DOMAIN_OF edge, to the range entity via RANGE_OF edge, and to the relation via RELATION_OF edge. The Data Graph G_D follows the same pattern, creating Statement nodes that explicitly capture each instance triple with complete metadata. As depicted in Figure 3–8, this reification structure enables comprehensive provenance tracking by making each triple an explicit entity in the graph.

The key distinction across models lies in how they balance semantic expressiveness with computational efficiency. Direct modeling minimizes traversal depth through single-edge connections but duplicates relation information. Singleton modeling elevates relations to first-class status enabling relation-centric queries but requires two-hop patterns that increase query complexity. Statement modeling maximizes provenance support through explicit triple reification but introduces three-edge patterns that add structural overhead. All three approaches maintain INSTANCE_OF edges ensuring seamless traversal between ontological and instance levels.

The data collection and transformation process is systematically guided by the Data Specification documents extracted from the

Validating Schema. These specifications define the required fields, data types, nullable flags, and format requirements for each entity type, ensuring that all collected data aligns with the ontological structure. The data collection strategy involves gathering raw datasets from authoritative sources relevant to the target domain, with careful attention to temporal consistency and spatial coverage. Raw data undergoes systematic transformation through schema alignment mapping each source to corresponding entities, property extraction enriching attributes according to semantic definitions, and spatial processing preparing geometric data for indexing.

For spatial entities specifically, the spatial processing implements a dual storage approach. All spatial geometries are standardized to Well-Known Text format and stored as geometry properties, enabling identification of spatial nodes $N_{spatial}$. Additionally, centroids are calculated and stored using the database's native point type to enable efficient distance computations. This dual strategy provides flexibility in spatial query execution while serving as the foundation for Phase 4's spatial indexing.

The data quality assurance process addresses missing values according to nullable flags in the Validating Schema. Required properties lead to record exclusion if values are absent, while optional properties are handled through default values or remain null. For spatial entities, any record lacking valid geometry is excluded as spatial properties are fundamental to subsequent operations. All coordinate systems are standardized to ensure compatibility across diverse data sources. This systematic transformation produces standardized output files organized by entity type, with spatial nodes in GeoJSON format, non-spatial nodes in CSV format, and all relational data containing clean and validated records.

The schema validation and data ingestion process begins with transforming the GQL Schema into database-level constraints. Semantic Properties including data types and nullable flags are systematically translated into database constraints, where required properties generate NOT NULL constraints and typed properties create type validation constraints. This automated translation

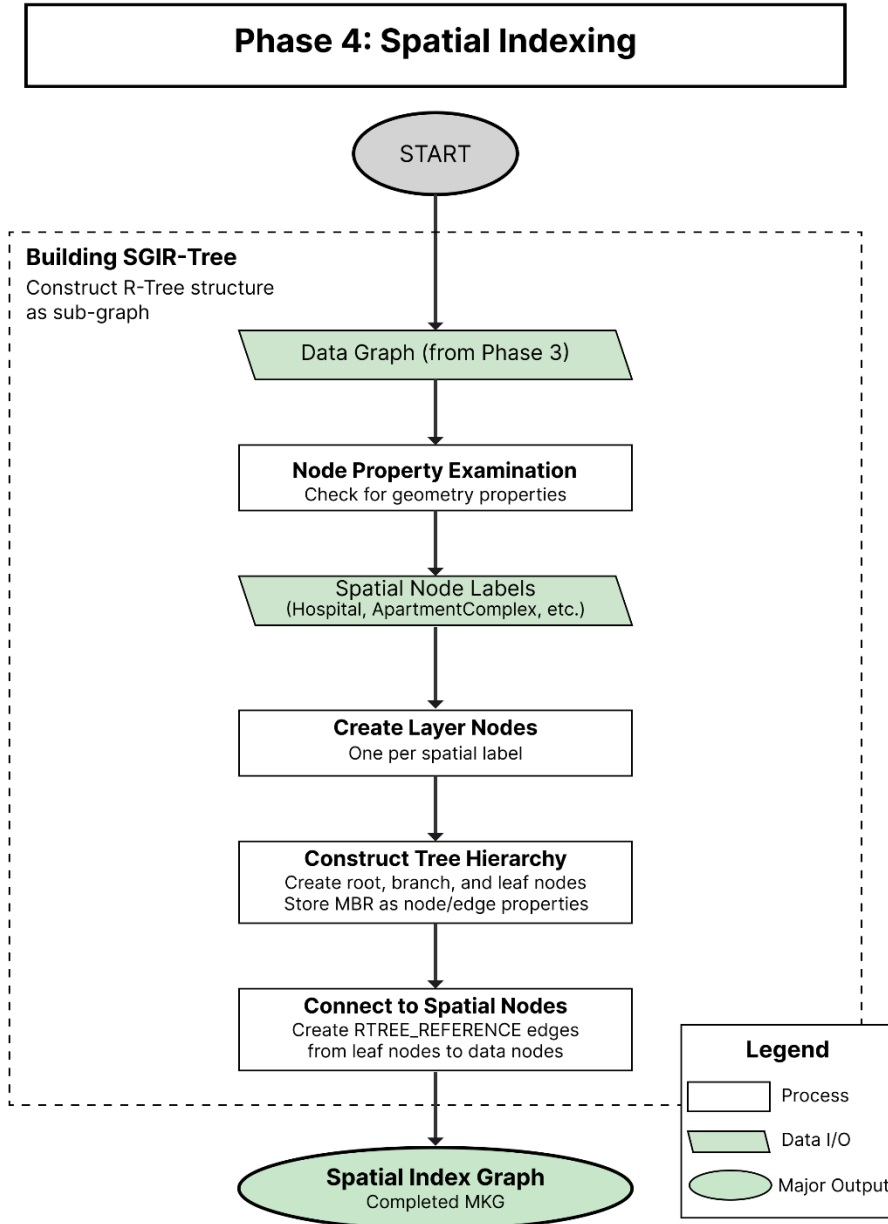
ensures formal ontology specifications are faithfully enforced throughout data ingestion.

The data ingestion follows a four-step sequence. The process first constructs Semantic Graph G_S by creating all concept and relation nodes from Semantic Schema, then loads instance nodes into Data Graph G_D with validation against GQL schema. Subsequently, it establishes instance triples following the selected LPG modeling approach patterns as illustrated in Figures 3-6 through 3-8 and finally creates INSTANCE_OF edges establishing inter-layer connections E_{SD} .

At completion of Phase 3, the methodology produces an integrated MSKG containing Semantic Graph G_S with complete ontological structure, Data Graph G_D with validated instance data, and foundational connections E_{SD} enabling semantic reasoning and query processing across layers. This integrated MSKG serves as the foundation for Phase 4's spatial indexing.

3.6. Spatial Index Graph Construction

This section presents the Spatial Index Graph G_I construction that enables efficient spatial operations within the MSKG architecture. This phase transforms the traditional hierarchical R-Tree structure into a native graph representation, leveraging the SGIR-Tree (Subgraph Integrated R-Tree) methodology to seamlessly integrate spatial indexing capabilities directly within the LPG GDBMS. The integration of spatial indices as first-class graph citizens eliminates the impedance mismatch between external spatial indices and graph traversals, enabling complex spatial-semantic queries to be executed as unified graph operations. As illustrated in Figure 3-9, the pipeline transforms the Data Graph G_D containing spatial entities into the fully integrated Spatial Index Graph G_I through four interconnected processes that identify spatial nodes, construct the SGIR-Tree structure, and enable efficient spatial query processing.



[Figure 3–9] Phase 4: Spatial Indexing Pipeline

3.6.1. Building SGIR–Tree

The initial process identifies all spatial nodes $N_{spatial} \subseteq N_D$ within the Data Graph that require indexing. Rather than performing an exhaustive traversal of the Data Graph, this process leverages the GQL Schema generated in Phase 3 to efficiently identify spatial labels

$L_{spatial}$. As previously defined in Phase 3, spatial nodes are characterized by the presence of geometry properties. The system examines each node type defined in the schema to determine whether a geometry property exists in its specifications, efficiently collecting the set of labels requiring spatial indexing without accessing the actual instance data.

The core of this phase involves constructing the SGIR-Tree structure that transforms the hierarchical R-Tree into a native graph representation. The SGIR-Tree provides a systematic approach to representing spatial indices within the graph database while maintaining the essential properties of R-Trees that enable efficient spatial query processing.

Definition 4 (SGIR-Tree Structure). For each spatial label $l \in L_{spatial}$, an SGIR-Tree is a labeled property subgraph $T_l = (N_{Tl}, E_{Tl}, \phi_{Tl}, \psi_{Tl}, \chi_{Tl}, \lambda_{Tl})$ where N_{Tl} represents the set of index nodes, $E_{Tl} \subseteq N_{Tl} \times (N_{Tl} \cup N_{spatial})$ contains both hierarchical connections and references to spatial data, ϕ_{Tl} and χ_{Tl} store MBRs in nodes and edges respectively, ψ_{Tl} assigns edge types $\{RTREE_ROOT, RTREE_CHILD, RTREE_REFERENCE\}$, and λ_{Tl} maps leaf nodes to their associated spatial entities.

[Table 3-2] MBR computation and storage mechanism

Algorithm 1: MBR computation
Function ComputeNodeMBR(node): 1: if node is leaf then 2: spatial_nodes $\leftarrow \lambda_{Tl}(\text{node})$ 3: envelope $\leftarrow \text{new Envelope}()$ 4: for each $s \in \text{spatial_nodes}$ do 5: envelope.expandToInclude($\phi_{Tl}(s)$) 6: end for 7: else 8: children $\leftarrow \text{GetChildren}(\text{node})$ 9: envelope $\leftarrow \text{new Envelope}()$

```

10:   for each child  $\in$  children do
11:       envelope.expandToInclude( $\phi_{Tl}(\text{child})$ )
12:   end for
13: end if
14: return envelope.getBounds()

```

Function UpdateMBR(node):

```

15: envelope  $\leftarrow$  ComputeNodeMBR(node)
16:  $\phi_{Tl}(\text{node}) \leftarrow$  envelope
17: if  $\exists$  parent: (parent, node)  $\in E_{Tl}$  then
18:     edge  $\leftarrow$  (parent, node)
19:      $\chi_{Tl}(\text{edge}) \leftarrow$  envelope
20: end if
21: return envelope

```

The dual MBR storage strategy stores bounding rectangles in both nodes (ϕ_{Tl}) and edges (χ_{Tl}), enabling efficient query processing through early pruning before node access. Table 3–2 presents the detailed MBR computation and storage mechanisms that govern how spatial bounds are calculated and maintained throughout the tree structure. The RTREE_ROOT edges connect layer nodes to root nodes, RTREE_CHILD edges form the hierarchical tree structure, and RTREE_REFERENCE edges directly link leaf nodes to actual spatial entities in the Data Graph, establishing the inter-layer connections E_{ID} that bridge G_I and G_D . This direct connectivity eliminates the indirection present in traditional spatial databases where leaf nodes store only object identifiers, enabling seamless transitions from spatial filtering to graph traversal operations.

The construction process builds upon the well-established Overlap Minimizing Top-down Bulk Loading algorithm (Lee et al., 2003) while introducing enhancements for graph integration. The process begins by creating a layer node that serves as the unique entry point to the index structure for each spatial label. The algorithm then applies to a top-down partitioning strategy that recursively divides spatial entities into groups while minimizing overlap between

resulting partitions. At each level, the algorithm evaluates potential split positions along both spatial dimensions to create balanced groups with minimal MBR overlap. For internal nodes, the algorithm creates `RTREE_CHILD` edges connecting parent nodes to their children, storing the child's MBR in the edge properties through χ_{TI} for efficient filtering. At leaf level, the algorithm creates `RTREE_REFERENCE` edges directly connecting leaf nodes to spatial entities, completing the integration between the Spatial Index Graph G_I and the Data Graph G_D .

The SGIR-Tree maintenance operations leverage established R-Tree algorithms (Greene, 1989; Guttman, 1984) while adapting them for the graph environment. Insertion operations follow the traditional R-Tree path selection based on minimum MBR enlargement, calculated as $\text{Enlargement}(n, g) = \text{Area}(\text{envelope}_{new}) - \text{Area}(\phi_{TI}(n))$ where envelope_{new} expands the node's MBR to include the new geometry's bounding rectangle. The algorithm traverses from the layer node through the root, evaluating at each level which child node requires the least MBR expansion to accommodate the new geometry. Upon reaching a leaf node, the system creates a direct `RTREE_REFERENCE` edge to the spatial node. When a leaf exceeds maximum capacity, the system triggers a split operation using Greene's method, dividing nodes along the longest dimension to minimize overlap between resulting partitions. MBR updates propagate upward from modified nodes through parent nodes to the root, ensuring all ancestor nodes maintain accurate bounding rectangles.

Deletion operations are simplified compared to traditional R-Trees due to the graph structure's flexibility. The system locates the target leaf through the incoming `RTREE_REFERENCE` relationship and removes it. If removal leaves a leaf empty, the cleanup process propagates upward, removing empty nodes and adjusting MBRs along the path to the root. Bulk operations employ an adaptive strategy based on insertion size relative to tree size. Large insertions that would significantly impact the tree structure trigger complete index rebuilding using the bulk loading algorithm, combining existing and

new nodes for optimal structure. Smaller bulk insertions utilize a seeded clustering approach that distributes new nodes among existing branches based on spatial proximity.

3.6.2. Spatial Query Processing

The SGIR-Tree enables efficient spatial query processing through adaptations of established spatial algorithms to the graph environment. The methodology supports three fundamental spatial operations that form the foundation for complex spatial-semantic queries.

Definition 5 (Spatial Graph Query). Given a spatial index graph G_I with SGIR-Trees T_l for each spatial label $l \in L_{spatial}$, a spatial graph query $Q_{SG} = \langle SP_q, G_q \rangle$ consists of a spatial predicate $SP_q = (op, L_l, param)$ where $op \in \{KNN, Range, Join\}$ specifies the operation type, $L_l \subseteq L_{spatial}$ identifies target spatial labels, and $param$ contains operation-specific parameters, combined with a graph pattern G_q to be matched after spatial filtering. Query processing follows a two-phase approach where spatial predicates identify candidate nodes through the SGIR-Tree, and the λ_{Tl} function provides direct access to spatial nodes for immediate graph traversal without intermediate mapping steps.

The K-Nearest Neighbor (KNN) query processing follows enhanced nearest neighbor search principles (Cheung & Fu 1998) adapted for graph-integrated architecture. The algorithm maintains a priority queue ordered by minimum distance, leveraging the hierarchical SGIR-Tree structure to efficiently prune search space. Starting from the root node, the algorithm performs a depth-first search while calculating minimum distances between node MBRs and the query location. The dual MBR storage through χ_{Tl} enables early pruning by checking edge-stored MBRs before accessing child nodes. Upon reaching leaf nodes, the λ_{Tl} function provides direct access to connected spatial entities. The complete algorithm specification is

provided in Table 3–3, which details the priority queue management and distance–based filtering strategy that ensures k nearest neighbors are identified with minimal node traversals.

[Table 3–3] KNN Query Algorithm

Algorithm: SGIR–Tree KNN Query Processing
Input: Query point loc, Number of neighbors k, SGIR–Tree T_l Output: Set of k nearest spatial nodes <pre> 1: Initialize priority queue PRQ with $(T_l.root, 0)$ 2: Initialize result set $R = \emptyset$, temp_dist = ∞ 3: while PRQ not empty and $R < k$ do 4: (node, dist) $\leftarrow$ PRQ.dequeue() 5: if dist > temp_dist then 6: continue 7: end if 8: if node has RTREE_CHILD edges then 9: for each child c with edge e do 10: child_dist $\leftarrow$ MinDist(loc, $\chi_{T_l}(e)$) 11: if child_dist < temp_dist then 12: PRQ.enqueue(c, child_dist) 13: end if 14: end for 15: else if node has RTREE_REFERENCE edges then 16: for each spatial node $s \in \lambda_{T_l}(node)$ do 17: actual_dist $\leftarrow$ Distance(loc, $s.geometry$) 18: if $R < k$ then 19: R.add(s, actual_dist) 20: if actual_dist < temp_dist then 21: temp_dist $\leftarrow$ max distance in R 22: end if 23: else if actual_dist < temp_dist then 24: Remove node with temp_dist from R 25: R.add(s, actual_dist) </pre>

```

26:      temp_dist ← max distance in R
27:    end if
28:  end for
29: end if
30: end while
31: return R

```

The Spatial Range query implements breadth-first traversal optimized for the SGIR-Tree structure, efficiently filtering spatial nodes based on containment within a query range Q . The algorithm exploits dual MBR storage by first checking edge MBRs through χ_{Tl} before accessing child nodes, enabling early pruning of entire subtrees that do not intersect with Q . At leaf nodes, the algorithm performs two-phase filtering by first checking MBR intersection for quick elimination, then verifying exact geometric containment only for candidates that pass the MBR test. The algorithm can simultaneously query multiple spatial entity types by initializing with multiple layer nodes and processing them in parallel, eliminating separate queries and result merging required by single-label approaches. The detailed algorithm is presented in Table 3-4, demonstrating how the breadth-first strategy systematically explores the tree while maintaining the result set of qualifying spatial nodes.

[Table 3-4] Range Query Algorithm

Algorithm: SGIR-Tree Range Query Processing
Input: Query region Q , SGIR-Tree T_l Output: Set of spatial nodes within Q 1: Initialize queue Queue with T_l .root 2: Initialize result set $R = \emptyset$ 3: while Queue not empty do 4: node ← Queue.dequeue() 5: if node has RTREE_CHILD edges then

```

6:   for each child c with edge e do
7:     if  $\chi_{Tl}(e) \cap Q \neq \emptyset$  then
8:       Queue.enqueue(c)
9:     end if
10:  end for
11: else if node has RTREE_REFERENCE edges then
12:   for each edge e to spatial node s do
13:     if  $\chi_{Tl}(e) \cap Q \neq \emptyset$  then
14:       if  $s.geometry \cap Q \neq \emptyset$  then
15:          $R \leftarrow R \cup \{s\}$ 
16:       end if
17:     end if
18:   end for
19: end if
20: end while
21: return R

```

The Spatial Join operation extends synchronized tree traversal (Brinkhoff et al., 1993), leveraging both dual MBR storage and direct node connections to optimize pair generation and verification. For spatial relationships based on the DE-9IM model (INTERSECTS, CONTAINS, WITHIN, OVERLAPS, TOUCHES, EQUALS, COVEREDBY, COVERS), the algorithm simultaneously traverses two SGIR-Trees while maintaining synchronized exploration. The sorted intersection test orders nodes by minimum x-coordinates, enabling early termination when $n_2.mbr.min_x > n_1.mbr.max_x$, as no further intersections are possible due to monotonic ordering. The algorithm first filters candidate pairs through MBR intersection tests, then applies the specific spatial operation to geometries. The complete spatial join algorithm is detailed in Table 3–5, illustrating the coordinated traversal strategy and the sorted intersection optimization that reduces unnecessary geometric comparisons.

[Table 3–5] Spatial Join Algorithm

Algorithm: SGIR–Tree Spatial Join Processing	
Input: SGIR–Tree T_{l1} , SGIR–Tree T_{l2} , spatial operation op	
Output: Set of spatial node pairs satisfying the join condition	
1:	Initialize queue Queue with $(T_{l1}.root, T_{l2}.root)$
2:	Initialize result set $R = \emptyset$
3:	Initialize JoinOperationExecutor with operation op
4:	while Queue not empty do
5:	$(n_1, n_2) \leftarrow \text{Queue.dequeue}()$
6:	if $\text{Intersect}(\phi_{Tl}(n_1), \phi_{Tl}(n_2)) = \emptyset$ then
7:	continue
8:	end if
9:	if both n_1 and n_2 are internal nodes then
10:	$children_1 \leftarrow \text{GetChildren}(n_1)$, $children_2 \leftarrow \text{GetChildren}(n_2)$
11:	Sort $children_1$ and $children_2$ by minimum x–coordinate
12:	$pairs \leftarrow \text{SortedIntersectionTest}(children_1, children_2)$
13:	for each (c_1, c_2) in pairs do
14:	Queue.enqueue((c_1, c_2))
15:	end for
16:	else if one is leaf and other is internal then
17:	WindowQuery(leaf_node, internal_node, Queue)
18:	else // both are leaf nodes
19:	geometry_pairs $\leftarrow \emptyset$
20:	for each $s_1 \in \lambda_{Tl1}(n_1)$ do
21:	for each $s_2 \in \lambda_{Tl2}(n_2)$ do
22:	if $\text{Intersect}(\text{mbr}(s_1), \text{mbr}(s_2)) \neq \emptyset$ then
23:	geometry_pairs $\leftarrow \text{geometry_pairs} \cup \{(s_1, s_2)\}$
24:	end if
25:	end for
26:	end for
27:	$R \leftarrow R \cup \text{ExecuteSpatialOperation}(\text{geometry_pairs}, op)$
28:	end if
29:	end while

30: return R

At completion of Phase 4, the methodology produces a fully integrated spatial index graph G_I that seamlessly connects to the Data Graph G_D through RTREE_REFERENCE edges, completing the MSKG structure. This integrated architecture enables complex spatial–semantic queries to execute as unified graph traversals that leverage both semantic reasoning from the ontology layer and efficient spatial filtering from the native index layer, eliminating the impedance mismatch and data marshaling overhead inherent in approaches that maintain spatial indices external to the graph database.

Chapter 4. A Case Study of Housing Exploration

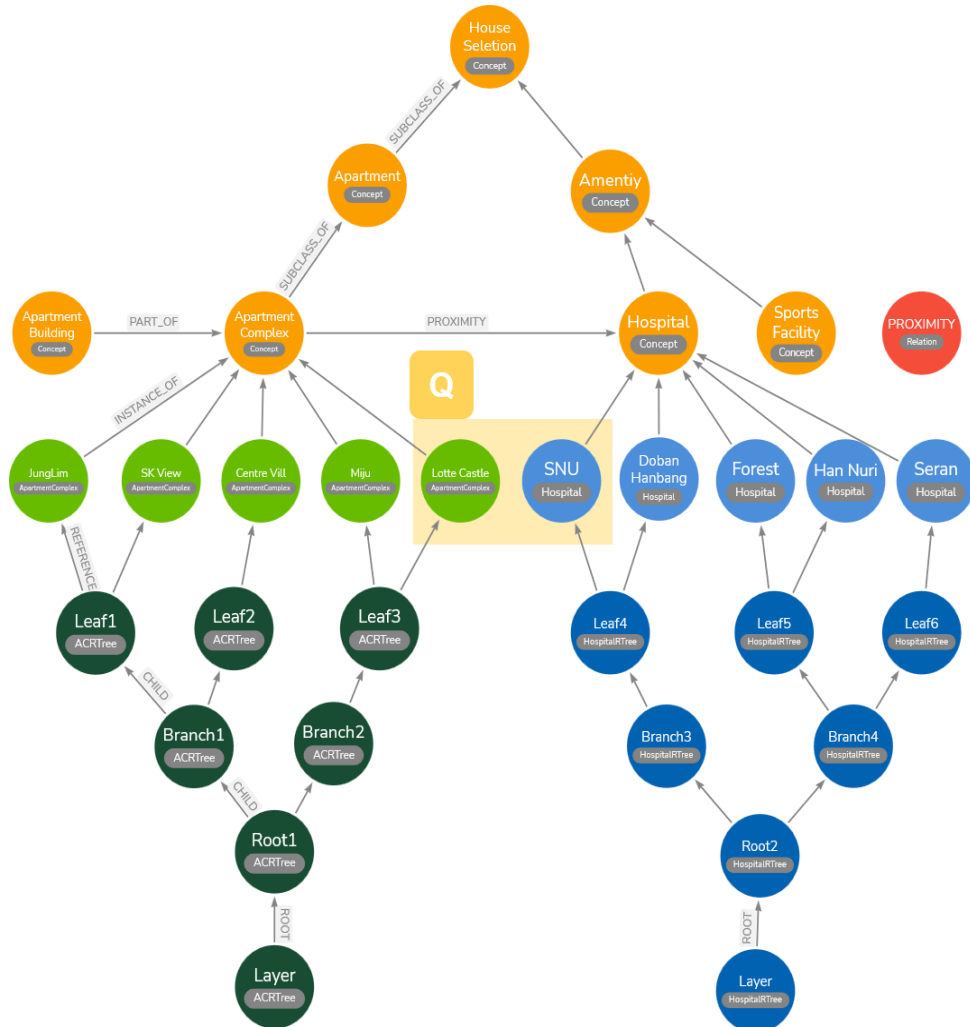
The preceding chapter established domain-independent architecture for constructing spatial KGs in LPG GDBMS environments. This chapter demonstrates architecture's practical applicability through a case study in the housing exploration domain. Housing exploration was selected because it comprehensively encompasses the core challenges our architecture addresses—users express requirements using abstract concepts that must be interpreted through spatial operations combined with semantic inference, the domain involves heterogeneous data sources requiring robust ETL processes, and query diversity enables empirical comparison of LPG modeling approaches. The case study applies to the MSKG architecture to construct an actual spatial KG for Seoul's housing market, evaluating performance across multiple dimensions.

4.1. Application

4.1.1. Housing Exploration Knowledge Graph Construction

The geographical scope encompasses Seoul, South Korea, with specific focus on apartment complexes, targeting data from August 2025 to ensure temporal consistency. This strategic choice reflects unique characteristics of the Korean housing market, where apartments constitute 65.3% of all housing units nationwide according to the 2024 Population and Housing Census by Statistics Korea. Unlike Western condominium models, Korean apartment complexes integrate residential units with comprehensive shared facilities, management systems, and community infrastructure, creating rich relational structures ideal for KG construction. Seoul, as Korea's capital with 9.33 million residents in 605 km² based on the same census, provides unparalleled data richness and urban complexity, representing the full spectrum of Korean residential patterns from luxury complexes in Gangnam to public housing in peripheral districts, with prices ranging from 200 million to over 10

billion KRW.



[Figure 4-1] Running Example Overview

Figure 4-1 illustrates how the MSKG architecture transforms initial domain understanding into structured ontological requirements through its four construction phase. Phase 1 (Specification & Ontology Extraction) begins with generating diverse Buyer Profiles that capture realistic housing seeker characteristics. For instance, one profile might specify "Retired couple in their 60s, budget of 1.9 billion KRW" with considerations for proximity to medical facilities and sports amenities. From these profiles, the methodology

generates Persona Questions such as "We are retired 60s couples. Are there apartments close to good hospitals and sports facilities within 1.9 billion KRW budget?" These complex questions are then decomposed into atomic CQs like "Are there apartments close to good hospitals?" and "Are there apartments within a specific budget?" Simultaneously, the methodology clusters related CQs by semantic similarity into themes such as Location & Accessibility and Budget & Cost. To enable concrete queries, abstract concepts undergo operationalization where qualitative requirements transform into measurable conditions. For example, "apartments close to good hospitals" becomes our running example "Find 3 apartments near a university hospital." Finally, these operationalized questions are abstracted back into Operationalized CQs (OpCQs) for ontology extraction, where the running example decomposes into "Are there apartments within a specific distance of a hospital?" and "Are there hospitals of a specific type?" This systematic process establishes the initial LPG Ontology structure, ensuring that the resulting ontology accurately captures user requirements while remaining queryable through concrete spatial and semantic operations.

Phase 2 (Ontology Completion) completed the LPG Ontology by systematically organizing the extracted elements through graph completion. Hospital was hierarchically structured: General Hospital –SUBCLASS_OF→ Hospital –SUBCLASS_OF→ Medical Facility –SUBCLASS_OF→ Amenity –SUBCLASS_OF→ Neighborhood Environment –SUBCLASS_OF→ House Selection. The four-way schema type classification applied during Validating Schema generation classified Hospital as label type (instantiable as actual hospital nodes), General Hospital as enum type, and PROXIMITY as semantic type to be processed through SGIR-Tree in Phase 4. According to Korean Medical Law, hospitals are classified into six categories, generating axioms such as (Hospital, HAS_TYPE, General Hospital) and (Hospital, HAS_TYPE, Secondary Hospital), but in the housing exploration context, Hospital remains a single concept with type variations stored as properties in instance data rather than proliferating into multiple labels. Inference rules

formalized transformations from abstract user expressions to concrete conditions. For our running example, the rule specifies: IF Query contains "Find 3 apartments near a university hospital" THEN Execute Cypher query with "h:Hospital{type='General Hospital'} AND ac:Apartment Complex AND KNN(h, ac, 3)". In total, 115 rules were produced from 151 CQs, with 36 CQs not requiring rules as they involved simple attribute retrieval without abstract concept interpretation. The completed LPG Ontology $\mathcal{O} = \mathcal{SS} \cup \mathcal{VS}$ provides the formal foundation for subsequent graph construction.

Phase 3 constructed the Semantic Graph G_S and Data Graph G_D by implementing the completed ontology was implemented in Neo4j. While all three LPG modeling approaches were implemented for comparison. Medical facility information collected from Seoul's public data portal passed validating schema checks before loading as instances, with entries like (:Hospital {name: "Seoul National University Hospital", type: "General Hospital", geometry: POINT(...)}) created. Phase 4 completed the MSKG by constructing the Spatial Index Graph G_I through SGIR-Tree by calculating MBRs from spatial attributes of all Hospital and Apartment Complex nodes and indexing them in R-Tree structure, enabling efficient KNN operations for the "Find 3 apartments near a university hospital" query. The resulting three-layer MSKG operates as an integrated structure within the single Neo4j instance, enabling unified spatial-semantic query processing.

Data collection proceeded systematically based on Data Specification documents generated from Validating Schema, collecting 122 diverse raw datasets from 19 major platforms. Public sources included Seoul Open Data Plaza, Public Data Portal, Ministry of Land Infrastructure databases providing apartment information, transaction prices, and administrative boundaries. Data cleaning proceeded through deduplication based on unique identifiers and spatial proximity thresholds, missing value imputation following Validating Schema nullable flags, and spatial data-specific operations detecting invalid geometries. For apartment unit internals, a generation process created realistic room layouts based on Korean

housing standards and typical apartment configurations. VISIBLE relations were selectively generated for key locations using simplified line-of-sight calculations where elevation data was available, with comprehensive 3D visibility analysis across the entire dataset acknowledged as beyond study scope. Other synthetic elements included certain inter-entity spatial relationships requiring extensive field surveys or manual verification.

Regarding node granularity decisions, rooms and windows were represented as independent nodes rather than properties to support specific CQs. For instance, queries like "Is there an apartment with good natural light?" required operationalization into conditions such as "window size / outer wall size $\geq$ certain threshold," necessitating separate nodes to enable appropriate aggregation and filtering operations. While this design choice significantly increased node count (rooms: 2.76 million nodes), it enabled the architecture to handle complex housing exploration queries that consider interior characteristics like ventilation and natural lighting. The primary contribution lies in the semi-automatic KG construction methodology utilizing LPG Ontology and MSKG architecture rather than exhaustive dataset completeness, with synthetic data demonstrating architecture capability while transparently acknowledging practical limitations.

The final constructed housing exploration spatial KG follows the ontological structure from Phase 2 with 246 concepts, 23 relations, 648 axioms, 204 property axioms, and 115 inference rules. Using Direct LPG modeling, the final KG contains approximately 22.5 million nodes and 46.5 million edges spanning Seoul's 467 administrative districts with approximately 2,000 apartment complexes.

4.1.2. Experimental Setup

We conducted experiments on a system equipped with an Intel Core i7-8700 processor operating at 3.20GHz, 32GB of DDR4 RAM. The software environment consisted of Windows 11 Professional (64-bit) as the operating system, with Neo4j Desktop Enterprise 2.0.3 and Neo4j version 2025.07.1 serving as the primary LPG

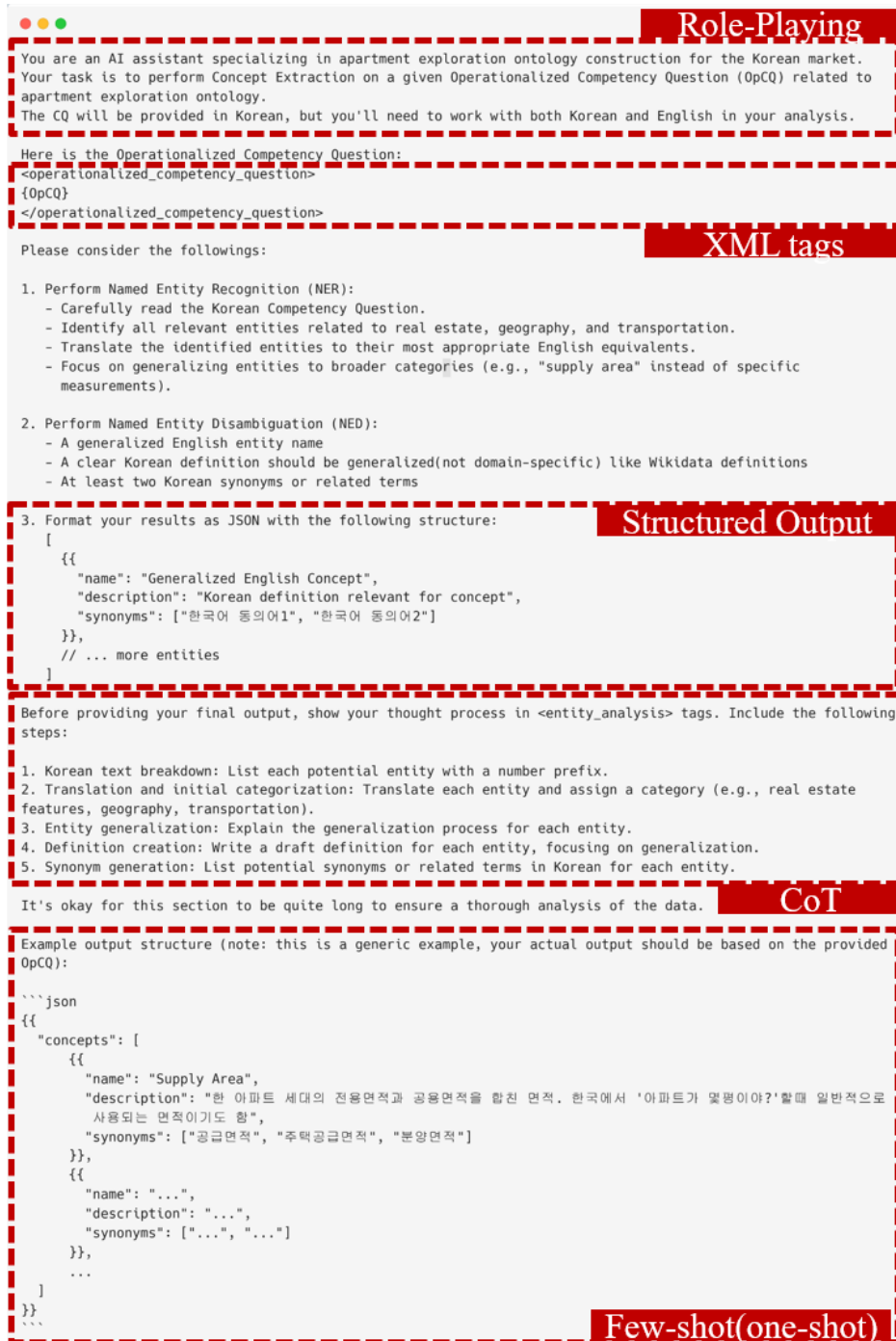
GDBMS platform. For ontology construction and refinement, we utilized OpenAI's GPT-o4-mini model and text-embedding-3-large model (as of May 2025) accessed through LangChain 0.3.23 framework with the reasoning-high configuration, chosen for its optimal balance between cost-effectiveness and output quality for structured knowledge extraction tasks.

The implementation leveraged Python 3.12 for data preprocessing and transformation pipelines, incorporating pandas 2.2.3 and geopandas 1.1.1 for tabular and geospatial data manipulation respectively. Spatial data processing utilized QGIS 3.34.14 for coordinate system standardization and geometry validation. The Neo4j Python driver 5.28.1 facilitated database interactions, with Cypher queries dynamically generated based on the ontological structures defined in the architecture. For the user interface supporting human-in-the-loop refinement, we developed a Streamlit 1.44.1 web application that enabled domain experts to review and modify LLM-generated outputs in a tabular format, ensuring consistency and quality throughout the semi-automatic construction process.

We implemented the SGIR-Tree spatial indexing component as a Neo4j plugin using Java 21, directly extending the database's native capabilities without requiring external spatial libraries. All experiments were executed with the Neo4j heap memory configured to 8GB and page cache set to 4GB, with query caches cleared between experimental runs to ensure consistent and reproducible measurements.

4.1.3. LLM Prompting Strategy

The effectiveness of semi-automatic ontology construction critically depends on the quality of prompts used to guide the LLM. We employed a systematic prompting strategy combining multiple established techniques to maximize output quality and consistency across different phases of the methodology.



[Figure 4-2] LLM Prompting Strategy for Concept Extraction

As illustrated in Figure 4-2, our prompting strategy incorporates four key techniques. First, role-playing prompting assigns specific expertise to the LLM by instructing it to act as "You are an AI

assistant specializing in apartment exploration ontology construction for the Korean market," establishing the appropriate context and vocabulary for domain-specific knowledge extraction (Xu et al., 2023).

Second, few-shot prompting provides 0–2 concrete examples depending on the task complexity (Brown et al., 2020). For straightforward extraction tasks, zero-shot prompting proved sufficient given the detailed task descriptions. For more complex tasks requiring specific output formats, we provided 1–2 examples demonstrating the expected structure. As shown in Figure 4–1, the concept extraction prompt includes one generic example illustrating the JSON output format with concepts like "Supply Area" to guide the model toward appropriate Korean real estate terminology.

Third, structured output prompting using XML tags and JSON structure enforces consistent formatting across all LLM outputs (M. X. Liu et al., 2024; Yamauchi et al., 2023). The prompt clearly delineates different sections using tags such as `<operationalized_competency_question>` for input and requires JSON-formatted output with specific fields like "name", "description", and "synonyms". This structured approach enabled reliable parsing and validation of generated content across hundreds of CQs and their transformations.

Fourth, Chain-of-Thought (CoT) prompting enhances the model's reasoning capabilities by explicitly requesting step-by-step analysis before generating final outputs (Wei et al., 2022). As shown in the prompt, the instruction "Before providing your final output, show your thought process in `<entity_analysis>` tags" ensures the model systematically considers Korean text breakdown, translation, generalization, and definition creation for each concept, resulting in more accurate and contextually appropriate extractions.

The prompting strategy incorporated domain-specific guidelines tailored to Korean housing terminology. Special attention was given to properly handling Korean-specific terms and their English equivalents, as demonstrated in the example where "전용면적" is correctly identified as "Supply Area" with appropriate Korean context

preserved in the description. Temperature parameters were set to 0.0 to ensure maximum consistency and determinism in structured outputs, eliminating randomness that could introduce inconsistencies across multiple extraction iterations.

4.1.4. Experimental Design

The experimental evaluation follows a multi-faceted approach designed to assess both the quality of the constructed KG and the performance of query processing across different dimensions. The experiments are organized into four primary categories, each addressing specific aspects of the architecture's effectiveness and employing distinct evaluation metrics summarized in Table 4-1.

KG Quality Assessment evaluates the completeness, consistency, and validity of the semi-automatically constructed ontology and its instantiation. In ontology engineering, CQ-based evaluation is an established methodology when golden standard ontologies are unavailable (Gruninger & Fox, 1995; Hogan et al., 2022; Suárez-Figueroa et al., 2012). While prior housing ontology research exists (Cho & Kang, 2011; Laddada et al., 2020; Hong, 2025), these ontologies are either not publicly available for reuse, focus on specific aspects such as decision support or neighborhood characteristics rather than comprehensive exploration requirements. Therefore, we focus on metrics evaluable on the generated KG itself. Completeness of Coverage validates whether the KG can answer its design questions by executing all 151 CQs and measuring the success rate, applied in Section 4.2.1. Construction Efficiency Assessment compares storage and time requirements across different LPG modeling approaches. Storage Efficiency measures disk space consumption (GB) for Direct, Singleton, and Statement models, while Construction Time captures elapsed time (seconds) for complete KG building from prepared data files. These metrics are critical for assessing practical deployment viability and determining update strategies. LPG Modeling Query Processing Performance Analysis investigates the impact of LPG modeling choices on complex query execution. Five representative cluster queries (C1-C5) are

executed across all three modeling approaches (Direct, Singleton, Statement), measuring Query Execution Time and Memory Overhead.

Spatial Indexing Performance Evaluation examines SGIR-Tree efficiency compared to baseline approaches, measuring both index construction overhead and query execution performance. Index Size quantifies storage consumed by spatial indices (MB), and Index Construction Time measures spatial index building duration (seconds), determining whether daily or real-time updates are feasible, both assessed in Section 4.2.2. Query execution performance is evaluated across three fundamental spatial operations: spatial join with DE-9IM operation, K-nearest neighbors with K ranging from 1 to 125, and spatial range queries with selectivity from 0.0001 to 0.1. SGIR-Tree is compared against two baselines: a disconnected spatial index duplicating spatial nodes within Neo4j, and an Out-DBMS approach using STR-Tree from Java Topology Suite (JTS).

Multi-Layer Integration Evaluation demonstrates how the multi-layer architecture operates cohesively for complex queries simultaneously requiring ontological reasoning, instance data traversal, and spatial computation. Five representative queries per cluster are designed to exercise all three graph layers, with evaluation measuring execution time, result counts, and analyzing specific multi-layer integration patterns, presented in Section 4.2.3.

Query Performance Metrics measure the practical implications of design choices on system responsiveness and scalability, essential for interactive housing exploration applications. Query Execution Time captures database processing latency from query receipt to result generation (ms), measured via IntelliJ profiler or Neo4j PROFILE command depending on comparison scenario. Memory Overhead quantifies heap memory consumption during execution (MB), directly impacting the number of concurrent users the system can support.

All performance measurements employ a hybrid cache protocol combining cold-cache initialization with hot-cache execution. For each distinct query, the Neo4j database is restarted to ensure cold-

cache conditions between different queries. Each query is then executed three consecutive times, with the first execution representing cold-cache performance and subsequent executions benefiting from hot-cache, and results are averaged across all three runs. This hybrid approach balances realistic assessment of cache warming effects within repeated query patterns against fair comparison across different query types. Table 4–1 maps each metric to specific evaluation sections, providing a roadmap for the subsequent experimental analysis.

[Table 4–1] Evaluation Metrics and Their Application Sections

Metric Category	Metric	Description	Applied in Sections
KG Quality	Completeness of Coverage	Percentage of CQs returning meaningful results	4.2.1
Construction Efficiency	Storage Efficiency	Disk space required for KG storage (GB)	4.2.1
	Construction Time	Elapsed time for KG building (seconds)	4.2.1
	Index Size	Storage consumed by spatial indices (MB)	4.2.2
	Index Construction Time	Time for spatial index building (seconds)	4.2.2
Query Performance	Query Execution Time	Database latency for query completion (ms)	4.2.1–4.2.3
	Memory Overhead	Heap memory consumption during execution (MB)	4.2.2

4.2. Evaluation

This section evaluates the results of applying architecture to the housing exploration domain from three perspectives. First, it assesses the quality of the constructed KG and the efficiency of the three LPG modeling approaches (4.2.1). Second, it compares and analyzes the performance of the SGIR-Tree spatial indexing technique against existing methods (4.2.2). Third, it verifies whether the three-layer structure of the semantic-data-spatial index operates integrally through complex queries (4.2.3).

4.2.1. Knowledge Graph Construction Results

This sub-section presents the evaluation results of the semi-automatically constructed spatial KG, assessing both the quality of the ontology-driven construction and the efficiency of different LPG modeling approaches.

The completeness of our KG was evaluated by testing its ability to answer the 151 CQs that defined the ontology scope. Each CQ was executed against the populated Neo4j database containing Seoul housing data. A CQ was considered successfully covered if it returned at least one meaningful result.

Out of 151 total CQs, 146 returned meaningful results, achieving a success rate of 96.7%. This high coverage demonstrates that our semi-automatic construction process successfully captured the essential concepts and relationships needed for practical housing exploration. The five queries that failed provide insights into data availability challenges rather than ontological limitations:

1. **Private Garden Query (CQ-8):** Failed due to data quality issues during preprocessing. While 16 terrace houses with private gardens existed in Seoul as of 2021, these instances were excluded during data preparation due to missing critical attributes including absent geometry data, missing UUIDs, and incomplete records in the source datasets.
2. **Multicultural Community Center Query (CQ-51):** Failed despite the existence of 26 multicultural community centers in Seoul. The

query specifically sought centers located within apartment complexes using the within spatial predicate, but none of the 26 centers were spatially contained within apartment complex boundaries. This reflects the actual geographic distribution pattern where such centers are typically established in separate public buildings rather than within apartment complexes themselves.

3. **Lunchbox Store Facility Query (CQ-54):** Failed due to insufficient granularity in spatial containment relationships. While commercial facilities are linked to apartment complexes, the nested spatial relationship (lunchbox stores PART_OF commercial facilities PART_OF apartment complexes) required for this query was not captured in public data sources.
4. **High Tech Industrial Complex Query (CQ-86):** Failed because high-tech industrial complexes exist in the greater Seoul metropolitan area (e.g. Pangyo, Bucheon, etc.) but not within Seoul's administrative boundaries. The ontology correctly models this concept, but no instances exist in our geographically scoped dataset.
5. **High Investment Value Apartment Query (CQ-91):** The query sought apartments with an average annual price increase rate of 5% or higher over the past 5 years. This failed due to temporal data coverage limitations, as our transaction dataset contains only 1 year of real estate transaction records (2023-2024), insufficient for calculating 5-year price trends.

The failure analysis reveals that all five unsuccessful queries stem from data availability and quality constraints rather than ontological deficiencies. The first query failed due to data quality issues where valid instances were excluded during preprocessing. The second and fourth queries highlight geographic and spatial distribution limitations where relevant entities exist but not within the dataset's scope or expected spatial relationships. The third query demonstrates the challenge of capturing fine-grained spatial hierarchies from public data sources. The fifth query illustrates

temporal coverage constraints in longitudinal analysis. Importantly, all failed queries executed correctly and returned empty results rather than producing errors, demonstrating the robustness of the schema design. These failures underscore the practical challenges of constructing comprehensive spatial KGs from public data sources, where achieving complete coverage requires not only sound ontological design but also sustained efforts in data collection, quality assurance, and temporal data maintenance.

The comparison of three LPG modeling approaches—Direct, Singleton, and Statement—exposes significant trade-offs between storage efficiency, semantic expressiveness, and query performance. It is important to note that this evaluation focuses exclusively on the KG representation layer (G_S and G_D as defined in Section 3.2), as spatial indexing operates independently of the chosen LPG modeling approach. Thus, spatial index performance metrics are evaluated separately in Section 4.2.2. Table 4–2 presents the storage requirements for each modeling approach when representing the identical KG content.

[Table 4–2] Storage Comparison for LPG Modeling Approaches

LPG Model	DB Storage (GB)	# of Nodes	# of Edges	Storage Ratio
Direct	8.14	22,379,614	45,375,098	1.00×
Singleton	20.5	22,379,614	90,750,196	2.52×
Statement	15.2	67,754,712	136,125,294	1.87×

The Direct model exhibits optimal storage efficiency at 8.14GB, serving as our baseline. Surprisingly, the Singleton model incurs the highest overhead at 20.50GB (2.52× increase) despite having fewer nodes and edges than the Statement model. This counterintuitive result stems from the Singleton model's property duplication strategy: each relationship requires storing the triple UUID redundantly in both DOMAIN and RANGE edges. Additionally, Neo4j's internal storage architecture imposes higher per-edge overhead for property-rich

edges.

The Statement model requires 15.20GB ($1.87\times$ increase), introducing 45.4 million additional Statement nodes for triple reification. However, this node-centric approach proves more storage-efficient than edge-centric property duplication. Each Statement node stores the UUID once, while connected edges remain lightweight with minimal properties.

When normalized to per-entity metrics, interesting patterns emerge: bytes per edge are 192.67 (Direct), 242.67 (Singleton), and 120.64 (Statement), while bytes per entity measure 129.03, 194.66, and 80.55 respectively. The Statement model, despite increasing total storage, achieves the best per-entity efficiency through its reification structure, suggesting it scales more favorably for KGs requiring extensive metadata attachment.

Table 4-3 presents the time required to build each LPG model from prepared GeoJSON/CSV source files, measuring the complete loading pipeline.

[Table 4-3] KG Construction Time by LPG Models

LPG Model	ontology construction time(s)	Instance construction time(s)	linking time(s)	Total Time (s)
direct	2.78	2776.38	1297.71	4,076.87
singleton	1.49	3683.11	3072.92	6,757.52
statement	2.33	3380.15	4210.20	7,592.68

The construction time analysis reveals performance patterns directly correlated with model complexity. The Direct model establishes the baseline at 4,076.87 seconds (approximately 68 minutes), while Singleton and Statement models require $1.66\times$ and $1.86\times$ longer respectively, reflecting their increased structural overhead.

Breaking down the three construction phases reveals where these differences originate. Ontology construction completes rapidly across all models (1.49–2.78s) as it only builds the semantic schema

independent of data volume. Instance construction, which creates the actual data, shows moderate variation: Direct (2,776.38s) creates simple typed edges, Singleton (3,683.11s) adds edge nodes with DOMAIN/RANGE relationships, and Statement (3,380.15s) introduces intermediate Statement nodes. However, the most significant performance gap emerges in the linking phase where instances connect to ontological classes. Here, the Direct model's simple entity-to-class edges (1,297.71s) contrast sharply with Singleton's two-edge connections (3,072.92s, $2.37\times$ slower) and Statement's three-edge-plus-node structure (4,210.20s, $3.24\times$ slower). This dramatic difference stems not only from the increased graph operations but also from the MATCH queries required to verify existing connections when processing millions of instances.

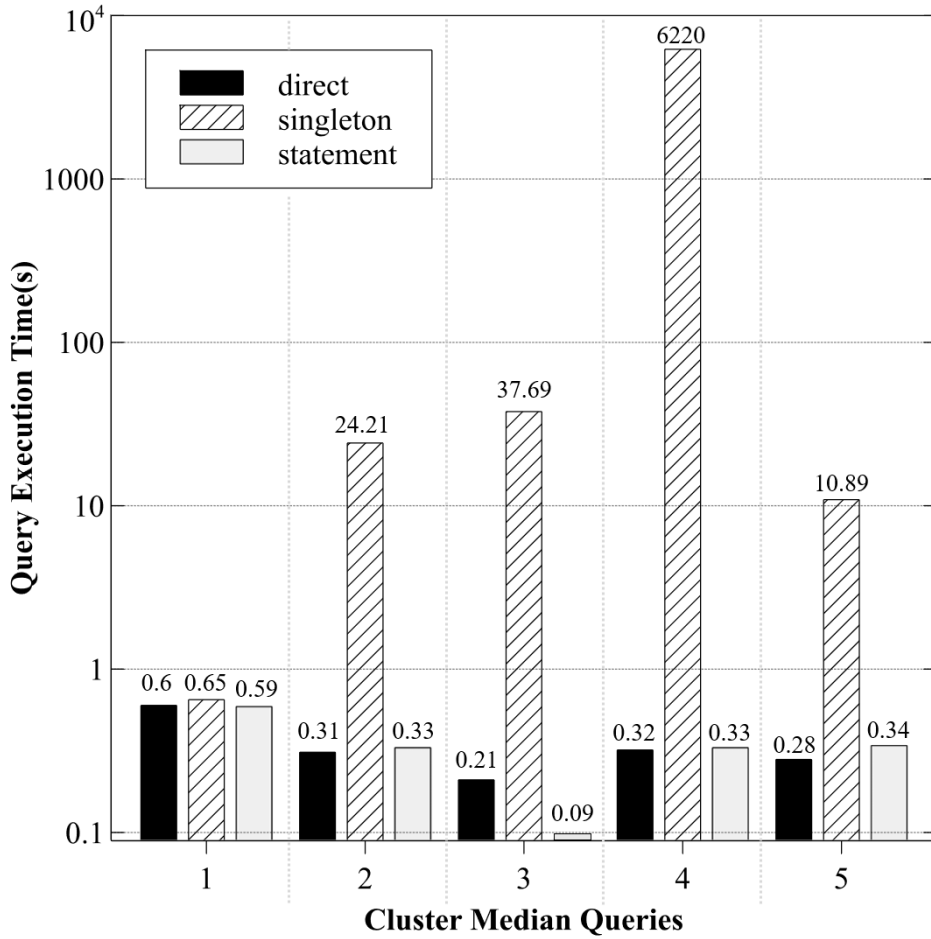
These timing characteristics directly impact deployment strategies. For dynamic housing markets requiring daily updates with new listings and price changes, the Direct model's efficiency becomes essential. Conversely, some scenarios that prioritize complete meta data tracking may justify the Statement model's overhead.

To comprehensively evaluate query performance across modeling approaches, we tested five representative cluster queries (C1–C5) that cover different aspects of housing exploration. These queries were selected as median-complexity representatives from each cluster after sorting the CQs by execution time, as the longest-running queries on the Singleton model failed to complete within 24 hours. The experimental setup utilized identical hardware configurations and Neo4j database instances to ensure fair comparison across the Direct, Singleton, and Statement modeling approaches.

[Table 4–4] Representative Cluster Queries

Cluster	Cluster Name	CQ ID	Query Description
C1	Location & Accessibility	284	Are there apartments with good commercial area accessibility?

C2	Family & Living Environment	435	Are there apartments that provide fancy living?
C3	Budget & Cost Efficiency	136	Are there apartments with low monthly rent?
C4	Apartment Features	3	Are there luxury apartments?
C5	Safety & Security	37	Are there apartments with security facilities?



[Figure 4–3] Query Execution Time Comparison Across LPG Models

The performance evaluation reveals striking disparities between the modeling approaches, with differences spanning several orders of magnitude as illustrated in Figure 4–3. The complete query implementations for each model are provided in Appendix A (Table

A-1), demonstrating the structural differences that contribute to these performance characteristics.

For location-based queries (C1), all three models performed comparably with execution times clustered around 0.6 seconds. The Direct model achieved 0.60s, while Singleton and Statement models completed in 0.65s and 0.59s respectively. This consistency across models stems from the fundamental nature of this query, which does not involve any graph traversal operations. Instead, it relies entirely on Neo4j's built-in spatial function (`point.distance`) to calculate distances between apartment complexes and commercial areas. Since all three models store spatial properties identically as node attributes, and the query simply performs distance calculations without navigating relationships, the modeling approach has minimal impact on performance. The slight variations observed likely result from query parsing overhead rather than fundamental structural differences.

The most dramatic performance divergence emerged in complex feature queries (C4), where the Singleton model required 6,220 seconds—approximately 1 hour and 44 minutes—compared to the Direct model's 0.32 seconds, representing a staggering 19,683-fold performance degradation. This query searches for luxury apartments with specific amenities, requiring multiple relationship traversals and property checks. The Singleton model's representation of each relationship as separate DOMAIN and RANGE edges creates an exponential explosion in the search space. As shown in the query implementation (Appendix A, Table A-1), the Singleton version must navigate through UUID-linked relation nodes, perform multiple joins to maintain relationship consistency, and handle complex nested EXISTS clauses that severely impact the query optimizer's ability to prune the search space effectively.

Interestingly, the Statement model demonstrated marginally better performance in budget-related queries (C3), achieving 0.09 seconds compared to Direct's 0.21 seconds and Singleton's 37.69 seconds. For this one-hop relationship query between Apartment Unit and Transaction nodes, the Statement model's reification

structure can occasionally provide more efficient access patterns, as the Statement nodes serve as indexed intermediaries that the query optimizer can leverage effectively. However, it is important to note that the absolute difference between Direct and Statement models (0.12 seconds) is relatively minor in practical terms, with both maintaining sub-second performance suitable for interactive applications. The more significant finding is the Singleton model's 178-fold performance degradation, which demonstrates its fundamental unsuitability even for simple one-hop traversals due to the UUID-based relationship matching overhead.

The family and living environment queries (C2) revealed another significant performance gap, with the Singleton model requiring 24.21 seconds versus sub-second performance for both Direct (0.31s) and Statement (0.33s) models. This query searches for apartments with luxury amenities including concierge services, proximity to department stores, and recreational facilities. Similarly, security-related queries (C5) showed a 39-fold performance penalty for Singleton (10.89s) compared to Direct (0.28s), while Statement maintained competitive performance at 0.34 seconds. These results consistently demonstrate that the Singleton model's approach of unpacking every semantic relationship into separate nodes and edges prevents effective utilization of graph database indexing and traversal optimizations.

Based on these empirical findings, we can provide clear recommendations for LPG model selection in practical applications. The Direct model remains the optimal choice for general-purpose applications requiring consistent sub-second response times across diverse query types. The Statement model presents a viable alternative when transaction-level metadata or relationship attribution is paramount, particularly excelling in scenarios involving heavily annotated edges. The Singleton model, despite its theoretical semantic expressiveness, proves unsuitable for any interactive application due to its severe performance penalties, and should be reserved exclusively for offline analytical workloads where semantic completeness outweighs runtime considerations.

4.2.2. Spatial Indexing Performance

This sub-section evaluates the performance characteristics of the SGIR-Tree spatial indexing implementation within the Neo4j LPG GDBMS, validating its effectiveness for spatial-graph query processing in housing exploration scenarios. The evaluation follows a comprehensive two-phase approach: first, we assess the overhead associated with index construction and storage to establish the baseline costs of spatial graph indexing; second, we systematically measure query execution performance across three fundamental spatial operations—spatial join, KNN, and spatial range queries—that form the building blocks of complex housing exploration queries. To contextualize the SGIR-Tree's performance advantages, we compare it against two baseline approaches: a disconnected spatial index that maintains spatial nodes separately within Neo4j, and an Out-DBMS approach using the STR-Tree implementation from the JTS library. These comparisons isolate the specific benefits of native integration by demonstrating how the SGIR-Tree's subgraph architecture eliminates data marshaling overhead, enables seamless spatial-graph query composition, and leverages the GDBMS query optimizer for holistic execution planning.

The storage and construction efficiency of spatial indices directly impacts the viability of integrating spatial capabilities for housing exploration. This section compares three indexing approaches implemented on the Seoul housing dataset containing 822,550 spatial nodes from 59 point-type spatial labels and 119,535 spatial nodes from 16 polygon-type spatial labels, totaling 942,085 spatial entities representing apartments, buildings, facilities, and administrative boundaries across Seoul.

[Table 4-5] Spatial Index Storage and Construction Comparison

Index Type	Storage Size (MB)	Size Ratio	Construction Time (s)	Time Ratio
SGIR-Tree	170.12	1.0 ×	28.75	1.0 ×

Disconnected	478.30	$2.81 \times$	35.43	$1.23 \times$
Out-DBMS (STR-Tree)	72.90	$0.43 \times$	33.06	$1.15 \times$

The storage characteristics reveal distinct architectural trade-offs. The SGIR-Tree consumes 170.12 MB, representing a balanced approach between the space-efficient STR-Tree (72.9 MB) and the storage-intensive disconnected index (478.3 MB). While the Out-DBMS STR-Tree achieves minimal storage at 43% of SGIR-Tree's size, this efficiency comes from its traditional R-Tree structure that stores only MBR boundaries and child pointers. In contrast, SGIR-Tree's $2.3 \times$ overhead stems from its graph-native storage model: each R-Tree node stores node labels for type-based filtering, edge properties containing MBR information for precise spatial predicates, and direct connections to spatial entities through RTREE_REFERENCE edges. The disconnected index exhibits the highest storage overhead at $2.81 \times$ the SGIR-Tree size, consuming 478.3 MB. This substantial overhead results from its architectural requirement to duplicate all spatial nodes within Neo4j to maintain isolation between the spatial index structure and the main graph.

Construction performance reveals the advantages of native integration. The SGIR-Tree achieves the fastest construction at 28.75 s, outperforming both the disconnected index (35.43 s, 23% slower) and the Out-DBMS approach (33.07 s, 15% slower). This performance advantage emerges from direct access to Neo4j's internal storage during construction, eliminating the overhead of node duplication required by the disconnected approach and the data marshaling costs incurred by external index synchronization. Native integration enables efficient processing of multiple spatial entity types without data transformation steps.

The storage overhead analysis reveals critical operational implications. SGIR-Tree's additional 97.22 MB compared to STR-Tree enables several performance optimizations: dual MBR storage in leaf-to-spatial edges reduces refinement steps during query execution, integrated label storage eliminates type checking

overhead during traversal, and native graph edges enable direct spatial-to-semantic query transitions without intermediate mapping structures. These design decisions trade $2.3\times$ additional storage over the minimal STR-Tree for substantial query performance gains reasonable exchange given modern storage economics and the performance-critical nature of interactive housing exploration applications.

We evaluate the standalone performance of SGIR-Tree across three fundamental spatial operations, measuring execution time and memory overhead as query parameters vary. These experiments assess only the spatial component SP_q with to $G_q = \emptyset$ isolate pure spatial indexing performance before comparing with baseline approaches in subsequent sections. The three operations represent distinct query patterns common in housing exploration: spatial join for topological relationship analysis, KNN for proximity-based searches, and spatial range for area-based filtering.

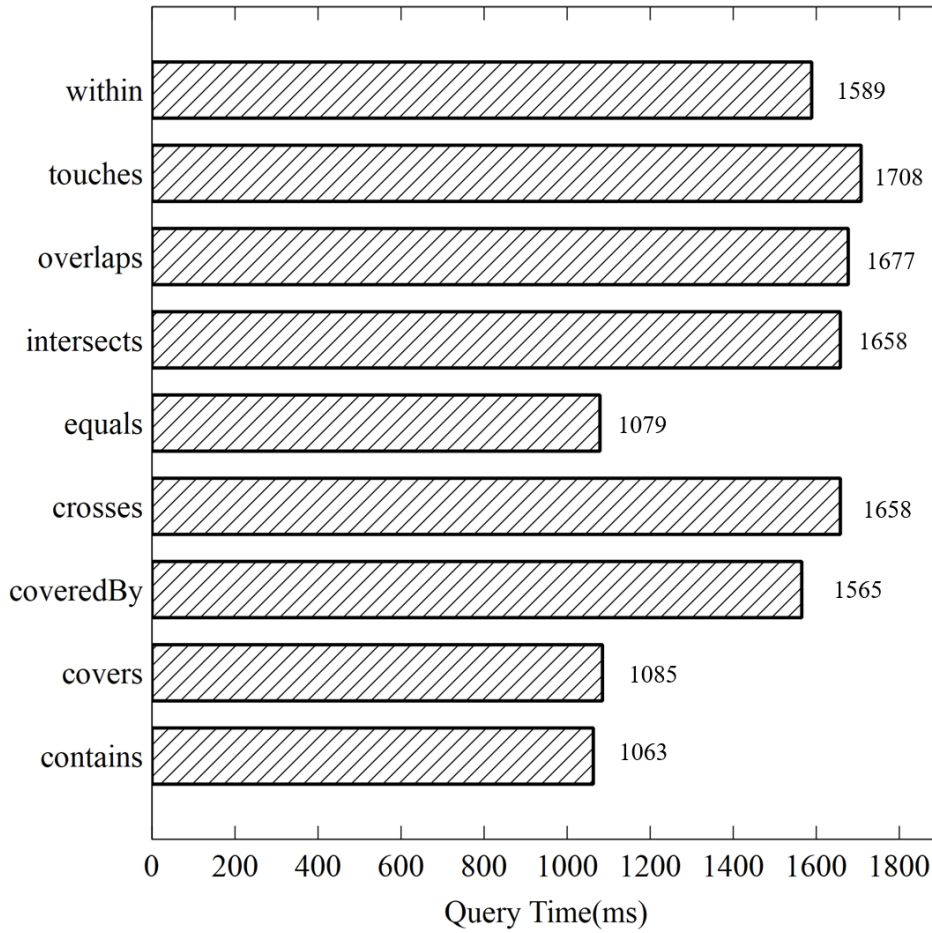
1. Spatial Join Performance

Spatial join operations tested nine DE-9IM predicates between Apartment Building entities (11,363 instances, second-largest polygon dataset) and Residential Area entities (5,987 instances, third-largest polygon dataset). These entity types were selected over Commercial Facility (93,784 instances, largest polygon dataset) due to their higher result counts, providing more robust performance measurements. The disjoint predicate was excluded as it semantically represents "finding all objects that do not intersect with any target objects," requiring inverse logic that cannot utilize spatial index filtering effectively-instead necessitating computation of all intersecting pairs first, then excluding them from the total set.

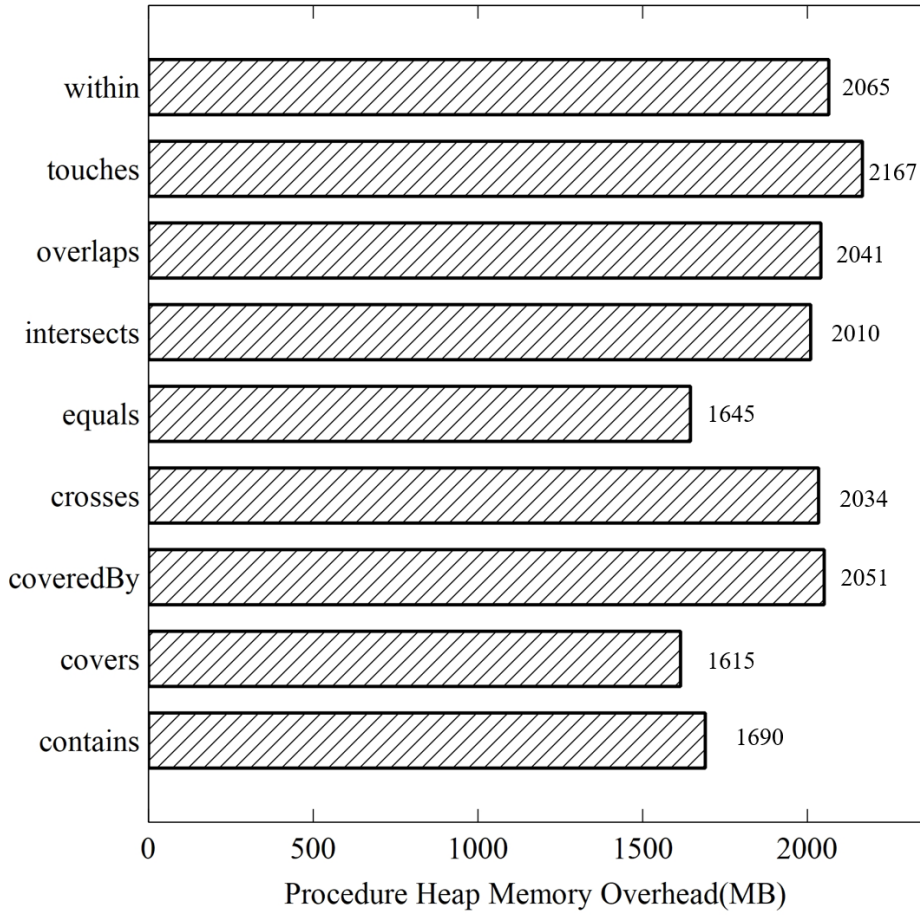
The spatial join query implementation follows Definition 5, where $Q_{SG} = \langle SP_q, G_q \rangle$ with $SP_q = (Join, \{ApartmentBuilding, ResidentialArea\}, \{predicate\})$ and $G_q = \emptyset$. Since this evaluation focuses solely on spatial performance, the graph traversal pattern is absent. The actual Cypher query executed for the intersects predicate was below in Table 4-6.

[Table 4–6] Spatial Join Query

```
CALL gspatial.rtree.query.join(  
    ['ApartmentBuilding', 'ResidentialArea'],  
    ['intersects']  
) YIELD node1, node2  
RETURN node1.uuid AS n_uuid, node2.uuid AS m_uuid
```



[Figure 4–4] Spatial Join Query Time Performance



[Figure 4–5] Spatial Join Query Memory Overhead

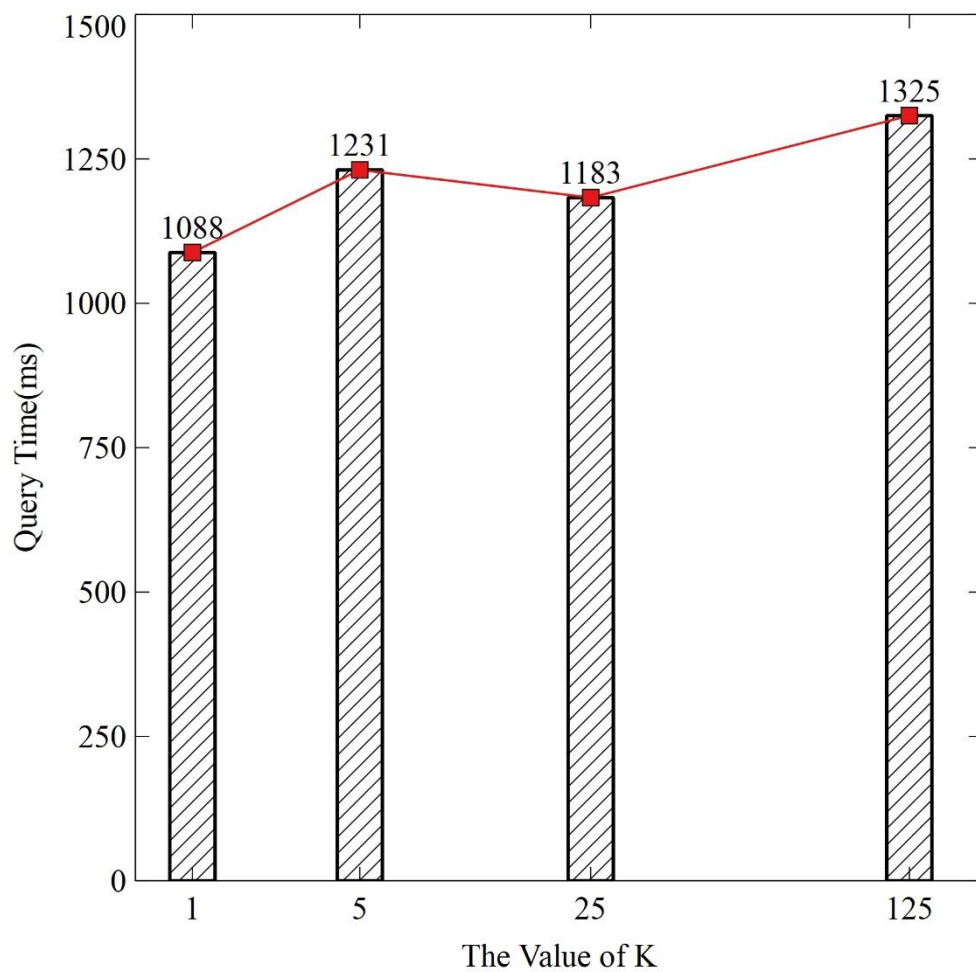
Figure 4–4 and 4–5 illustrate execution characteristics across the nine predicates. Predicates with substantial result counts (intersects: 12,564 pairs, coveredBy: 12,193 pairs, within: 12,193 pairs) consistently require 1,565–1,658ms processing time and 2,010–2,065MB memory due to increased candidate evaluation. Even zero-result predicates (contains, covers, equals, touches, crosses) incur 1,063–1,708ms overhead as complete MBR filtering must occur at all tree levels before determining non-qualification through geometric refinement. The intersects operation, exhibiting both high overhead (1,658ms, 2,065MB) and maximum result count, was selected for subsequent comparative analysis as it represents the worst-case scenario for spatial join processing.

2. K-Nearest Neighbors Performance

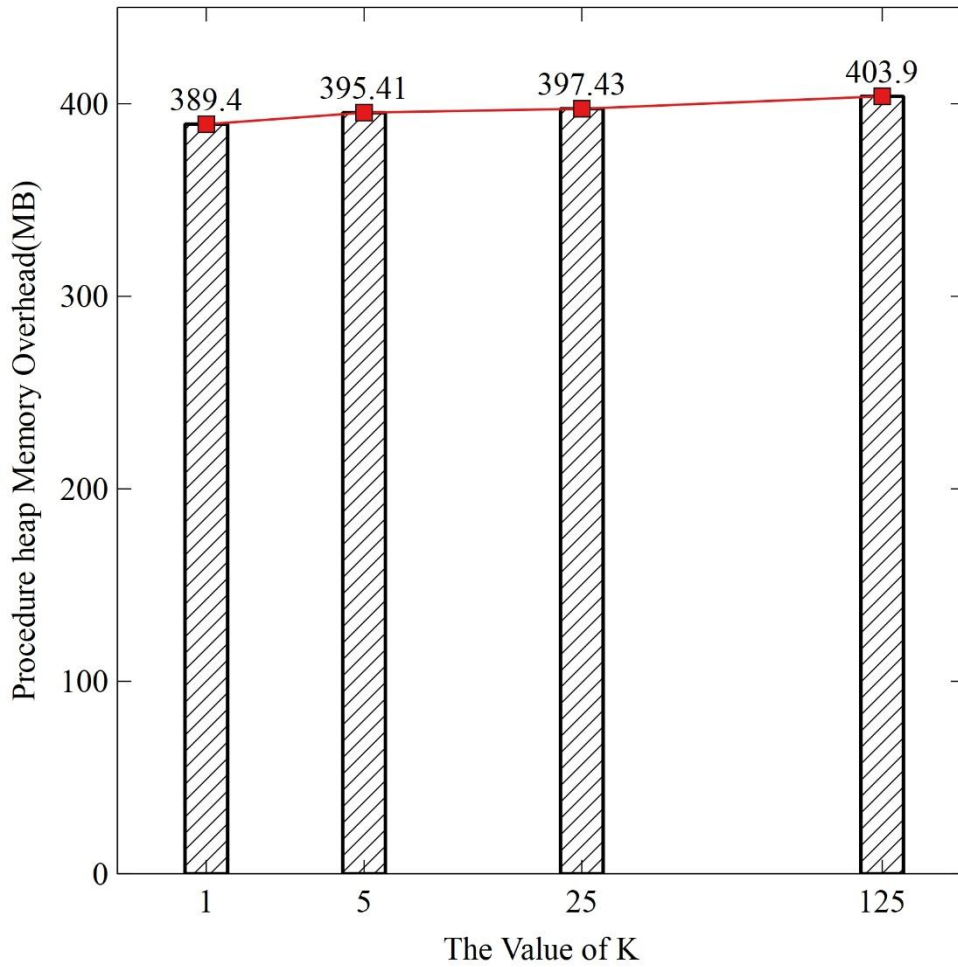
KNN queries evaluated scaling with K values of 1, 5, 25, and 125, representing search patterns from single nearest facility to comprehensive area analysis. The query point was positioned at Seoul City Hall (126.977° , 37.565°), chosen as it represents the geographic center appearing when searching for "Seoul" on major mapping services, reflecting typical user search patterns. Following Definition 5, $Q_{SG} = \langle SP_q, G_q \rangle$ where $SP_q = (KNN, L_{spatial}, \{loc, K\})$ with $L_{spatial}$ encompassing all 75 spatial entity types and $G_q = \emptyset$ for pure spatial performance measurement. The actual Cypher query executed for K=125 was Table 4-7.

[Table 4-7] KNN Query

```
CALL gspatial.rtree.query.knn(  
    ['Landmark', 'SecurityFacility', ... // all 75 spatial labels],  
    [[126.977247283209635, 37.565168749895506], 125]  
) YIELD node, distance  
RETURN node.uuid AS n_uuid, distance
```



[Figure 4-6] KNN Query Time Performance



[Figure 4-7] KNN Query Memory Overhead

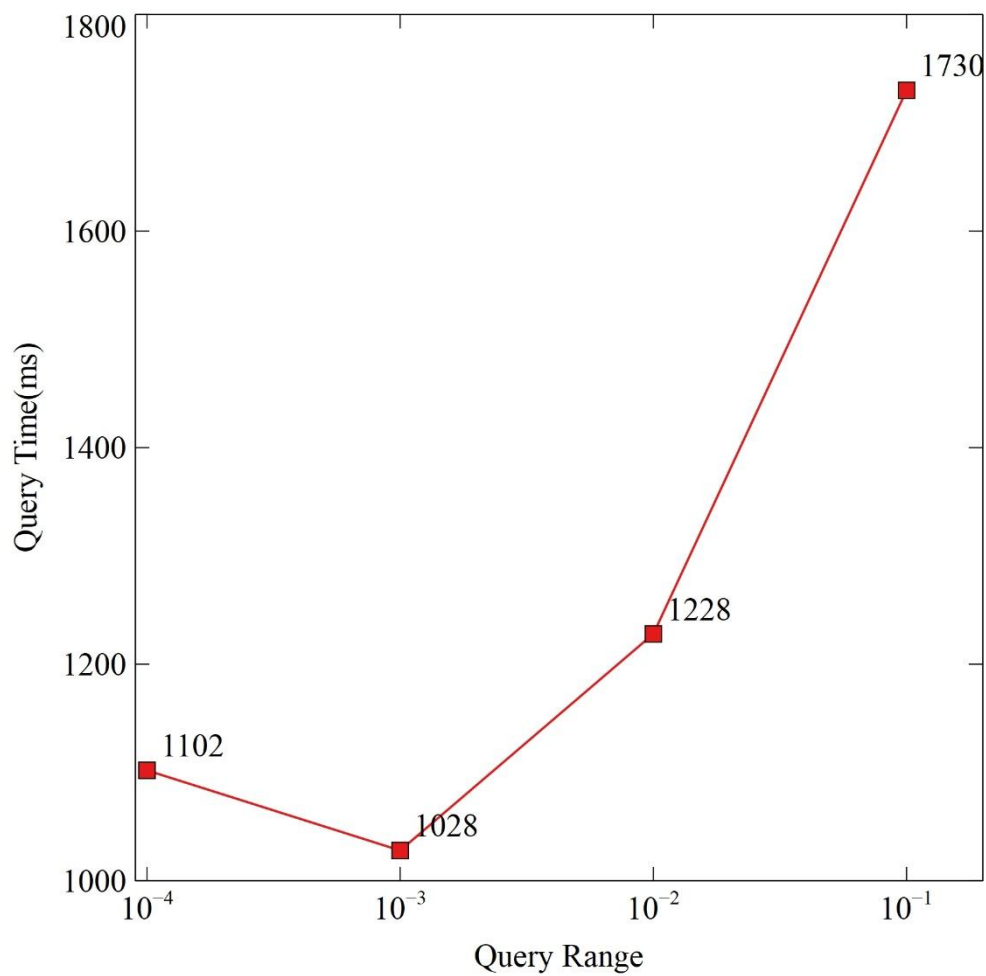
Figure 4-6 and 4-7 demonstrate near-linear scaling characteristics achieved through the optimized priority queue implementation. Performance measurements reveal remarkable efficiency: K=1 processes in 1,088ms (389.4MB), while K=125 completes in 1,325ms (403.9MB)—only 22% time increase for 125× result expansion. This sublinear scaling reflects the SGIR-Tree's efficient priority queue implementation that maintains balanced performance across different K values. Minor execution time variations between K values (e.g., K=5: 1,231ms, K=25: 1,183ms) fall within measurement noise range under cold-cache conditions, with the overall trend showing expected increase as K grows larger.

3. Spatial Range Performance

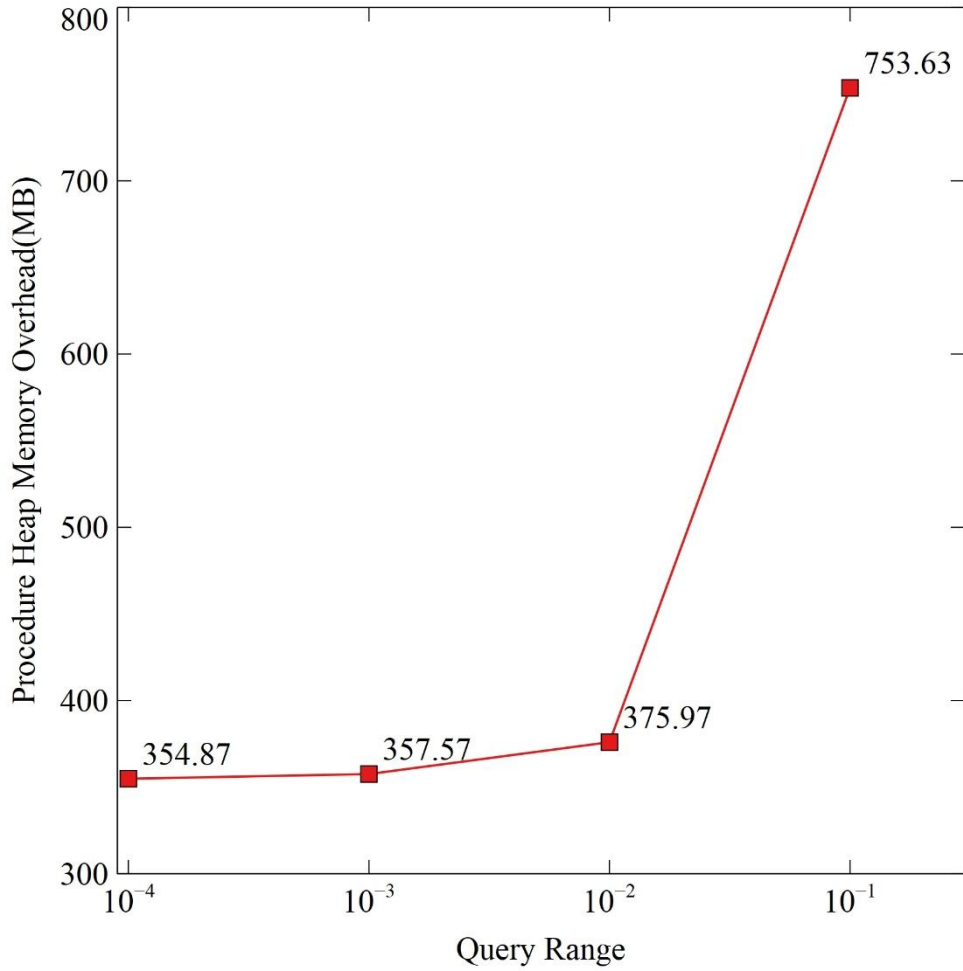
Range queries tested selectivity from 0.0001 to 0.1, corresponding to meaningful urban distances in Seoul's context: 0.0001 ($\sim 390\text{m}^2$) for building-level searches, 0.001 ($\sim 0.039\text{km}^2$) for neighborhood exploration, 0.01 ($\sim 3.9\text{km}^2$) for multi-district dong analysis, and 0.1 ($\sim 390\text{km}^2$) covering over half of Seoul area. The query follows $Q_{SG} = \langle SP_q, G_q \rangle$ where $SP_q = (Range, L_{spatial}, \{Q\})$ with $G_q = \emptyset$. The query range parameter Q can be specified in two ways: directly as a bounding box using lower-left and upper-right corner points, or indirectly through a center point with a buffer radius. When using the buffer approach, the system creates a circular region with the specified radius, then constructs the MBR that encloses this circle for the actual range query execution. In our experiments, we employed the buffer approach with the query center at Seoul City Hall (126.977° , 37.565°), where the buffer radius determines the selectivity parameter. The executed Cypher query for selectivity 0.1 was Table 4–8.

[Table 4–8] Spatial Range Query

```
CALL gspatial.rtree.query.range(
    ['Landmark', 'SecurityFacility', ... // all 75 spatial labels],
    [[126.977247283209635, 37.565168749895506], 0.1]
) YIELD node
RETURN node.uuid AS n_uuid
```



[Figure 4-8] Spatial Range Query Performance



[Figure 4–9] Spatial Range Query Memory Overhead

Figure 4–8 and 4–9 show the algorithm's breadth–first traversal strategy starting from root nodes of all relevant SGIR–Trees. The χ_{TI} function enables rapid pruning by checking edge–stored MBR intersections with the query range before node exploration. At the smallest selectivity (0.0001), the algorithm completes the empty result check in 1,102ms with minimal memory overhead. As selectivity increases to 0.1, the query encompasses 690,307 entities (73% of all spatial nodes) yet maintains sub–2–second response times (1,730ms) with memory scaling from 354.9MB to 753.6MB. The performance characteristics demonstrate the effectiveness of the hierarchical pruning strategy. Even at maximum selectivity,

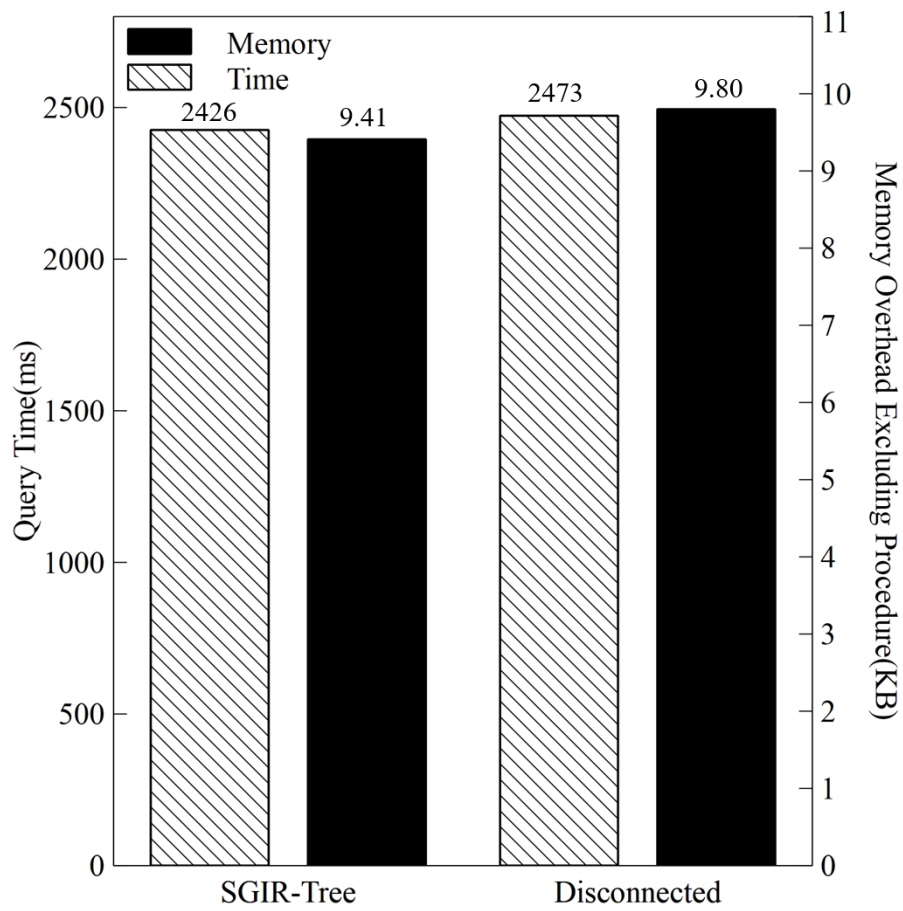
where most tree branches must be explored, the dual MBR storage in both nodes and edges minimizes redundant computations.

Having established SGIR-Tree's baseline performance characteristics, we now compare it against a disconnected spatial index to quantify the benefits of native integration. Figure 4-10 to 4-12 present the comparison between SGIR-Tree and the disconnected spatial index using Neo4j's PROFILE command to isolate graph traversal overhead after spatial filtering. The evaluation employed parameters imposing maximum system stress: intersects for spatial join (highest result count), K=125 for KNN (maximum neighbors), and selectivity=0.1 for range queries (largest area). The graph traversal component G_q was designed to maximize post-spatial processing through high-cardinality relationships, specifically the PART_OF relationship connecting spatial entities to their subcomponents. For the disconnected index comparison, the following query structure was used in Table 4-9.

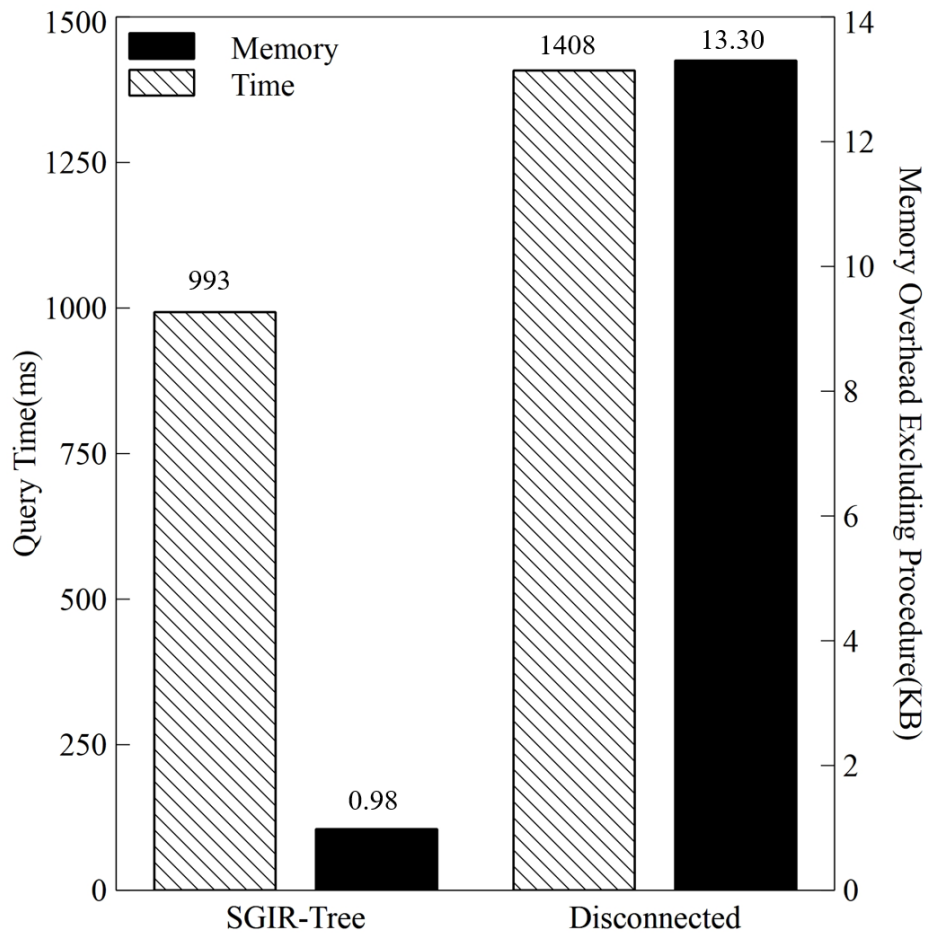
[Table 4-9] Spatial Graph Queries for Comparison

Spatial Join and Graph Traverse Query	
SGIR-Tree	Disconnected Index
PROFILE CALL gspatial.rtree.query.join(['ApartmentBuilding', 'ResidentialArea'], ['intersects']) YIELD node1 as ab, node2 as ra MATCH (au:ApartmentUnit) -[:PART_OF]->(ab) RETURN DISTINCT au.uuid LIMIT 100	PROFILE CALL gspatial.rtree.query.join(['ApartmentBuilding', 'ResidentialArea'], ['intersects'], true) YIELD node1, node2 MATCH (au:ApartmentUnit) -[:PART_OF]->(ab:ApartmentBuilding) WHERE ab.uuid = node1.uuid RETURN DISTINCT au.uuid LIMIT 100
KNN and Graph Traverse Query	
SGIR-Tree	Disconnected Index

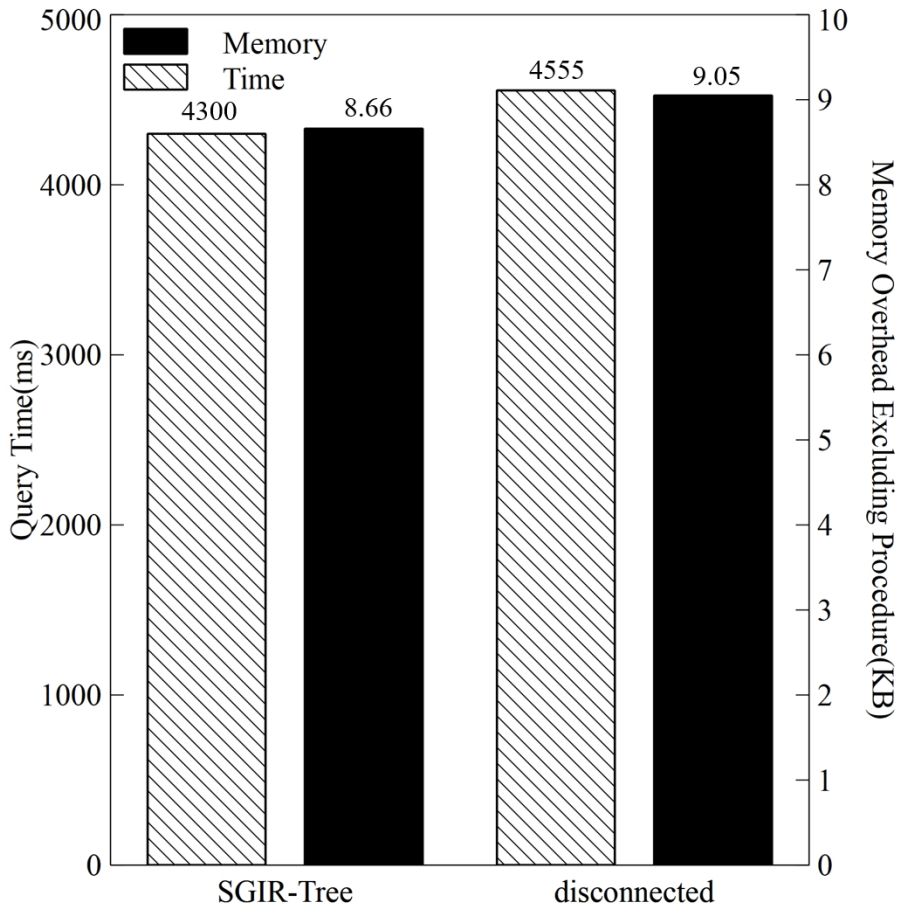
PROFILE CALL gspatial.rtree.query.knn(['Landmark', 'SecurityFacility', ... // all 75 spatial labels], [[126.977247283209635, 37.565168749895506], 125]) YIELD node as com, distance WHERE com:Company MATCH (ce:Entrance) – [:PART_OF] –>(com) RETURN DISTINCT ce.uuid	PROFILE CALL gspatial.rtree.query.knn(['Landmark', 'SecurityFacility', ... // all 75 spatial labels], [[126.977247283209635, 37.565168749895506], 125], true) YIELD node, distance MATCH (com:Company) WHERE com.uuid = node.uuid MATCH (ce:Entrance) – [:PART_OF] –>(com) RETURN DISTINCT ce.uuid
Spatial Range and Graph Traverse Query	
SGIR–Tree	Disconnected Index
PROFILE CALL gspatial.rtree.query.range(['Landmark', 'SecurityFacility', ... // all 75 spatial labels], [[126.977247283209635, 37.565168749895506], 0.1]) YIELD node as com WHERE com:Company MATCH (ce:Entrance) – [:PART_OF] –>(com) RETURN DISTINCT ce.uuid LIMIT 100	PROFILE CALL gspatial.rtree.query.range(['Landmark', 'SecurityFacility', ... // all 75 spatial labels], [[126.977247283209635, 37.565168749895506], 0.1], true) YIELD node MATCH (com:Company) WHERE com.uuid = node.uuid MATCH (ce:Entrance) – [:PART_OF] –>(com) RETURN DISTINCT ce.uuid LIMIT 100



[Figure 4-10] SGIR-Tree vs Disconnected Index Spatial Join Query Comparison



[Figure 4-11] SGIR-Tree vs Disconnected Index KNN Query Comparison



[Figure 4–12] SGIR–Tree vs Disconnected Index Range Query Comparison

For spatial join operations (Figure 4–10), SGIR–Tree completes in 2,426ms (9.41KB) versus 2,473ms (9.80KB) for the disconnected approach—a modest 47ms improvement (1.9%) that falls within typical measurement variance. Range queries (Figure 4–12) show similar patterns, with SGIR–Tree at 4,300ms (8.66KB) versus 4,555ms (9.05KB) for the disconnected approach—a 255ms difference (5.6%) that, while consistent, does not represent a transformative performance gain. These comparable performance characteristics stem largely from the UUID indexing optimization applied to the disconnected approach, which enables efficient correlation between spatial index results and graph nodes. Without this optimization, the disconnected approach would require full table

scans for UUID matching, significantly degrading performance (Kim et al., 2024).

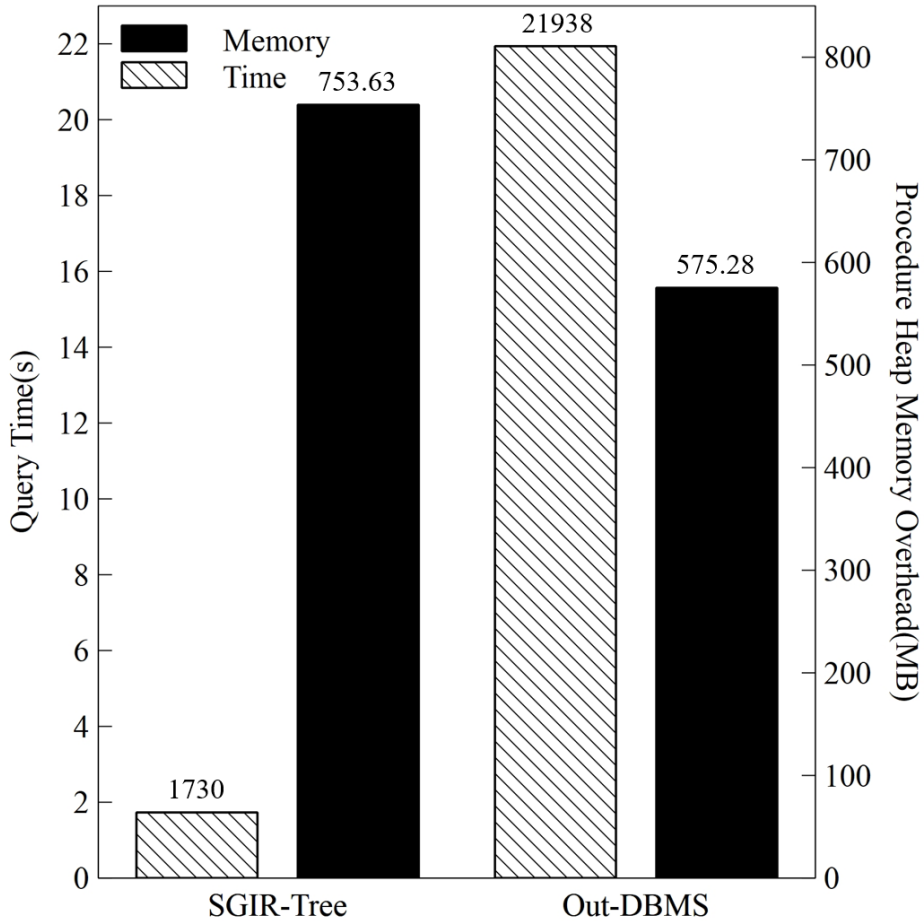
However, KNN queries (Figure 4–11) reveal a more substantial difference: SGIR–Tree processes in 993ms (0.98KB) while the disconnected index requires 1,408ms (13.30KB)– $1.42\times$ slower with $13.6\times$ higher memory consumption. This significant memory overhead results from two compounding factors: first, the KNN operation yields both node and distance values, unlike other spatial operations that return only nodes; second, when these tuples enter the subsequent MATCH clause for UUID–based joining, they create a Cartesian product effect between the 125 nearest neighbors and their distance values, significantly amplifying the intermediate result set that must be maintained in memory during the correlation with graph nodes.

While query execution performance differences are modest for most operations under optimal UUID indexing conditions, SGIR–Tree's primary advantages lie in its architectural design. The unified representation eliminates the $2.81\times$ storage overhead from maintaining duplicate spatial nodes and removes synchronization complexity during updates.

To further contextualize SGIR–Tree's performance advantages, we compare it against an external spatial indexing approach using the STR–Tree from the JTS. Figure 4–13 compares SGIR–Tree with the Out–DBMS approach, measuring only the spatial component SP_q performance since the graph traversal component G_q would be identical for both approaches after spatial results are imported into Neo4j. This direct comparison reveals the pure spatial query efficiency difference between native and external indexing approaches. Since STR–Tree supports only range queries among the three spatial operations, the comparison focuses on range queries with selectivity 0.1. The queries used are the same as in Table 4–8 for SGIR–Tree and Table 4–10 below for STR–Tree.

[Table 4–10] Spatial Range Query for STR–Tree

```
CALL gspatial.strtree.query.range(  
  ['Landmark', 'SecurityFacility', ... // all 75 spatial labels],  
  [[126.977247283209635, 37.565168749895506], 0.1])  
YIELD node  
RETURN node.uuid AS node_uuid
```



[Figure 4–13] SGIR–Tree vs Out–DBMS Index Range Query Comparison

The experimental results display a dramatic performance gap. SGIR–Tree completes the spatial–graph query in 1,730ms, while the Out–DBMS approach requires 21,938ms—a $12.7\times$ performance degradation. Detailed profiling reveals that approximately 15–18

seconds (70–82% of total time) are consumed solely by loading the external STR–Tree index for all spatial entities. The remaining overhead stems from multiple factors: data serialization between Neo4j and JTS, result transformation from JTS geometry objects to Neo4j node identifiers, and the complete loss of query optimization opportunities that would normally be available within the integrated GDBMS environment.

While the Out–DBMS approach demonstrates slightly better query memory efficiency (575.28 MB versus SGIR–Tree's 753.63 MB for selectivity 0.1), this 23.7% memory advantage is entirely negated by the $12.7\times$ query performance degradation. The memory savings become operationally meaningless when the approach requires 15–18 seconds merely to load the index—a latency that fundamentally precludes interactive housing exploration applications regardless of runtime memory efficiency.

This significant performance difference underscores the fundamental advantage of spatial graph indexing for spatial KGs. The SGIR–Tree's subgraph architecture enables direct access through standard graph traversal operations, eliminating overhead from external index management, data marshaling, and context switching between computational environments. More critically for spatial KG applications, native integration allows the query optimizer to consider both spatial and semantic aspects simultaneously. This enables holistic execution plans that leverage the interconnected nature of spatial entities, their relationships, and ontological structures. These results validate that for interactive spatial KG applications like housing exploration, Native integration is not merely a performance optimization but an architectural necessity.

4.2.3. Multi–Layer Integration Evaluation

This sub–section validates the practical effectiveness of the integrated MSKG demonstrating how the three–subgraph architecture (G_S, G_D, G_I) operates cohesively for complex queries.

The effectiveness of our architecture fundamentally depends on the seamless integration of all three subgraphs—Semantic Graph (G_S),

Data Graph (G_D), and Spatial Index Graph (G_I)—working in concert to answer complex housing exploration queries. Previous sections evaluated individual components. Section 4.2.1 assessed KG quality, while Section 4.2.2 measured spatial indexing performance. This subsection illustrates how complete MSKG architecture operates as a unified system. The system processes queries that simultaneously require ontological reasoning, instance data traversal, and spatial computation.

We created five representative queries from CQ clusters C1–C5 identified during the specification phase. These queries exercise all three graph layers simultaneously, demonstrating different patterns of multi-layer interaction. Each query represents a distinct housing exploration scenario that real users might encounter, from basic location-based searches to complex multi-criteria evaluations involving safety, education, and community amenities. Table 4–11 presents the detailed characteristics of each query, including their execution times, result counts, and multi-layer integration patterns. The complete Cypher query implementations for all five cases are provided in Appendix B (Table B–1), demonstrating how abstract user requirements are translated into executable graph traversals that span the semantic, data, and spatial index layers.

[Table 4–11] Multi-Layer Query Characteristics and Execution Results

Cluster	Query Description	Execution Time (ms)	Results	Multi-Layer Integration Pattern
C1	Location & Accessibility/ "Apartments and neighborhood environment near SNU Building 35"	2,860	143	G_S : Resolve PROXIMITY concept and neighbor types G_I : Range query (4km radius) + KNN (top-3 neighbors per apartment) G_D : Filter apartment instances and aggregate results
C2	Family &	5,496	819	G_S : Educational facility

	Living Environment/ "Apartments Good for raising teenage children"			taxonomy (School, Academy, etc.) G_D : Filter by rooms (≥ 3) and size ($\geq 85m^2$) G_I : KNN (5 nearest facilities) + diversity check (≥ 3 types)
C3	Budget & Transportation/ "Cheap Monthly Rent Apartments with convenient transit"	1,463	194	G_S : Public transportation concept hierarchy G_D : Recent rental transactions ($\leq 3000/50$ 만원, within 6 months) G_I : KNN (2 nearest transit stations) with distance filter ($\leq 500m$)
C4	Apartment Features/ "District and Apartments with Diverse community facilities"	1,549	7	G_S : PART_OF hierarchy for facility types G_D : Count facility diversity ($\geq 50\%$ of total types) G_I : Spatial join (apartment WITHIN district)
C5	Safety & Security/ "Good security Apartment"	564	16	G_S : Security facility types and properties G_I : Spatial join (facilities WITHIN complex, ≥ 10 required) G_D : Property-based filtering and scoring

The execution results reveal distinct patterns of multi-layer interaction, with each cluster demonstrating different optimization opportunities and bottlenecks. C2 exhibits the highest execution time (5,496ms) primarily due to extensive graph traversal overhead rather than spatial operation complexity. The query requires traversing all Room nodes (2.76 million). This pattern generates

significantly more intermediate results compared to C1, despite C1 involving 60+ neighbor environment label types versus C2's approximately 10 educational facility types. The semantic layer retrieves only about 10 educational facility labels, but the instance layer must scan potentially thousands of Room nodes and perform expensive aggregation operations across the PART_OF hierarchy.

C5 achieves the fastest execution time (564ms) among all five queries through efficient cross-layer traversal patterns. This query identifies apartments with comprehensive internal security infrastructure (≥ 10 security facilities within complex boundaries) by operationalizing the abstract "good for security" concept. The execution flow demonstrates seamless layer transitions: it begins with spatial join operations in G_I to identify SecurityFacility nodes contained within ApartmentComplex boundaries, traverses through G_S to resolve the semantic hierarchy of security concepts (e.g., Building Entrance Security Lock, Security Guard) via HAS_PROPERTY and PART_OF relationships, and descends to G_D to verify whether apartment sub-facilities possess the derived security properties. The rapid execution confirms that RTREE_REFERENCE edges (E_{ID}) and INSTANCE_OF edges (E_{SD}) enable unified query processing across all three layers without intermediate result materialization, making the MSKG architecture particularly effective for queries requiring bidirectional traversals from spatial filtering through ontological reasoning to instance-level verification.

C1, C3, and C4 demonstrate moderate complexity with execution times ranging from 1,463ms to 2,860ms, each exhibiting distinct patterns of spatial-graph integration. C1 requires the most intensive spatial processing (2,860ms) due to multiple KNN queries identifying top-3 neighbors for each of 143 apartment complexes. C3 (1,463ms) benefits from simpler spatial requirements despite additional transaction data filtering and temporal constraints. C4 (1,549ms) shows balanced integration where spatial join operations (determining district membership) and graph-structural operations (PART_OF hierarchy traversal for facility diversity verification) contribute roughly equally to overall latency. These moderate-

complexity queries confirm that the architecture maintains consistent sub-3-second performance across diverse operation patterns combining spatial filtering, semantic resolution, and instance traversal in varying proportions.

The results validate several critical aspects of our MSKG architecture. First, queries leveraging all three layers maintain sub-second to low-second response times (0.5–5.5s) despite their complexity, confirming that the inter-layer connections (E_{SD} connecting semantic and data layers, E_{ID} connecting spatial index and data layers) enable efficient traversal across the MSKG structure. Second, the breakdown of execution patterns reveals that spatial operations (G_I) dominate in location-centric queries (C1, C2, C5), while semantic resolution (G_S) adds minimal overhead (typically < 100ms based on profiling data). This confirms that the ontology-driven approach does not significantly impact query performance while providing essential semantic abstraction capabilities. The semantic layer effectively serves its intended purpose of translating abstract user concepts like "good for security" into concrete, queryable conditions without becoming a performance bottleneck.

4.3. Discussion

This section synthesizes experimental findings and discusses practical implications for deployment.

4.3.1. Key Findings from Housing Exploration Application

The experimental evaluation reveals several critical findings that extend beyond the housing domain to inform broader spatial KG construction practices.

The Direct model establishes itself as optimal for large-scale interactive applications, with 68-minute construction time for 22.4 million entities and consistent sub-second query performance averaging 0.32 seconds. This enables practical daily full rebuilds while maintaining interactive user experience, providing a viable interim solution until incremental update mechanisms mature. From a

user perspective, this architectural choice directly translates to tangible service improvements: housing seekers can access continuously updated market information with response times indistinguishable from static database queries, while service operators can maintain data currency through overnight batch updates without service interruption. In stark contrast, the Singleton model's 19,683-fold query slowdown demonstrates that certain semantic representation choices create insurmountable computational barriers. The UUID-based relationship matching overhead introduces catastrophic performance degradation even for simple one-hop traversals, rendering the Singleton model suitable only for offline analytical workloads where semantic completeness outweighs runtime considerations. The Statement model occupies an intermediate position, offering superior semantic expressiveness through relationship-level metadata attachment at approximately $2\times$ storage overheads compared to Direct modeling, making it appropriate for domains where comprehensive provenance tracking justifies the performance trade-off.

The SGIR-Tree architecture validates that spatial graph indexing provides performance benefits extending beyond implementation convenience. The 12.7-fold query speedup compared to out-DBMS indices stems from enabling holistic query optimization across spatial and semantic dimensions. Multi-layer queries maintain reasonable performance because `RTREE_REFERENCE` edges enable seamless transitions between spatial filtering and graph traversal without intermediate result materialization. For housing exploration specifically, this means users can iteratively refine spatial constraints—adjusting search radii, adding proximity requirements to amenities, or constraining administrative boundaries—without experiencing degraded response times that would disrupt the exploratory decision-making process characteristic of housing selection. As the KG scales beyond the current 1 million spatial entities, this architectural advantage becomes increasingly critical.

The semi-automatic ontology construction methodology

achieved 96.7 percent coverage with 146 of 151 CQs successfully operationalized. The five failed CQs stemmed from data availability limitations rather than ontological deficiencies, underscoring that data collection comprehensiveness rather than architecture capability often constitutes the primary bottleneck for production deployment. The Validating Schema mechanism proved valuable in detecting constraint violations impractical to identify through manual review of millions of instances.

These findings suggest broader applicability beyond housing. The MSKG architecture separating semantic graph, data graph, and spatial index graph provides a generalizable pattern for domains requiring combined semantic reasoning and spatial analysis, including urban planning, healthcare facility optimization, and transportation network analysis.

4.3.2. Real-World Applicability and User Needs Validation

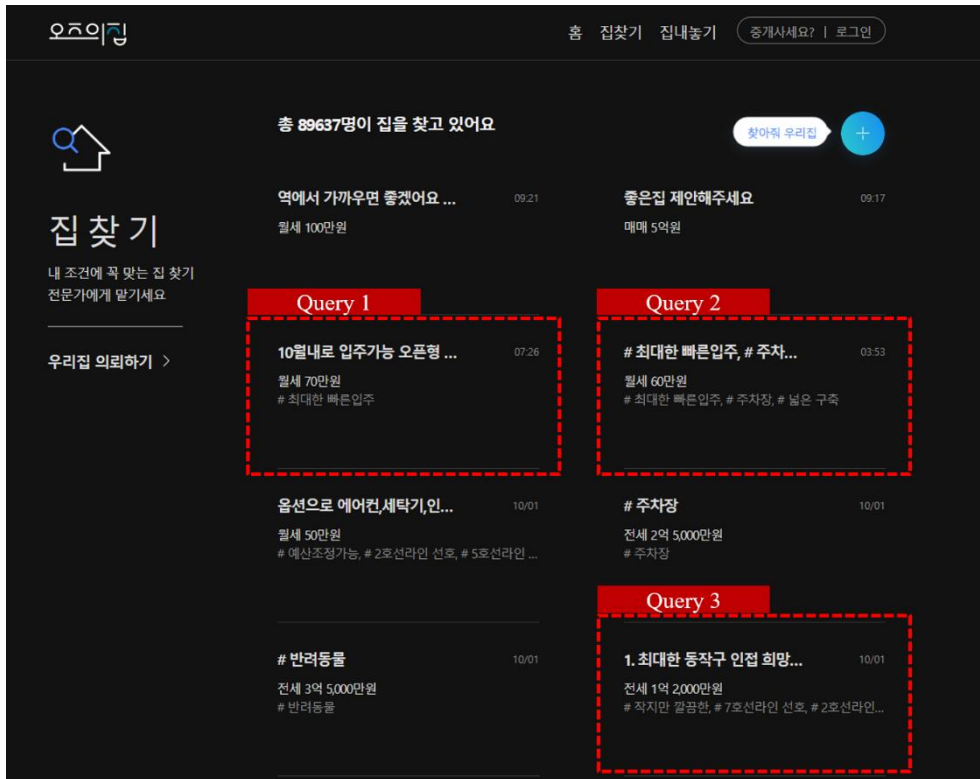
Section 4.2.1 demonstrated that the constructed KG successfully answers 96.7% of the designed CQs, establishing quantitative evidence of functional completeness. However, a natural concern arises regarding the CQs themselves: since the same researcher designed both the CQs and the evaluated KG, questions of objectivity may emerge. To address this concern, this section presents supplementary validation through two complementary approaches: comparison with Hong (2025)'s data-driven clustering of real user queries from Oz's House, and demonstration of operational feasibility using actual complex queries from the same platform.

At a macroscopic level, we benchmarked our CQs against the data-driven ontology proposed by Hong (2025), which clustered 17,909 natural language queries from Oz's House, Korea's popular online real estate community. As detailed in Table B-2, the comparative analysis reveals substantial alignment across key housing exploration categories—ranging from safety and financial considerations to pet policies and location preferences. This convergence between our systematic specification analysis and Hong's bottom-up data mining confirms that the CQs capture

authentic information needs rather than researcher assumptions. Furthermore, while Hong's clustering approach effectively captures high-frequency user intents, our systematic methodology extends this foundation by articulating significantly more granular requirements that address long-tail user needs.

Complementing this conceptual validation, we demonstrated the operational capabilities of the proposed architecture by processing actual, complex user queries from Oz's House. Three representative Seoul-focused queries were selected and transformed into structured requirements using operationalization approach (Figure 4-14). All queries executed correctly with sub-second response times, returning either appropriate results or meaningful explanations. The first query successfully returned 22 ranked apartments meeting multi-criteria requirements including budget constraints, size specifications, and preference-based scoring for parking and pet policies. The second and third queries returned zero results, but analysis revealed these stemmed from data availability gaps and market constraints rather than architecture limitations. This validates that the proposed methodology transforms traditional manual search through semantic query expansion, graph spatial processing via SGIR-Tree, and transparent multi-criteria ranking. Complete query implementations are provided in Appendix B (Table B-3).

Beyond technical validation, these results demonstrate user-centric benefits that address practical housing exploration challenges. Traditional housing platforms require users to manually translate abstract preferences (e.g., "safe neighborhood," "good schools") into concrete filter combinations, often through trial-and-error across multiple search sessions. The proposed MSKG architecture automates this translation through the semantic graph's ontology-driven query expansion, reducing cognitive load while ensuring comprehensive consideration of relevant criteria. The SGIR-Tree's spatial graph indexing further enables real-time proximity analysis that would be impractical with conventional external indexing approaches, allowing users to dynamically adjust distance thresholds and spatial relationships during their exploration process.



[Figure 4-14] Oz's House Homepage on October 2, 2025

Chapter 5. Conclusions

This dissertation addressed fundamental challenges in constructing domain-specific spatial KGs by developing a comprehensive architecture that systematically integrates ontology design, semi-automatic construction, and spatial graph indexing for LPG databases. The research originated from four critical gaps: the absence of systematic ontology engineering methodologies for LPG environments, the prohibitively high cost of KG construction despite recent LLM advances, fundamental architectural limitations in GDBMSs for spatial data handling, and the unresolved trade-off between query flexibility and storage efficiency in existing spatial KG systems.

In response, this research makes four primary contributions. First, we established a comprehensive and systematic methodology for integrating ontological structures with instance data in unified LPG representations, explicitly formalizing Direct, Singleton, and Statement modeling approaches through mathematical definitions and comparative evaluation. The empirical analysis demonstrated that the Direct model achieves optimal performance for large-scale interactive applications with consistent sub-second query performance averaging 0.32 seconds, while the Singleton model exhibits catastrophic 19,683-fold performance degradation for complex queries due to UUID-based relationship matching overhead. Second, we presented a comprehensive semi-automatic construction methodology that systematically integrates LLM-based ontology engineering with expert refinement, achieving 96.7% coverage with 146 of 151 CQs successfully operationalized in the Seoul housing domain. Third, we integrated spatial indices as native subgraphs within GDBMSs through the SGIR-Tree methodology, demonstrating 12.7-fold query speedup compared to out-DBMS approaches by enabling unified query optimization across spatial and graph operations. Fourth, we introduced a lightweight architecture paradigm that eliminates pre-computed spatial relationship edges,

replacing them with on-demand computation that maintains interactive response times of 0.32 to 5.49 seconds for complex queries over 22.4 million entities while reducing graph complexity.

The housing exploration KG validates these contributions empirically, demonstrating that principled architectural choices enable spatial knowledge systems balancing expressiveness, efficiency, and maintainability. From a practical perspective, architecture enables housing exploration services that would be infeasible with conventional approaches: users can express requirements in natural terms that the semantic graph translates to precise queries, spatial constraints can be refined iteratively without performance degradation, and the underlying knowledge base can be updated daily to reflect market changes. These validated design decisions provide general architecture applicable beyond housing to other domains requiring integrated spatial-semantic reasoning, such as urban planning systems, healthcare facility location optimization, or supply chain management integrating warehouse locations with delivery route networks.

While this dissertation advances spatial KG construction methodologies, several directions for future research emerge from current limitations. The methodology currently requires human expert involvement at critical decision points, particularly for validating LLM-generated concepts and resolving ambiguous cases in data mapping. Advancing toward fuller automation through fine-tuning LLMs on domain ontology engineering tasks, implementing multi-agent architectures, or employing ensemble techniques represents a promising direction, though complete automation may prove neither feasible nor desirable as domain expertise often involves tacit knowledge difficult to capture computationally. The current implementation also lacks native reasoning capabilities comparable to RDF-based systems with OWL semantics. Developing reasoning mechanisms for LPG-based KGs through rule-based systems, graph neural network approaches, or hybrid architectures combining symbolic reasoning with statistical learning could address this gap while maintaining the performance advantages that motivated

the shift from RDF to LPG.

Methodology currently requires complete KG rebuilding when incorporating new data. While the Direct model's construction efficiency enables practical daily full rebuilding, truly real-time applications demand incremental update mechanisms that carefully handle spatial index maintenance, schema evolution, and temporal versioning. Additionally, the current implementation adopts uniform Direct modeling for all relationships, but hybrid strategies that selectively apply Statement pattern reification to critical relationships requiring comprehensive provenance tracking while maintaining Direct modeling efficiency for the majority merit investigation. Finally, cross-domain validation applying architecture to healthcare facility networks, transportation infrastructure planning, or environmental monitoring systems would test the methodology's true domain independence and reveal refinements needed for different spatial distribution characteristics.

In conclusion, this dissertation establishes theoretical foundations and practical methodologies for constructing domain-specific spatial KGs in LPG databases. By systematically addressing the core element technologies for each subgraph construction—LPG ontology design for the semantic graph, semi-automatic construction for the data graph, spatial graph indexing for the spatial index graph—this research provides essential foundations for building performant spatial knowledge systems that bridge the gap between semantic richness and computational efficiency. The methodologies, empirical findings, and identified research directions presented herein provide guidance for researchers and practitioners developing the next generation of spatial knowledge systems across diverse application domains.

Bibliography

- Abolhasani, M. S., & Pan, R. (2024). *Leveraging LLM for Automated Ontology Extraction and Knowledge Graph Generation* (No. arXiv:2412.00608). arXiv. <https://doi.org/10.48550/arXiv.2412.00608>
- Amiri, A. M., Samavati, F., & Peterson, P. (2015). Categorization and conversions for indexing methods of discrete global grid systems. *ISPRS International Journal of Geo-Information*, 4(1), 320–336.
- Angles, R., Bonifati, A., Dumbrava, S., Fletcher, G., Green, A., Hidders, J., Li, B., Libkin, L., Marsault, V., Martens, W., Murlak, F., Plantikow, S., Savkovic, O., Schmidt, M., Sequeda, J., Staworko, S., Tomaszuk, D., Voigt, H., Vrgoc, D., ... Zivkovic, D. (2023). PG-Schema: Schemas for Property Graphs. *Proc. ACM Manag. Data*, 1(2), 198–223. <https://doi.org/10.1145/3589778>
- Bast, H., Brosi, P., Kalmbach, J., & Lehmann, A. (2021). An Efficient RDF Converter and SPARQL Endpoint for the Complete OpenStreetMap Data. *Proceedings of the 29th International Conference on Advances in Geographic Information Systems*, 536–539. <https://doi.org/10.1145/3474717.3484256>
- Battle, R., & Kolas, D. (2012). Enabling the geospatial Semantic Web with Parliament and GeoSPARQL. *Semant. Web*, 3(4), 355–370.
- Beckmann, N., Kriegel, H.-P., Schneider, R., & Seeger, B. (1990). The R*-tree: An efficient and robust access method for points and rectangles. *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*, 322–331. <https://doi.org/10.1145/93597.98741>
- Brinkhoff, T., Kriegel, H.-P., & Seeger, B. (1993). Efficient processing of spatial joins using R-trees. *ACM SIGMOD Record*, 22(2), 237–246. <https://doi.org/10.1145/170036.170075>
- Brodt, A., Nicklas, D., & Mitschang, B. (2010). Deep integration of spatial query processing into native RDF triple stores. *Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 33–42. <https://doi.org/10.1145/1869790.1869799>
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chaudhri, V. K., Baru, C., Chittar, N., Dong, X. L., Genesereth, M., Hendler, J., Kalyanpur, A., Lenat, D. B., Sequeda, J., Vrandečić, D., & Wang, K. (2022). Knowledge graphs: Introduction, history, and perspectives. *AI Magazine*, 43(1), 17–29. <https://doi.org/10.1002/aaai.12033>
- Chen, B., & Bertozzi, A. L. (2023). *AutoKG: Efficient Automated Knowledge Graph Generation for Language Models* (No. arXiv:2311.14740). arXiv. <https://doi.org/10.48550/arXiv.2311.14740>
- Cheung, K. L., & Fu, A. W.-C. (1998). Enhanced nearest neighbour search on the R-tree. *ACM SIGMOD Record*, 27(3), 16–21. <https://doi.org/10.1145/290593.290596>
- Cho, J. H., & Kang, M. H. (2011). Design of a Real Estate Knowledge

- Information System Based on Semantic Search. *Journal of the Korea Industrial Information Systems Society*, 16(2), 111–124. <https://doi.org/10.9723/JKSIIS.2011.16.2.111>
- Di Pierro, D., Ferilli, S., & Redavid, D. (2023). LPG-Based Knowledge Graphs: A Survey, a Proposal and Current Trends. *Information*, 14(3), Article 3. <https://doi.org/10.3390/info14030154>
- Dsouza, A., Tempelmeier, N., Gottschalk, S., Yu, R., & Demidova, E. (2023). WorldKG: World-Scale Completion of Geographic Information. In D. Burghardt, E. Demidova, & D. A. Keim (Eds.), *Volunteered Geographic Information: Interpretation, Visualization and Social Context* (pp. 3–19). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-35374-1_1
- Femi Aminu, E., Oyefolahan, I. O., Bashir Abdullahi, M., & Salaudeen, M. T. (2020). A Review on Ontology Development Methodologies for Developing Ontological Knowledge Representation Systems for various Domains. *International Journal of Information Engineering and Electronic Business*, 12(2), 28–39. <https://doi.org/10.5815/ijieeb.2020.02.05>
- Feng, X., Wu, X., & Meng, H. (2024). *Ontology-grounded Automatic Knowledge Graph Construction by LLM under Wikidata schema* (No. arXiv:2412.20942). arXiv. <https://doi.org/10.48550/arXiv.2412.20942>
- Finkel, R. A., & Bentley, J. L. (1974). Quad trees a data structure for retrieval on composite keys. *Acta Informatica*, 4(1), 1–9. <https://doi.org/10.1007/BF00288933>
- Francis, N., Green, A., Guagliardo, P., Libkin, L., Lindaaker, T., Marsault, V., Plantikow, S., Rydberg, M., Selmer, P., & Taylor, A. (2018). Cypher: An Evolving Query Language for Property Graphs. *Proceedings of the 2018 International Conference on Management of Data*, 1433–1445. <https://doi.org/10.1145/3183713.3190657>
- Greene, D. (1989). *An implementation and performance analysis of spatial data access methods*. 606–615. <https://doi.org/10.1109/ICDE.1989.47268>
- Gruninger, M., & Fox, M. S. (1995). Methodology for the Design and Evaluation of Ontologies. *IJCAI-95*. International Joint Conference on Artificial Intelligence. <http://www.eil.utoronto.ca/wp-content/uploads/enterprise-modelling/papers/gruninger-ijcai95.pdf>
- Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data - SIGMOD '84*, 47. <https://doi.org/10.1145/602259.602266>
- Hadjieleftheriou, M., Hoel, E., & Tsotras, V. J. (2005). SaIL: A Spatial Index Library for Efficient Application Integration. *GeoInformatica*, 9(4), 367–389. <https://doi.org/10.1007/s10707-005-4577-6>
- Hernández, D., Hogan, A., Riveros, C., Rojas, C., & Zerega, E. (2016). Querying Wikidata: Comparing SPARQL, Relational and Graph Databases. In P. Groth, E. Simperl, A. Gray, M. Sabou, M. Krötzsch, F. Lecue, F. Flöck, & Y. Gil (Eds.), *The Semantic Web – ISWC 2016* (pp. 88–103). Springer International Publishing. https://doi.org/10.1007/978-3-319-46547-0_10
- Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F., & Rudolph, S. (2012, December 11). *OWL 2 Web Ontology Language Primer (Second Edition)* [W3C Recommendation. World Wide Web Consortium]. W3C

- Recommendation. <https://www.w3.org/TR/2012/REC-owl2-primer-20121211/>
- Hofer, M., Obraczka, D., Saeedi, A., Köpcke, H., & Rahm, E. (2024). Construction of Knowledge Graphs: Current State and Challenges. *Information*, 15(8), Article 8. <https://doi.org/10.3390/info15080509>
- Hoffart, J., Suchanek, F. M., Berberich, K., & Weikum, G. (2013). YAGO2: A spatially and temporally enhanced knowledge base from Wikipedia. *Artificial Intelligence*, 194, 28–61. <https://doi.org/10.1016/j.artint.2012.06.001>
- Hogan, A., Blomqvist, E., Cochez, M., d’Amato, C., Melo, G. de, Gutierrez, C., Gayo, J. E. L., Kirrane, S., Neumaier, S., Polleres, A., Navigli, R., Ngomo, A. – C. N., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2022). Knowledge Graphs. *ACM Computing Surveys*, 54(4), Article 4. <https://doi.org/10.1145/3447772>
- Hong, S. (2025). *Development of Spatial Ontology through Relation-based Clustering: A Case of Housing Search* [Master’s thesis]. Seoul National University.
- Huang, W., Raza, S. A., Mirzov, O., & Harrie, L. (2019). Assessment and Benchmarking of Spatially Enabled RDF Stores for the Next Generation of Spatial Data Infrastructure. *ISPRS International Journal of Geo-Information*, 8(7), Article 7. <https://doi.org/10.3390/ijgi8070310>
- ISO/IEC 39075:2024—Information technology—Database languages—GQL (Graph Query Language) (PDF No. 39075; Version 2024). (2024). <https://www.iso.org/standard/76120.html> (Published)
- Joachimiak, M. P., Miller, M. A., Caufield, J. H., Ly, R., Harris, N. L., Tritt, A., Mungall, C. J., & Bouchard, K. E. (2024). The Artificial Intelligence Ontology: LLM-Assisted Construction of AI Concept Hierarchies. *Applied Ontology*, 19(4), 408–418. <https://doi.org/10.1177/15705838241304103>
- Karalis, N., Mandilaras, G., & Koubarakis, M. (2019). Extending the YAGO2 Knowledge Graph with Precise Geospatial Knowledge. In C. Ghidini, O. Hartig, M. Maleshkova, V. Svátek, I. Cruz, A. Hogan, J. Song, M. Lefrançois, & F. Gandon (Eds.), *The Semantic Web – ISWC 2019* (pp. 181–197). Springer International Publishing. https://doi.org/10.1007/978-3-030-30796-7_12
- Kim, J., Hong, S., Jeong, S., Park, S., & Yu, K. (2024). SGIR-Tree: Integrating R-Tree Spatial Indexing as Subgraphs in Graph Database Management Systems. *ISPRS International Journal of Geo-Information*, 13(10), 346. <https://doi.org/10.3390/ijgi13100346>
- Kommineni, V. K., König-Ries, B., & Samuel, S. (2024). *From human experts to machines: An LLM supported approach to ontology and knowledge graph construction* (No. arXiv:2403.08345). arXiv. <http://arxiv.org/abs/2403.08345>
- Laddada, W., Duchateau, F., Favetta, F., & Moncla, L. (2020). Ontology-Based Approach for Neighborhood and Real Estate Recommendations. *Proceedings of the 4th ACM SIGSPATIAL Workshop on Location-Based Recommendations, Geosocial Networks, and Geoadvertising*, 1–10. <https://doi.org/10.1145/3423334.3431452>
- Lee, T., Moon, B., & Lee, S. (2003). Bulk Insertion for R-Tree by Seeded Clustering. In V. Mařík, W. Retschitzegger, & O. Štěpánková (Eds.), *Database and Expert Systems Applications* (pp. 129–138). Springer. https://doi.org/10.1007/978-3-540-45227-0_14

- Liagouris, J., Mamoulis, N., Bouros, P., & Terrovitis, M. (2014). An effective encoding scheme for spatial RDF data. *Proceedings of the VLDB Endowment*, 7(12), 1271–1282. <https://doi.org/10.14778/2732977.2733000>
- Lippolis, A. S., Lodi, G., & Nuzzolese, A. G. (2025). The Water Health Open Knowledge Graph. *Scientific Data*, 12(1), 274. <https://doi.org/10.1038/s41597-025-04537-4>
- Lissandrini, M., Brugnara, M., & Velegrakis, Y. (2018). Beyond macrobenchmarks: Microbenchmark-based graph database evaluation. *Proc. VLDB Endow.*, 12(4), 390–403. <https://doi.org/10.14778/3297753.3297759>
- Liu, M. X., Liu, F., Fiannaca, A. J., Koo, T., Dixon, L., Terry, M., & Cai, C. J. (2024). “We Need Structured Output”: Towards User-centered Constraints on Large Language Model Output. *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 1–9. <https://doi.org/10.1145/3613905.3650756>
- Liu, Y., Ding, J., Fu, Y., & Li, Y. (2023). UrbanKG: An Urban Knowledge Graph System. *ACM Transactions on Intelligent Systems and Technology*, 14(4), 60:1–60:25. <https://doi.org/10.1145/3588577>
- MALCZEWSKI, J., & JELOKHANI-NIARAKI, M. (2012). An ontology-based multicriteria spatial decision support system: A case study of house selection. *Geo-Spatial Information Science*, 15(3), 177–185. <https://doi.org/10.1080/10095020.2012.715900>
- Nguyen, V., Yip, H. Y., Thakkar, H., Li, Q., & Bodenreider, O. (2019). Singleton Property Graph: Adding A Semantic Web Abstraction Layer to Graph Databases. *The Joint International Workshop on Blockchain and Semantic Web, and Contextualized Knowledge Graphs (BlockSW-CKG 2019)*, 2599, 1–13. https://ceur-ws.org/Vol-2599/CKG2019_paper_4.pdf
- Ning, Y., & Liu, H. (2024). *UrbanKGent: A Unified Large Language Model Agent Framework for Urban Knowledge Graph Construction* (No. arXiv:2402.06861). arXiv. <http://arxiv.org/abs/2402.06861>
- Pan, J., Deng, Y., Yang, Y., & Zhang, Y. (2023). Location-allocation modelling for rational health planning: Applying a two-step optimization approach to evaluate the spatial accessibility improvement of newly added tertiary hospitals in a metropolitan city of China. *Social Science & Medicine*, 338, 116296. <https://doi.org/10.1016/j.socscimed.2023.116296>
- Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2024). Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7), 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>
- Park, S. (2023). Spatial Knowledge Graph for Analyzing Traffic Accident Data. *18th International Conference on Location Based Services*, 174–178. <https://doi.org/10.34726/5748>
- Patras, V., Laskas, P., Koritsoglou, K., Fudos, I., & Karvounis, E. (2021). A comparative evaluation of RDBMS and GDBMS for shortest path operations on pedestrian navigation data. *2021 6th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)*, 1–5. <https://doi.org/10.1109/SEEDA-CECNSM53056.2021.9566235>
- Procopiuc, O., Agarwal, P. K., Arge, L., & Vitter, J. S. (2003). Bkd-Tree: A

- Dynamic Scalable kd-Tree. In T. Hadzilacos, Y. Manolopoulos, J. Roddick, & Y. Theodoridis (Eds.), *Advances in Spatial and Temporal Databases* (pp. 46–65). Springer. https://doi.org/10.1007/978-3-540-45072-6_4
- Schmidt, M., Bebee, B., Broekema, W., Elzare, M., Enriquez, C. M. L., Neyman, M., Schmedding, F., Steigmiller, A., Thompson, B., Varkey, G., Williams, G. T., & Xiang, A. (2024). openCypher over RDF: Connecting Two Worlds. *Proceedings of the ISWC 2024 Posters, Demos and Industry Tracks: From Novel Ideas to Industrial Practice*, 3828. https://ceur-ws.org/Vol-3828/#ISWC2024_paper_29
- Sharma, G., & Patil, G. R. (2024). Urban spatial structure and equity for urban services through the lens of accessibility. *Transport Policy*, 146, 72–90. <https://doi.org/10.1016/j.tranpol.2023.10.017>
- Shortliffe, E. (2012). *Computer-Based Medical Consultations: MYCIN* (Vol. 2). Elsevier.
- Stadler, C., Lehmann, J., Höffner, K., & Auer, S. (2012). LinkedGeoData: A core for a web of spatial open data. *Semantic Web*, 3(4), 333–354. <https://doi.org/10.3233/SW-2011-0052>
- Suárez-Figueroa, M. C., Gómez-Pérez, A., & Fernández-López, M. (2012). The NeOn Methodology for Ontology Engineering. In M. C. Suárez-Figueroa, A. Gómez-Pérez, E. Motta, & A. Gangemi (Eds.), *Ontology Engineering in a Networked World* (pp. 9–34). Springer. https://doi.org/10.1007/978-3-642-24794-1_2
- Sun, L., & Jin, B. (2023). Improving NoSQL Spatial-Query Processing with Server-Side In-Memory R*-Tree Indexes for Spatial Vector Data. *Sustainability*, 15(3), Article 3. <https://doi.org/10.3390/su15032442>
- Sun, Y., & Sarwat, M. (2019). A spatially-pruned vertex expansion operator in the Neo4j graph database system. *GeoInformatica*, 23(3), 397–423. <https://doi.org/10.1007/s10707-019-00361-2>
- Sun, Y., & Sarwat, M. (2021). Riso-Tree: An Efficient and Scalable Index for Spatial Entities in Graph Database Management Systems. *ACM Transactions on Spatial Algorithms and Systems*, 7(3), 1–39. <https://doi.org/10.1145/3450945>
- Theocharidis, K., Liagouris, J., Mamoulis, N., Bouros, P., & Terrovitis, M. (2019). SRX: Efficient management of spatial RDF data. *The VLDB Journal*, 28(5), 703–733. <https://doi.org/10.1007/s00778-019-00554-z>
- Wang, C.-J., Ku, W.-S., & Chen, H. (2012). Geo-Store: A spatially-augmented SPARQL query evaluation system. *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*, 562–565. <https://doi.org/10.1145/2424321.2424416>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.
- Xu, B., Yang, A., Lin, J., Wang, Q., Zhou, C., Zhang, Y., & Mao, Z. (2023). *ExpertPrompting: Instructing Large Language Models to be Distinguished Experts* (No. arXiv:2305.14688). arXiv. <https://doi.org/10.48550/arXiv.2305.14688>
- Yamauchi, R., Sonoda, S., Sannai, A., & Kumagai, W. (2023). *LPML: LLM-*

Prompting Markup Language for Mathematical Reasoning (No. arXiv:2309.13078). arXiv. <https://doi.org/10.48550/arXiv.2309.13078>

Yang, H., Xiao, L., Zhu, R., Liu, Z., & Chen, J. (2024). An LLM supported approach to ontology and knowledge graph construction. *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 5240–5246. <https://doi.org/10.1109/BIBM62325.2024.10822222>

Ye, J., Jiang, Y., Chen, J., Liu, Z., & Guo, R. (2021). Joint optimisation of transfer location and capacity for a capacitated multimodal transport network with elastic demand: A bi-level programming model and paradoxes. *Transportation Research Part E: Logistics and Transportation Review*, 156, 102540. <https://doi.org/10.1016/j.tre.2021.102540>

Zhang, B., Carriero, V. A., Schreiberhuber, K., Tsaneva, S., González, L. S., Kim, J., & de Berardinis, J. (2024). *OntoChat: A Framework for Conversational Ontology Engineering using Language Models* (No. arXiv:2403.05921). arXiv. <http://arxiv.org/abs/2403.05921>

Zhang, B., & Soh, H. (2024). *Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction* (No. arXiv:2404.03868). arXiv. <https://doi.org/10.48550/arXiv.2404.03868>

Zhang, H., Wang, X., Pan, J., & Wang, H. (2024). *SAKA: An Intelligent Platform for Semi-automated Knowledge Graph Construction and Application* (No. arXiv:2410.08094). arXiv. <https://doi.org/10.48550/arXiv.2410.08094>

Zhu, J., Chong, H.-Y., Zhao, H., Wu, J., Tan, Y., & Xu, H. (2022). The Application of Graph in BIM/GIS Integration. *Buildings*, 12(12), Article 12. <https://doi.org/10.3390/buildings12122162>

Zhu, R., Shimizu, C., Stephen, S., Fisher, C. K., Thelen, T., Currier, K., Janowicz, K., Hitzler, P., Schildhauer, M., Li, W., Rehberger, D., Barua, A., Christou, A., Cai, L., Dalal, A., D’Onofrio, A., Eells, A., Faulk, M., Liu, Z., ... Zalewski, J. (2025). *The KnowWhereGraph: A Large-Scale Geo-Knowledge Graph for Interdisciplinary Knowledge Discovery and Geo-Enrichment* (No. arXiv:2502.13874). arXiv. <https://doi.org/10.48550/arXiv.2502.13874>

Zhu, Y., Wang, X., Chen, J., Qiao, S., Ou, Y., Yao, Y., Deng, S., Chen, H., & Zhang, N. (2024). LLMs for knowledge graph construction and reasoning: Recent capabilities and future opportunities. *World Wide Web*, 27(5), 58. <https://doi.org/10.1007/s11280-024-01297-w>

Abstract (in Korean)

본 논문은 레이블 속성 그래프(Label Property Graph, LPG) 환경에서의 도메인 특화 공간 지식 그래프(Spatial Knowledge Graph)를 효율적으로 구축하고 활용하기 위한 포괄적인 아키텍처, 다층형 공간 지식 그래프(Multi-layered Spatial Knowledge Graph, MSKG)를 제안한다. 기존의 공간 KG 구축은 많은 시간과 비용이 소요되는 수작업에 의존하고 공간 정보의 통합이 어려웠으며, LPG 환경에서 명시적인 온톨로지 구조가 부재하여 복잡한 의미 추론에 한계가 있었다. 이러한 문제를 해결하기 위해, 본 연구는 시맨틱 그래프(Semantic Graph), 데이터 그래프(Data Graph), 공간 인덱스 그래프(Spatial Index Graph)의 세 가지 핵심 레이어를 단일 그래프 데이터베이스 내에 유기적으로 통합하였다.

제안하는 MSKG 아키텍처는 다음의 4단계로 구축된다: (1) 거대 언어모델 (Large Language Model, LLM)과 전문가 협업을 통해 KG 요구사항을 분석하고 초기 LPG 온톨로지를 반자동으로 추출한다. (2) 추출된 초기 온톨로지를 기반으로 LLM-전문가 상호작용을 지속하여 온톨로지를 보완하고 검증 스키마와 추론 규칙을 생성하여 LPG 온톨로지를 완성한다. (3) 그래프 데이터베이스에 LPG 모델링 접근법 (Direct, Singleton, Statement)에 따라 시맨틱 및 데이터 그래프를 구축한다. (4) SGIR-Tree 공간 그래프 인덱싱을 통해 공간 인덱스 그래프를 구축하여 MSKG를 완성한다.

본 아키텍처의 적용 가능성을 검증하기 위해 서울시 데이터를 활용한 주택 탐색 공간 KG를 구축하고 성능을 평가하였다. 실험 결과, LPG 모델링 접근법에 따른 구축 용량 및 쿼리 성능 차이를 규명하여 실시간 상호작용이 필요한 도메인에는 Direct 모델, 메타데이터가 풍부한 도메인에는 Statement 모델, 오프라인 분석에는 Singleton 모델이 적합하다는 설계 지침을 도출하였다. 또한, SGIR-Tree는 공간 및 그래프 연산

의 통합 최적화를 통해 쿼리 속도를 상당히 향상시켰으며, 반자동 방법론은 도메인 요구사항에 대해 높은 KG 커버리지를 달성하였다.

본 연구는 주요 기여는 다음과 같다. 첫째, LPG 온톨로지를 정의하고 LPG 환경에서 온톨로지와 인스턴스를 체계적으로 통합하는 방법론을 정립하였다. 둘째, KG 구축의 전략적 태스크 분해와 LLM-전문가 상호작용을 통한 반자동 접근법으로 고품질 KG 구축이 가능함을 입증하였다. 셋째, 그래프 공간 인덱싱을 통해 공간 연산과 그래프 순회 간의 임피던스 불일치를 해소하였다. 넷째, 사전 계산된 공간 관계를 실시간 연산으로 대체하고 공간-의미 쿼리를 가능하게 하는 경량화된 공간 지식 그래프 구조를 제시하였다. 이러한 기여는 LPG 환경에서의 공간 지식 그래프 구축을 위한 도메인 독립적 방법론을 확립하며, 통합된 공간-의미 추론이 필요한 모든 도메인에 적용 가능한 검증된 설계 지침을 제공한다. 향후 연구 방향으로서는 실시간 서비스를 위한 중분 업데이트 메커니즘 개발, RDF 기반 시스템에 필적하는 네이티브 추론 기능 통합, 그리고 주택 탐색 사례 연구를 넘어선 교차 도메인 일반화 가능성 검증이 포함된다.

Appendix

Appendix A: LPG Model Query Implementations

[Table A-1] Comparative Query Implementations Across LPG Models

Cluster	Query	Direct Query	Singleton Query	Statement Query
C1	상권접근성이 좋은 아파트가 있나요?	MATCH (ac:ApartmentComplex) MATCH (ca:CommercialArea) WITH ac, point.distance(ac.centroid, ca.centroid) AS distance WHERE distance <= 500 WITH ac, min(distance) AS minDistance RETURN DISTINCT ac.name AS apartment, minDistance ORDER BY minDistance ASC	MATCH (ac:ApartmentComplex) MATCH (ca:CommercialArea) WITH ac, point.distance(ac.centroid, ca.centroid) AS distance WHERE distance <= 500 WITH ac, min(distance) AS minDistance RETURN DISTINCT ac.name AS apartment, minDistance ORDER BY minDistance ASC	MATCH (ac:ApartmentComplex) MATCH (ca:CommercialArea) WITH ac, point.distance(ac.centroid, ca.centroid) AS distance WHERE distance <= 500 WITH ac, min(distance) AS minDistance RETURN DISTINCT ac.name AS apartment, minDistance ORDER BY minDistance ASC
C2	화려한 생활환경을 제공하는 아파트가	MATCH (ac:ApartmentComplex) MATCH (ds:DepartmentStore) WHERE 'Cleaning Service' IN ac.concierge_services AND point.distance(ac.centroid, ds.centroid) <= 1000 AND exists((:SwimmingPool)- [:PART_OF*.2]->(ac)) AND exists((:Gym)- [:PART_OF*.2]->(ac)) RETURN DISTINCT ac.name AS apartment	MATCH (ac:ApartmentComplex) WHERE 'Cleaning Service' IN ac.concierge_services AND EXISTS { MATCH (ac)<- [r1:RANGE]-(:Relation {name:'PART_OF'})<- [d1:DOMAIN]- (cf:CommunityFacility) WHERE d1.uuid = r1.uuid AND EXISTS { MATCH (cf)<- [r2:RANGE]-(:Relation {name:'PART_OF'})<- [d2:DOMAIN]- (f:SwimmingPool) WHERE d2.uuid = r2.uuid } }	MATCH (ac:ApartmentComplex) WHERE 'Cleaning Service' IN ac.concierge_services AND EXISTS { MATCH (ds:DepartmentStore) WHERE point.distance(ac.centroid, ds.centroid) <= 1000 } CALL (ac) { WITH ac MATCH (rel:Relation {name:'PART_OF'}) MATCH (rel)- [:RELATION_OF]- >(s1:Statement)<-

	있 나 요?		<pre> AND EXISTS { MATCH (cf) <- [r3:RANGE] - (:Relation {name:'PART_OF'}) <- [d3:DOMAIN] - (f:Gym) WHERE d3.uuid = r3.uuid } } AND EXISTS { MATCH (ds:DepartmentStore) WHERE point.distance(ac.centroid, ds.centroid) <= 1000 } RETURN ac.name AS apartment; </pre>	<pre> [:RANGE_OF] - (ac) MATCH (cf:CommunityFacility) - [:DOMAIN_OF] ->(s1) CALL (cf, rel) { WITH cf, rel MATCH (rel) - [:RELATION_OF] - >(s:Statement) <- [:RANGE_OF] - (cf) MATCH (dom) - [:DOMAIN_OF] ->(s) WHERE dom:SwimmingPool OR dom:Gym WITH DISTINCT CASE WHEN dom:SwimmingPool THEN 'SP' ELSE 'GYM' END AS kind LIMIT 2 WITH collect(kind) AS kinds RETURN size(kinds) = 2 AS hasBoth } WITH hasBoth WHERE hasBoth RETURN 1 AS ok LIMIT 1 } RETURN DISTINCT ac.name AS apartment; </pre>
C3	월 세 가 저 렴 한 아 파 트 가 있 나 요?	<pre> MATCH (au:ApartmentUnit) - [:HAS_TRANSACTION] - >(t:Transaction) WHERE t.monthly_rental_fee <= 100 AND t.type = 'Monthly Rent' RETURN au.name, t.monthly_rental_fee AS monthlyRentalFee ORDER BY monthlyRentalFee ASC </pre>	<pre> MATCH (au:ApartmentUnit) - [d:DOMAIN] ->(rel:Relation {name: "HAS_TRANSACTION"}) - [r:RANGE] ->(t:Transaction) WHERE d.uuid = r.uuid AND t.monthly_rental_fee <= 100 AND t.type = 'Monthly Rent' RETURN au.name, t.monthly_rental_fee AS monthlyRentalFee ORDER BY monthlyRentalFee ASC </pre>	<pre> MATCH (au:ApartmentUnit) - [:DOMAIN_OF] - >(s:Statement) <- [:RANGE_OF] - (t:Transaction) WHERE EXISTS {(:Relation {name: "HAS_TRANSACTION"}) - [:RELATION_OF] ->(s)} AND t.monthly_rental_fee <= 100 AND t.type = 'Monthly Rent' RETURN au.name, t.monthly_rental_fee AS monthlyRentalFee ORDER BY monthlyRentalFee ASC </pre>
C4	고 급 스	<pre> MATCH (:CommunityFacility) - [:PART_OF] - >(ac:ApartmentComplex) WHERE </pre>	<pre> MATCH (ac:ApartmentComplex) WHERE ac.construction_company_type </pre>	<pre> MATCH (ac:ApartmentComplex) WHERE ac.construction_company_type </pre>

러 운 아 파 트 를 찾 고 있 습 니 다	<pre> ac.construction_company_type = 'Large Enterprise' AND 'Breakfast Service' IN ac.concierge_services AND ac.construction_year <= date() - duration({years:5}) WITH ac MATCH (r:Room) - [:PART_OF] - >(au:ApartmentUnit) - [p:PART_OF*..2]->(ac) WHERE au.exclusive_area >= 30 WITH ac, COUNT(r) AS roomCnt WHERE roomCnt >= 3 RETURN DISTINCT ac.name </pre>	<pre> = 'Large Enterprise' AND 'Breakfast Service' IN ac.concierge_services AND ac.construction_year <= date() - duration({years:5}) AND EXISTS { MATCH (ac)<- [r1:RANGE] - (:Relation {name:'PART_OF'})<- [d1:DOMAIN] - (:CommunityFacility) WHERE d1.uuid = r1.uuid } CALL (ac) { WITH ac MATCH (ac)<- [r2:RANGE] - (:Relation {name:'PART_OF'})<- [d2:DOMAIN] - (ab:ApartmentBuilding) WHERE d2.uuid = r2.uuid MATCH (ab)<- [r3:RANGE] - (:Relation {name:'PART_OF'})<- [d3:DOMAIN] - (au:ApartmentUnit) WHERE d3.uuid = r3.uuid AND au.exclusive_area >= 30 MATCH (au)<- [r4:RANGE] - (:Relation {name:'PART_OF'})<- [d4:DOMAIN] - (r:Room) WHERE d4.uuid = r4.uuid WITH DISTINCT r LIMIT 3 RETURN count(r) AS roomCnt } WITH ac, roomCnt WHERE roomCnt = 3 RETURN DISTINCT ac.name; </pre>	<pre> = 'Large Enterprise' AND 'Breakfast Service' IN ac.concierge_services AND ac.construction_year <= date() - duration({years:5}) AND EXISTS { MATCH (rel:Relation {name:'PART_OF'}) - [:RELATION_OF] - >(s1:Statement)<- [:RANGE_OF] - (ac) MATCH (:CommunityFacility) - [:DOMAIN_OF] ->(s1) } CALL (ac) { WITH ac MATCH (rel:Relation {name:'PART_OF'}) MATCH (rel) - [:RELATION_OF] - >(s_ab:Statement)<- [:RANGE_OF] - (ac) MATCH (ab:ApartmentBuilding) - [:DOMAIN_OF] ->(s_ab) MATCH (rel) - [:RELATION_OF] - >(s_au:Statement)<- [:RANGE_OF] - (ab) MATCH (au:ApartmentUnit) - [:DOMAIN_OF] ->(s_au) WHERE au.exclusive_area >= 30 MATCH (rel) - [:RELATION_OF] - >(s_r:Statement)<- [:RANGE_OF] - (au) MATCH (r:Room) - [:DOMAIN_OF] ->(s_r) WITH DISTINCT r LIMIT 3 RETURN count(r) AS roomCnt } WITH ac, roomCnt WHERE roomCnt = 3 RETURN DISTINCT ac.name AS apartment; </pre>
--	---	---	--

C5	경비시설이 있는 아파트가 있나요?	<pre> CALL () { CALL gspatial.rtree.query.join(["SecurityFacility","ApartmentComplex"], ["within"]) YIELD node1 AS sf, node2 AS ac WITH ac, sf WHERE sf.type = 'CCTV' RETURN DISTINCT ac } MATCH (ab:ApartmentBuilding)-[:PART_OF]->(ac)<-[:PART_OF]-(mo:ManagementOffice) WHERE ab.building_entrance_security_lock = true AND mo.security_guard >= 1 RETURN DISTINCT ac.name AS apartmentComplex </pre>	<pre> CALL () { CALL gspatial.rtree.query.join(["SecurityFacility","ApartmentComplex"], ["within"]) YIELD node1 AS sf, node2 AS ac WITH ac, sf WHERE sf.type = 'CCTV' RETURN DISTINCT ac } WITH ac WHERE EXISTS { MATCH (ac)<-[r1:RANGE]-(:Relation {name:'PART_OF'})<-[d1:DOMAIN]-(ab:ApartmentBuilding) WHERE d1.uuid = r1.uuid AND ab.building_entrance_security_lock = true } AND EXISTS { MATCH (ac)<-[r2:RANGE]-(:Relation {name:'PART_OF'})<-[d2:DOMAIN]-(mo:ManagementOffice) WHERE d2.uuid = r2.uuid AND mo.security_guard >= 1 } RETURN DISTINCT ac.name AS apartmentComplex </pre>	<pre> CALL () { CALL gspatial.rtree.query.join(["SecurityFacility","ApartmentComplex"], ["within"]) YIELD node1 AS sf, node2 AS ac WITH ac, sf WHERE sf.type = 'CCTV' RETURN DISTINCT ac } WITH ac MATCH (rel:Relation {name:'PART_OF'}) WHERE EXISTS { MATCH (rel)-[:RELATION_OF]->(s_ab:Statement)<-[:RANGE_OF]-(ac) MATCH (ab:ApartmentBuilding {building_entrance_security_lock: true})-[:DOMAIN_OF]->(s_ab) } AND EXISTS { MATCH (rel)-[:RELATION_OF]->(s_mo:Statement)<-[:RANGE_OF]-(ac) MATCH (mo:ManagementOffice)-[:DOMAIN_OF]->(s_mo) WHERE mo.security_guard >= 1 } RETURN DISTINCT ac.name AS apartmentComplex; </pre>
----	--------------------	--	--	---

Note: The table demonstrates how the same semantic query requires increasingly complex structures when moving from Direct to Singleton models, directly impacting performance.

Appendix B: Use Case Queries

[Table B-1] Multi-Layer Structure Queries

Cluster	NL Query	Cypher Query
C1	서울대학교 35동 근처에 있는 아파트들과 아파트 주변환경을 알고 싶습니다	<pre> MATCH (ac:ApartmentComplex) -[:INSTANCE_OF]-> (c_ac:Concept) WITH DISTINCT c_ac MATCH (c_ac) -[:SUBCLASS_OF]-> (c_apt) -[:PROXIMITY]-> (ne) <-[:SUBCLASS_OF*1..5] - (neighbor_concept) WHERE neighbor_concept.lpg_schema = "label" WITH collect(DISTINCT apoc.text.upperCamelCase(neighbor_concept.name)) AS neighbor_labels, [126.9521161822409852,37.4565889438855493] as dong_35 CALL gspatial.rtree.query.range(["ApartmentComplex"], [dong_35, 0.045]) YIELD node AS ac CALL (ac, neighbor_labels) { WITH ac, neighbor_labels CALL gspatial.rtree.query.knn(neighbor_labels, [[ac.centroid.x, ac.centroid.y], 3]) YIELD node AS neighbor_node, distance RETURN collect(neighbor_node.name) AS neighbor_names } RETURN ac.name, neighbor_names </pre>
C2	청소년 자녀를 키우기 좋은 아파트를 알려주세요	<pre> MATCH (edu_type:Concept) -[:SUBCLASS_OF*1..3]-> (edu_concept:Concept {name: 'Educational Facility'}) WHERE edu_type.lpg_schema = 'label' AND edu_type.name <> "School District" WITH collect(DISTINCT apoc.text.upperCamelCase(edu_type.name)) AS education_labels MATCH (r:Room) -[:PART_OF]-> (au:ApartmentUnit) -[:PART_OF*1..2]-> (ac:ApartmentComplex) WHERE au.supply_area >= 85 WITH DISTINCT ac, education_labels, count(r) AS room_count WHERE room_count >= 3 CALL (ac, education_labels) { WITH ac, education_labels CALL gspatial.rtree.query.knn(education_labels, [[ac.centroid.x, ac.centroid.y], 5]) YIELD node AS edu_node, distance RETURN count(DISTINCT labels(edu_node)[0]) AS edu_diversity, avg(distance) AS avg_edu_distance } WITH ac, edu_diversity, avg_edu_distance WHERE edu_diversity >= 3 AND avg_edu_distance <= 500 RETURN DISTINCT ac.name AS apartmentComplex, </pre>

		edu_diversity AS educationDiversity, round(avg_edu_distance) AS avgEducationDistance;
C3	최근에 월세로 거래되었고 대중교통이 편리하며 아파트가 있나요?	MATCH (transport_concept:Concept {name: 'Public Transportation'}) MATCH (transport_type:Concept) -[:SUBCLASS_OF]->(transport_concept) WITH collect(DISTINCT apoc.text.upperCamelCase(transport_type.name)) AS transport_labels MATCH (ac:ApartmentComplex) <-[:PART_OF*1..2]- (au:ApartmentUnit) - [:HAS_TRANSACTION]->(t:Transaction) WHERE t.type = 'Monthly Rent' AND t.monthly_rental_fee <= 50 AND t.security_deposit <= 3000 AND t.date >= date() - duration({months: 6}) CALL (ac, transport_labels) { WITH DISTINCT ac, transport_labels CALL gspatial.rtree.query.knn(transport_labels, [[ac.centroid.x, ac.centroid.y], 2]) YIELD node AS transport_node, distance RETURN min(distance) AS nearest_transport, collect({type: labels(transport_node)[0], name: transport_node.name, distance: round(distance)}) [0] AS closest_transport } WITH ac, au, t, nearest_transport, closest_transport, au.exclusive_area / (t.monthly_rental_fee + t.security_deposit * 0.01) AS area_per_cost WHERE nearest_transport <= 500 RETURN au.name AS apartmentUnit, round(au.exclusive_area, 1) AS exclusiveArea, t.monthly_rental_fee AS monthlyRent, t.security_deposit AS deposit, round(area_per_cost, 2) AS areaPerCost, round(nearest_transport) AS nearestTransport, closest_transport.type AS transportType, closest_transport.name AS transportName
C4	다양한 커뮤니티 시설을 가지고 있는 아파트들이 많은 동네는 어디인가요?	MATCH (cf_concept:Concept {name: 'Community Facility'}) MATCH (cf_type:Concept) -[:PART_OF*1..3]->(cf_concept) WHERE cf_type.lpg_schema = "label" WITH count(DISTINCT cf_type) AS total_facility_types MATCH (facility) -[:PART_OF]->(:CommunityFacility) -[:PART_OF]- >(ac:ApartmentComplex) WITH DISTINCT ac, count(DISTINCT labels(facility)) as facility_count, total_facility_types WHERE facility_count >= total_facility_types / 2 CALL (ac) { WITH ac CALL gspatial.rtree.query.join(["ApartmentComplex", "AdministrativeDistrictDong"], ["within"]) YIELD node1 AS ac_node, node2 AS dong WHERE ac_node = ac RETURN dong.name AS dong_name } RETURN ac.name, facility_count, dong_name
C5	보안이 좋은 아파트가	CALL gspatial.rtree.query.join(["SecurityFacility", "ApartmentComplex"],

	있나요?	<pre> ["within"]) YIELD node1 AS security_node, node2 AS ac WITH ac, count(security_node) AS internal_security_count WHERE internal_security_count >= 10 MATCH (related_labels:Concept) - [:HAS_PROPERTY] - (security_type:Concept) - [:PART_OF] -> (c_sf:Concept {name: 'Security Facility'}) WITH ac, internal_security_count, collect(DISTINCT toLower(replace(security_type.name, ' ', '_'))) AS security_properties, collect(DISTINCT related_labels) AS related_label_list UNWIND related_label_list AS related_label MATCH (related_label) <- [:INSTANCE_OF] - (instance) - [:PART_OF] -> (ac) WHERE any(k IN security_properties WHERE coalesce(instance[k], false) = true OR coalesce(instance[k], false) >= 1) RETURN DISTINCT ac.name AS apartmentComplex, internal_security_count ORDER BY internal_security_count DESC </pre>
--	------	---

[Table B- 2] Comparison of CQs: This Dissertation vs. Hong (2025)

Category	This Dissertation CQs	Hong (2025) Representative Sentences	Overlap
Safety/Floor	CQ-5: 보안 시설이 완비된 아파트가 있나요? CQ-38: 경비 시설이 있는 아파트가 있나요? CQ-95: 특정 층수에 위치한 아파트가 있나요?	Cluster-9: 층수는 지하나 1 층이 아니었으면 좋겠습니다	●
Financial/Loan	CQ-11: 특정 예산 내에서 구매할 수 있는 아파트가 있나요? CQ-29: 월세가 저렴한 아파트가 있나요? CQ-129: 신혼부부 주거 지원 혜택이 가능한 아파트가 있나요?	Cluster-2: 대출 포함 최대 3 억이었으면 좋겠습니다 Cluster-8: 보증금 최대 7000 까지 가능한데 최대한 월세 낮추고 싶습니다 Cluster-13: 중기청 대출이 가능한 집으로 알아보고 있습니다 Cluster-15: 혹은 은행권 전세 대출 70 프로	●
Pet-Friendly	CQ-74: 애완동물 친화적인 아파트가 있나요?	Cluster-10: 강아지와 고양이 있는데 같이 기를 수 있는 집이면 좋겠어요	●
Transportation	CQ-16: 대중교통 접근성이 좋은 아파트가 있나요? CQ-26: 역세권에 위치한 아파트가 있나요? CQ-62: 버스 정류장 근처에 위치한 아파트가 있나요?	Cluster-5: 역과 가까우면 좋을 것 같습니다 Cluster-1: 부산 문현역 인근 방구합니다 Cluster-14: 그 주변에 정없다면	●

		버스로 생활권 편한곳으로 찾고있습니다	
Parking	CQ-49: 넓은 주차장이 있는 아파트가 있나요?	Cluster-7: 주차는 할 수 있으면 좋구여	
Terrace Garden /	CQ-9: 작은 정원이 있는 아파트가 있나요? CQ-50: 넓은 테라스가 있는 아파트가 있나요? CQ-59: 루프탑 테라스가 있는 아파트가 있나요?	Cluster-12: 2 순위 빌라형 테라스 마당	
Childcare	CQ-7: 아이 키우기 좋은 환경의 아파트가 있나요? CQ-138: 유치원 접근성이 좋은 아파트가 있나요?	Cluster-4: 애기가있어서 어린이집이 근처에있었으면 좋겠습니다	
Room Structure	CQ-48: 주방의 크기가 큰가요? CQ-76: 주방이 오픈형인가요? CQ-118: 방의 개수가 특정 개수 이상인 아파트가 있나요?	Cluster-0: 공간분리가 잘 되어있으면 1.5 룸도 괜찮아요 투룸 선호하긴 합니다 Cluster-3: 화장실에 샤워부스나 샤워공간이 나누어져 있었으면 좋겠습니다	●
Building Age	CQ-24: 신축 아파트가 있나요? CQ-89: 최신 리모델링된 아파트가 있나요?	Cluster-17: 깔끔한 신축이면 너무 좋을 거 같습니다	●
Size/Area	CQ-100: 특정크기 이상의 아파트가 있나요?	Cluster-6: 실坪수 9 평이상을 찾고 있습니다 Cluster-11: 방 크기가 비슷했음 좋겠고	●
Natural Light/ View	CQ-86: 채광이 좋은 아파트가 있나요? CQ-111: 남향인 아파트가 있나요? CQ-12: 강 뷰를 가진 아파트가 있나요?	Cluster-19: 창이 크고 옆 건물에 가리지 않았으면 좋겠어요	●
Facilities / Options	CQ-67: 스마트홈 시스템이 갖춰진 아파트가 있나요?	Cluster-16: 옵션은 가전만 있어도 됨	●
Location	CQ-96: 특정 행정구역에 속한 아파트가 있나요? CQ-115: 동네에 속한 아파트가 있나요?	-	○
Education (Schools)	CQ-15: 국제 학교 도보 거리 내에 위치한 아파트가 있나요? CQ-84: 좋은 학군에 위치한 아파트가 있나요?	-	○

	CQ-88: 초등학교 도보 거리 내에 위치한 아파트가 있나요?		
Community / Amenities	CQ-18: 대형 쇼핑몰이 인접한 아파트가 있나요? CQ-28: 운동 시설을 내부시설로 갖춘 아파트가 있나요? CQ-40: 커뮤니티 시설이 있는 아파트가 있나요?	—	○
Environment / Atmosphere	CQ-10: 조용한 주거 환경을 제공하는 아파트가 있나요? CQ-54: 도보 가능한 산책로가 있는 아파트가 있나요?	—	○
Medical Facility	CQ-20: 병원 근처인 아파트가 있나요? CQ-120: 보건소 근처인 아파트가 있나요?	—	○
Investment	CQ-34: 환금성이 높은 아파트가 있나요? CQ-92: 투자 가치가 높은 아파트가 있나요?	—	○

Note: ● = Substantial overlap; ○ = This dissertation only (not represented in Hong's 20 clusters). Hong's CQs were derived from 17,909 user queries through TCL clustering, while this dissertation's 151 CQs were derived through LLM-assisted specification analysis with expert refinement.

[Table B- 3] Real World User Queries

Query	NL Query	Cypher Query
1	10월내로 입주가능 오픈형 원룸이면 9평 이상 주차 가능하면 좋으나 팔생각도 있음 고양이 키울수있으면 좋지만 포기가능 예산 월세 8,000만원/70만 원 이사날짜 2개월이내	<pre> CALL () { MATCH (dong:AdministrativeDistrictDong {name: '문정동'}) CALL gspatial.rtree.query.join(["ApartmentComplex", "AdministrativeDistrictDong"], ["within"]) YIELD node1 AS ac, node2 AS dong_filtered WHERE dong_filtered = dong RETURN ac } WITH ac MATCH (ac)-[:PART_OF*..2]-(au:ApartmentUnit)-[:HAS_TRANSACTION]->(t:Transaction) OPTIONAL MATCH (r:Room)-[:PART_OF]->(au) WITH ac, au, t, count(r) AS roomCount WHERE t.security_deposit <= 8000 </pre>

	구조 분리형 원룸, 투룸 선호하는 지역 서울특별시 송파구 문정 주변시설 옵션 # 최대한 빠른입주	<pre> AND t.monthly_rental_fee <= 70 AND t.type = 'Monthly Rent' AND ((roomCount = 1 AND au.exclusive_area >= 30) OR roomCount = 2) OPTIONAL MATCH (ac) -[:HAS_RESTRICT] -> (mic:MoveInConditions) WITH ac, au, t, roomCount, mic, CASE WHEN ac.parking_ratio_per_household >= 0.5 THEN 1 ELSE 0 END AS parking_score, CASE WHEN mic.pet_policy = true THEN 1 ELSE 0 END AS pet_score RETURN DISTINCT ac.name AS apartmentComplex, au.name AS apartmentUnit, roomCount AS rooms, au.exclusive_area AS exclusiveArea, au.supply_area AS supplyArea, t.security_deposit AS deposit, t.monthly_rental_fee AS monthlyRent, ac.parking_ratio_per_household AS parkingRatio, mic.pet_policy AS petAllowed, (parking_score + pet_score) AS preferenceScore ORDER BY preferenceScore DESC, monthlyRent ASC LIMIT 20 </pre>
2	# 최대한 빠른입주 # 주차장 # 넓은 구축 예산 월세 500만원/60만원 이사날짜 2개월이내 구조 투룸 선호하는 지역	<pre> CALL () { MATCH (dong:AdministrativeDistrictDong {name: '우이동'}) CALL gspatial.rtree.query.join(["ApartmentComplex", "AdministrativeDistrictDong"], ["within"]) YIELD node1 AS ac, node2 AS dong_filtered WHERE dong_filtered = dong RETURN ac } WITH ac MATCH (ac) <-[:PART_OF*,..2]-(au:ApartmentUnit)-[:HAS_TRANSACTION]->(t:Transaction) OPTIONAL MATCH (r:Room)-[:PART_OF]->(au) WITH ac, au, t, count(r) AS roomCount WHERE </pre>

	서울특별시 강북구 우이 주변시설 옵션 # 최대한 빠른입주# 주차장# 넓은 구축	<pre> t.security_deposit <= 500 AND t.monthly_rental_fee <= 60 AND t.type = 'Monthly Rent' AND roomCount = 2 AND ac.construction_year <= date('2005-01-01') WITH ac, au, t, roomCount, CASE WHEN au.exclusive_area >= 50 THEN 2 // WHEN au.exclusive_area >= 40 THEN 1 // ELSE 0 END AS space_score, CASE WHEN ac.parking_ratio_per_household >= 1.0 THEN 2 WHEN ac.parking_ratio_per_household >= 0.5 THEN 1 ELSE 0 END AS parking_score RETURN DISTINCT ac.name AS apartmentComplex, au.name AS apartmentUnit, roomCount AS rooms, au.exclusive_area AS exclusiveArea, au.supply_area AS supplyArea, t.security_deposit AS deposit, t.monthly_rental_fee AS monthlyRent, ac.parking_ratio_per_household AS parkingRatio, (space_score + parking_score) AS preferenceScore ORDER BY preferenceScore DESC, exclusiveArea DESC LIMIT 20 </pre>
3	1. 최대한 동작구 인접 희망 2. 지상층 3. 사용승인일 10년 이내 예산 전세 1억 2,000만원 이사날짜 2개월이내 구조 오픈형 원룸, 분리형 원룸	<pre> CALL () { MATCH (dong:AdministrativeDistrictDong) WHERE dong.name IN ['봉천동', '봉천1동', '봉천2동'] CALL gspatial.rtree.query.join(["ApartmentComplex", "AdministrativeDistrictDong"], ["within"]) YIELD node1 AS ac, node2 AS dong_filtered WHERE dong_filtered = dong RETURN ac } WITH ac MATCH (ac)-[:PART_OF*..2]-(au:ApartmentUnit)-[:HAS_TRANSACTION]->(t:Transaction) OPTIONAL MATCH (r:Room)-[:PART_OF]->(au) WITH ac, au, t, count(r) AS roomCount WHERE t.security_deposit <= 20000 AND t.type = 'Jeonse' AND roomCount = 1 </pre>

선호하는 지역 서울특별시 관악구 봉천 주변시설 옵션 # 작지만 깔끔함# 7 호선라인 선호# 2호선라인 선호 전세대출 옵션 LH전세대출	<pre> AND au.floor >= 1 AND (ac.construction_year >= date('2015-01-01') OR ac.remodeling_period >= date('2015-01-01')) WITH ac, au, t, roomCount OPTIONAL MATCH (subway:SubwayStation) WHERE '7호선' IN subway.subway_line OR '2호선' IN subway.subway_line WITH ac, au, t, roomCount, min(point.distance(ac.centroid, subway.centroid)) AS subwayDistance WITH ac, au, t, roomCount, subwayDistance OPTIONAL MATCH (dongjak:AdministrativeDistrictGu {name: '동작구'}) WITH ac, au, t, roomCount, subwayDistance, coalesce(point.distance(ac.centroid, dongjak.centroid), 99999) AS dongjak_distance WITH ac, au, t, roomCount, subwayDistance, dongjak_distance, CASE WHEN dongjak_distance <= 500 THEN 3 WHEN dongjak_distance <= 1000 THEN 2 WHEN dongjak_distance <= 2000 THEN 1 ELSE 0 END AS proximity_score, CASE WHEN subwayDistance <= 500 THEN 3 WHEN subwayDistance <= 800 THEN 2 WHEN subwayDistance <= 1000 THEN 1 ELSE 0 END AS subway_score, CASE WHEN ac.construction_year >= date('2020-01-01') AND au.exclusive_area <= 30 THEN 2 WHEN ac.remodeling_period >= date('2020-01-01') AND au.exclusive_area <= 30 THEN 2 WHEN ac.construction_year >= date('2018-01-01') AND au.exclusive_area <= 35 THEN 1 WHEN ac.remodeling_period >= date('2018-01-01') AND au.exclusive_area <= 35 THEN 1 ELSE 0 END AS clean_compact_score RETURN DISTINCT ac.name AS apartmentComplex, au.name AS apartmentUnit, roomCount AS rooms, au.floor AS floor, au.exclusive_area AS exclusiveArea, au.supply_area AS supplyArea, t.security_deposit AS jeonsePrice, ac.construction_year AS constructionYear, </pre>
---	---

		ac.remodeling_period AS remodelingPeriod, round(subwayDistance) AS subwayDistance, round(dongjak_distance) AS dongjakDistance, (proximity_score + subway_score + clean_compact_score) AS preferenceScore ORDER BY preferenceScore DESC, jeonsePrice ASC LIMIT 20
--	--	--